Desenvolvimento de Serviços com Spring Boot

Professor Ricardo Frohlich da Silva

Maven

- Maven é uma ferramenta de automação de construção de projetos amplamente utilizada na comunidade de desenvolvimento de software Java.

- Ele fornece uma maneira eficiente de gerenciar a construção, relatórios e documentação de projetos, além de facilitar a gestão de dependências.

- Aqui estão os principais componentes e conceitos do Maven:

  - POM (Project Object Model): No coração do Maven está o arquivo POM, que é um arquivo XML que descreve o projeto e suas configurações. O POM contém informações como versões de dependências, plugins a serem usados, configurações de compilação, testes e empacotamento do projeto.

  - Convenção sobre Configuração: Maven segue o princípio de "convenção sobre configuração", o que significa que muitas configurações padrão são assumidas com base em convenções estabelecidas. Isso reduz a necessidade de configuração manual e facilita o uso do Maven em projetos comuns.

  - Gestão de Dependências: Uma das características mais poderosas do Maven é sua capacidade de gerenciar as dependências do projeto de forma automática. Você pode especificar as dependências do seu projeto no arquivo POM e o Maven cuidará de baixar as bibliotecas necessárias e suas transições de forma transparente.

Maven

- Aqui estão os principais componentes e conceitos do Maven: (continuação)

  - Ciclo de Vida de Construção: Maven define um ciclo de vida de construção padrão, que consiste em fases como compilação, teste, empacotamento e implantação. Cada fase do ciclo de vida é associada a um conjunto de objetivos predefinidos, que são executados em uma determinada ordem.

  - Plugins: Maven é altamente extensível através de plugins. Existem plugins para realizar uma variedade de tarefas, como compilar código, executar testes, gerar relatórios, empacotar artefatos, publicar em repositórios e muito mais. Os plugins podem ser configurados no arquivo POM e ativados conforme necessário.

  - Repositórios Maven: Maven faz uso de repositórios remotos e locais para armazenar e recuperar dependências e artefatos. Existem repositórios públicos amplamente utilizados, como o Maven Central, que contém milhares de bibliotecas Java prontas para uso.

- Em resumo, o Maven simplifica o processo de construção e gerenciamento de projetos Java, fornecendo uma estrutura consistente e automatizada para lidar com dependências, compilação, teste e distribuição de software. Ele promove boas práticas de desenvolvimento, como a reutilização de componentes e a padronização da estrutura do projeto

Gradle

- O Gradle é uma ferramenta de automação de construção de projetos que oferece uma abordagem flexível e poderosa para o gerenciamento de projetos e construção de software.

- Ele foi desenvolvido para simplificar o processo de construção, permitindo que os desenvolvedores descrevam a estrutura do projeto e as dependências de forma declarativa, usando uma DSL (Domain-Specific Language) baseada em Groovy ou Kotlin.

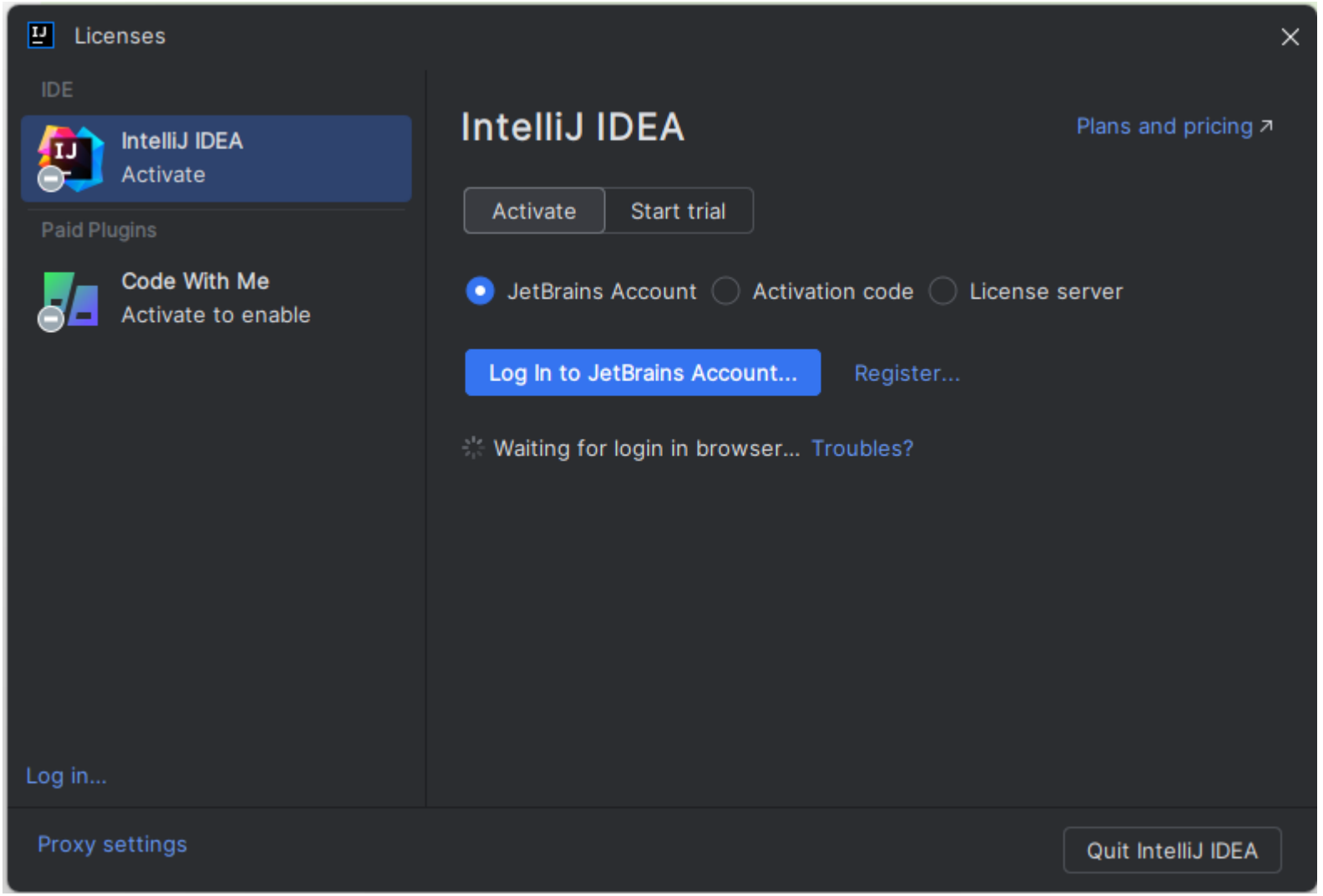
Maven x Gradle

- A escolha entre Maven e Gradle depende de várias considerações, incluindo as necessidades específicas do projeto, preferências da equipe de desenvolvimento e familiaridade com as ferramentas.
  - Se o projeto é grande e complexo, com requisitos personalizados de construção, o Gradle pode ser mais adequado devido à sua flexibilidade e capacidade de lidar com cenários complexos de construção. Por outro lado, se o projeto é mais simples e segue as convenções do Maven, o Maven pode ser uma escolha mais direta.
  - Se você prefere uma abordagem mais "opinionada" e baseada em convenções, o Maven pode ser a escolha certa, pois possui uma estrutura mais rígida e orientada por padrões. Por outro lado, se você prefere uma abordagem mais flexível e personalizável, o Gradle oferece mais liberdade para configurar o projeto de acordo com suas necessidades específicas.

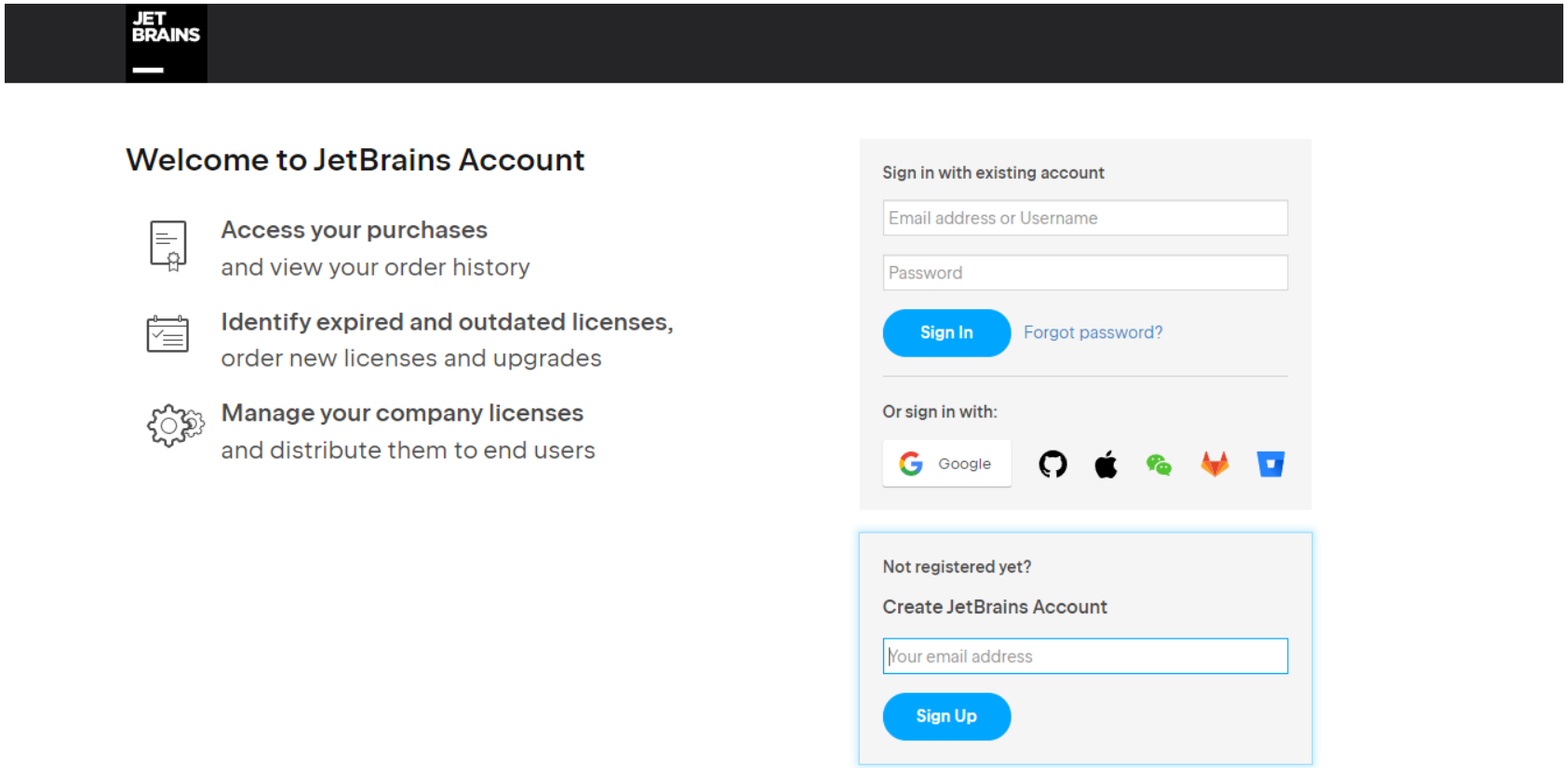
Maven x Gradle

- A escolha entre Maven e Gradle depende de várias considerações, incluindo as necessidades específicas do projeto, preferências da equipe de desenvolvimento e familiaridade com as ferramentas. (Continuação)
  - O Gradle é geralmente mais rápido do que o Maven, especialmente em projetos grandes, devido à sua execução incremental e capacidade de paralelização. Se o desempenho é uma consideração importante para o seu projeto, o Gradle pode ser uma escolha preferencial.
  - Maven tem uma vasta base de usuários e uma grande quantidade de plugins disponíveis, tornando mais fácil encontrar soluções para problemas comuns e integrar-se a outras ferramentas. Se o suporte da comunidade e a disponibilidade de plugins são importantes para você, o Maven pode ser mais atraente nesse aspecto.

IntelliJ


Licenses
IDE
IntelliJ IDEA
Activate
Paid Plugins
Code With Me
Activate to enable
Log in...
IntelliJ IDEA
Plans and pricing ↗
Activate
Start trial
JetBrains Account
Activation code
License server
Log In to JetBrains Account...
Register...
Waiting for login in browser... Troubles?
Proxy settings
Quit IntelliJ IDEA

IntelliJ


JET BRAINS
Welcome to JetBrains Account
Access your purchases
and view your order history
Identify expired and outdated licenses,
order new licenses and upgrades
Manage your company licenses
and distribute them to end users
Sign in with existing account
Email address or Username
Password
Sign In
Forgot password?
Or sign in with:
Google
Not registered yet?
Create JetBrains Account
Your email address
Sign Up

IntelliJ

Ricardo Frohlich da Silva

Notifications

Two-factor authentication (2FA) is available!
To enable an extra layer of security for your JetBrains Account, turn on two-factor auth.

No Available Licenses

We found no JetBrains product licenses associated with your JetBrains Account. You can:

- Buy new personal license
- Contact the person who manages commercial licenses in your company and request an invitation to use them
- Apply for a free student or teacher license for educational purposes

Your JetBrains Account is a single interaction point for activating JetBrains products and accessing the following services:

- JetBrains Account website (you are here)
- Products Support
- Product Blogs
- Plugin Repository for .NET products (e.g. Resharper)
- Plugin Repository for other IDEs (e.g. IntelliJ IDEA, WebStorm and so on)

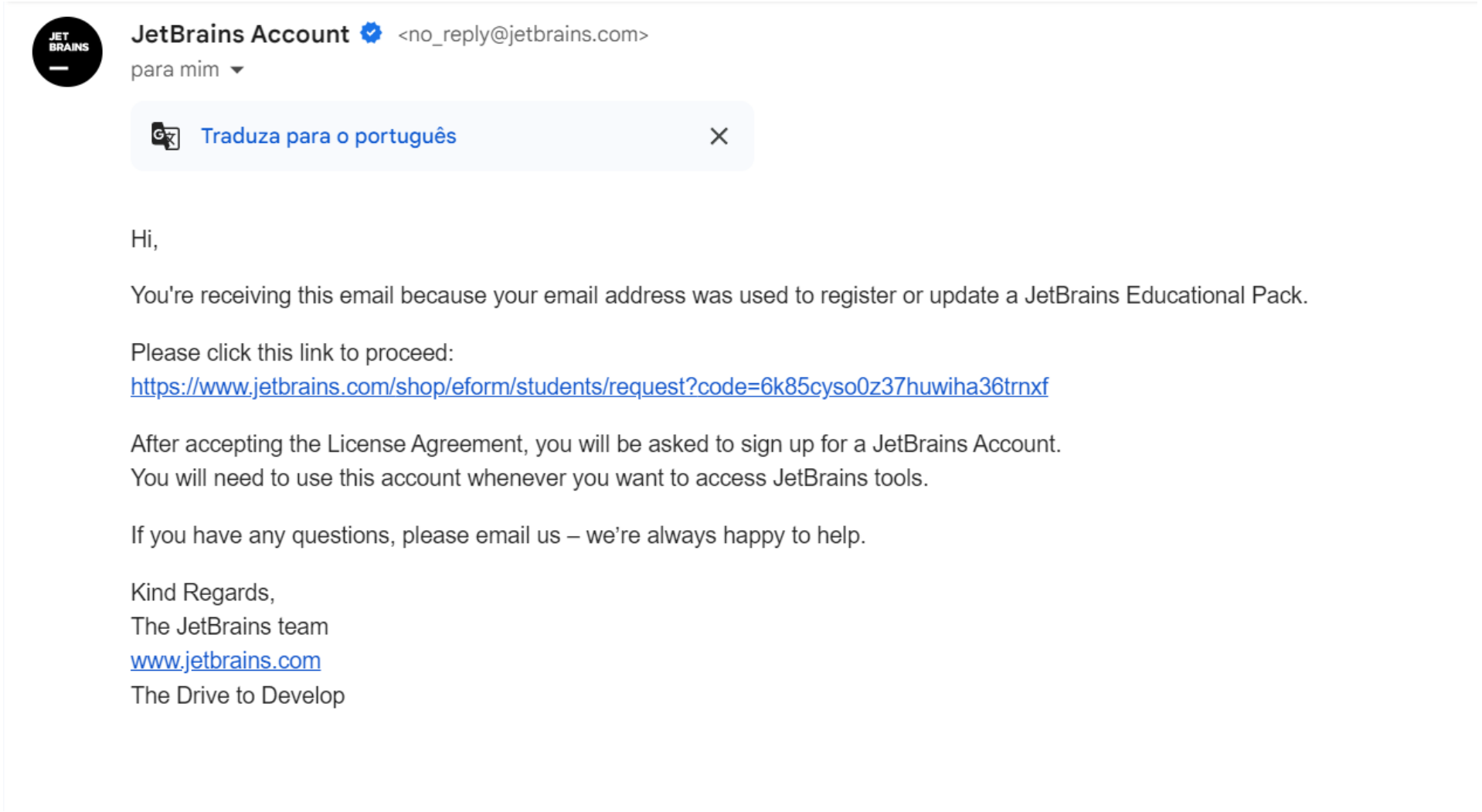
IntelliJ



Get free access to all developer tools from JetBrains!

Apply now

IntelliJ



JetBrains Account <no_reply@jetbrains.com>
para mim

Traduza para o português

Hi,

You're receiving this email because your email address was used to register or update a JetBrains Educational Pack.

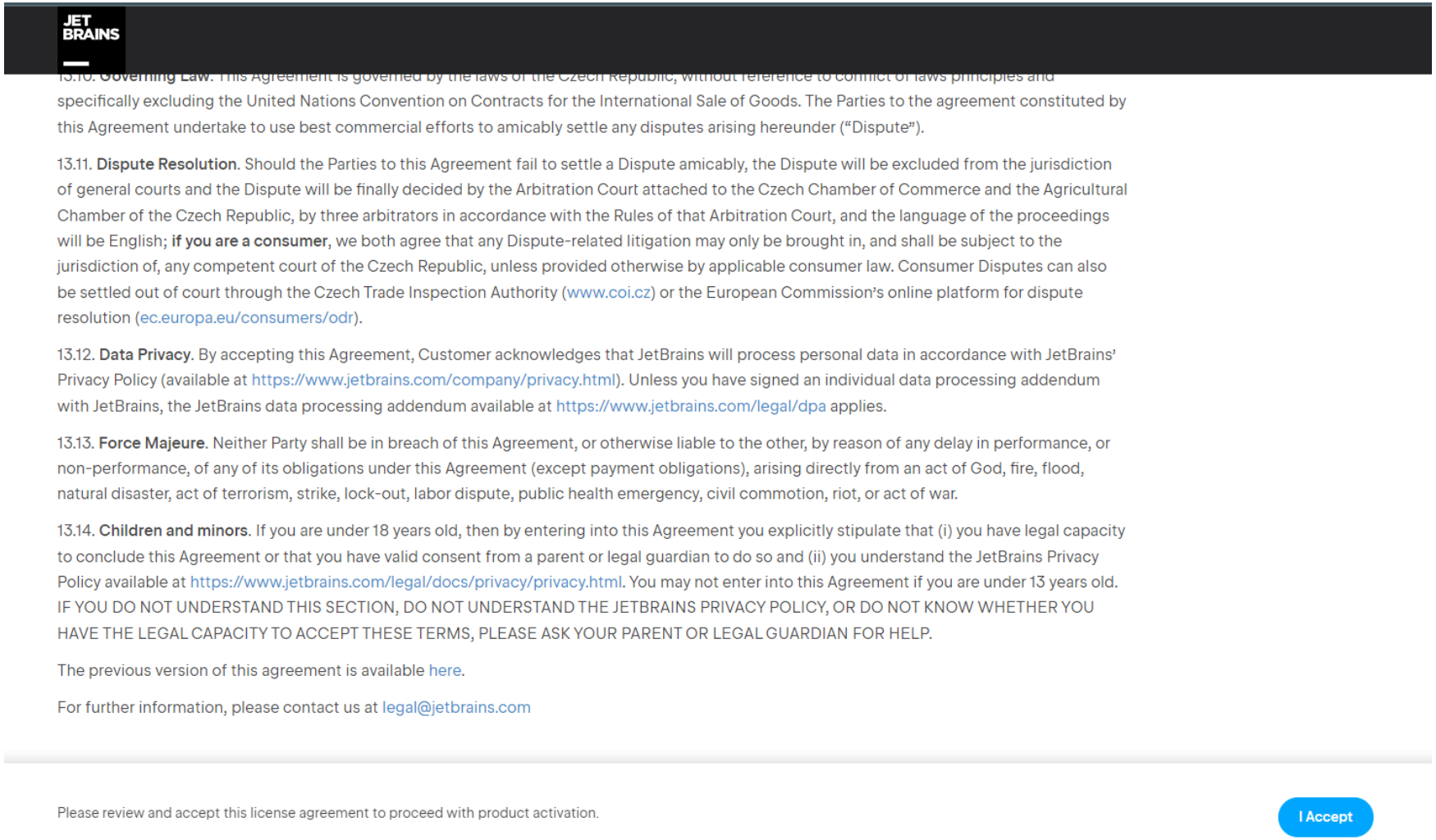
Please click this link to proceed:
https://www.jetbrains.com/shop/eform/students/request?code=6k85cyso0z37huwiha36trnxf

After accepting the License Agreement, you will be asked to sign up for a JetBrains Account.
You will need to use this account whenever you want to access JetBrains tools.

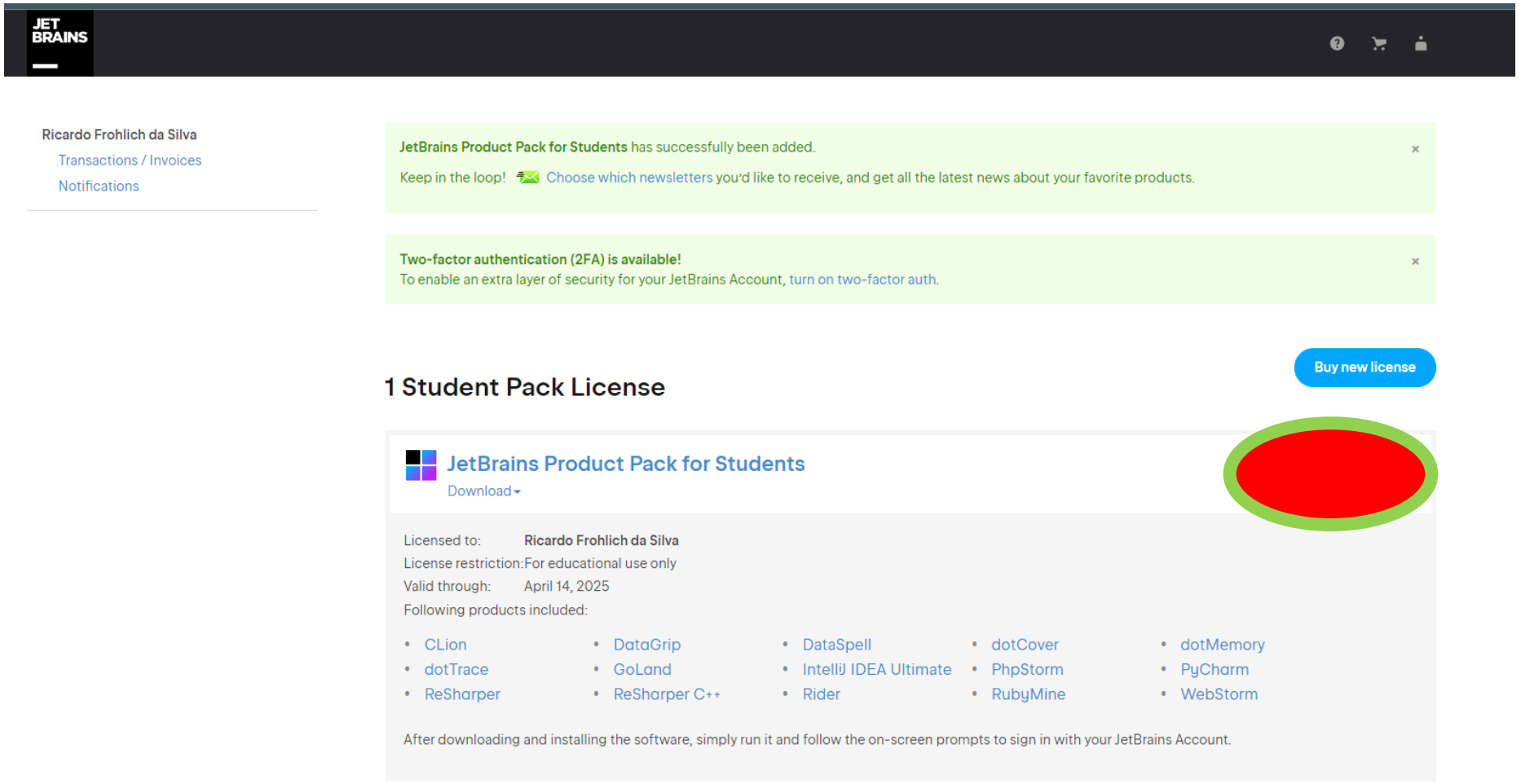
If you have any questions, please email us – we're always happy to help.

Kind Regards,
The JetBrains team
www.jetbrains.com
The Drive to Develop

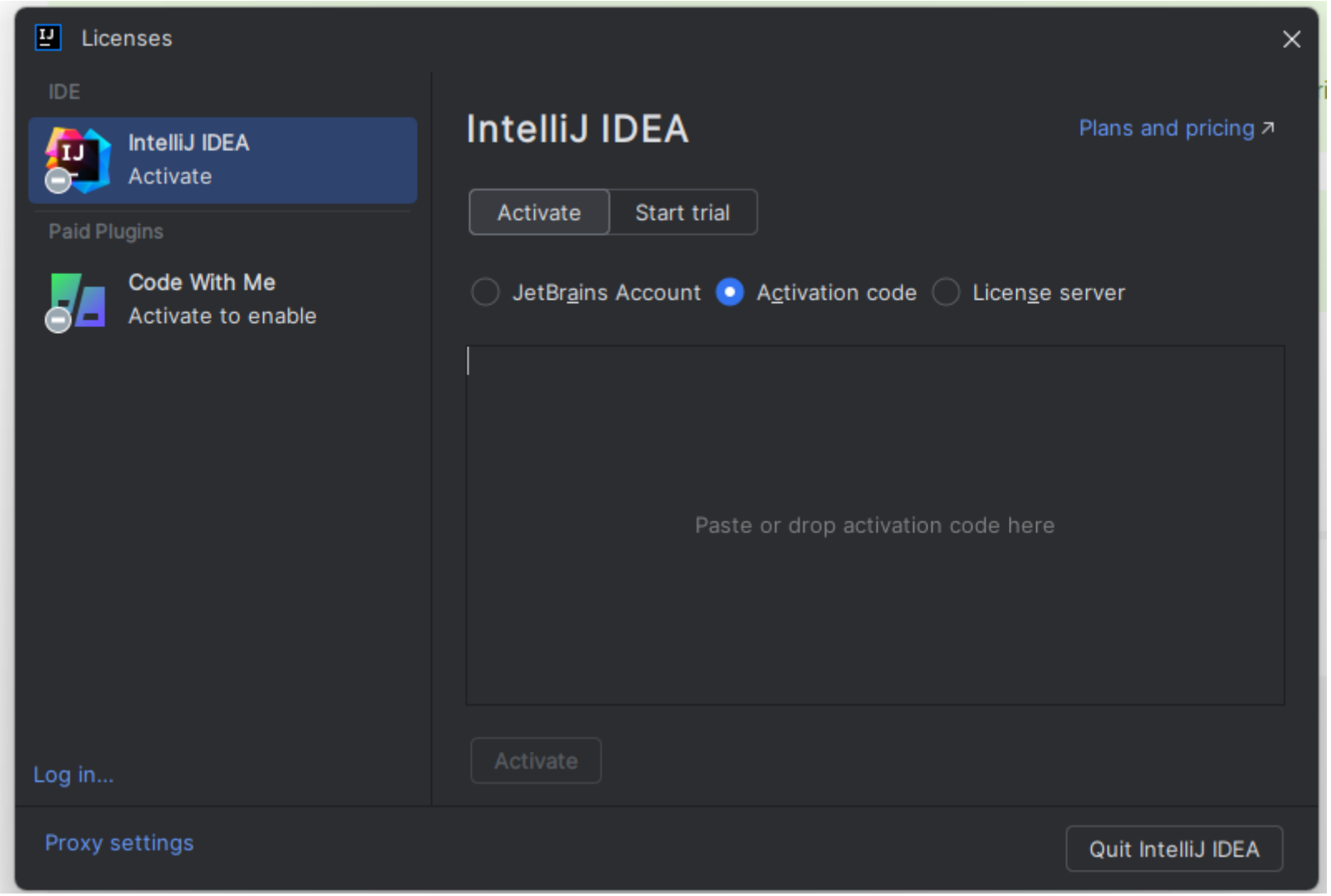
IntelliJ


JET BRAINS
13.10. Governing Law. This Agreement is governed by the laws of the Czech Republic, without reference to conflict of laws principles and specifically excluding the United Nations Convention on Contracts for the International Sale of Goods. The Parties to the agreement constituted by this Agreement undertake to use best commercial efforts to amicably settle any disputes arising hereunder ("Dispute").
13.11. Dispute Resolution. Should the Parties to this Agreement fail to settle a Dispute amicably, the Dispute will be excluded from the jurisdiction of general courts and the Dispute will be finally decided by the Arbitration Court attached to the Czech Chamber of Commerce and the Agricultural Chamber of the Czech Republic, by three arbitrators in accordance with the Rules of that Arbitration Court, and the language of the proceedings will be English; if you are a consumer, we both agree that any Dispute-related litigation may only be brought in, and shall be subject to the jurisdiction of, any competent court of the Czech Republic, unless provided otherwise by applicable consumer law. Consumer Disputes can also be settled out of court through the Czech Trade Inspection Authority (www.coi.cz) or the European Commission's online platform for dispute resolution (ec.europa.eu/consumers/odr).
13.12. Data Privacy. By accepting this Agreement, Customer acknowledges that JetBrains will process personal data in accordance with JetBrains' Privacy Policy (available at https://www.jetbrains.com/company/privacy.html). Unless you have signed an individual data processing addendum with JetBrains, the JetBrains data processing addendum available at https://www.jetbrains.com/legal/dpa applies.
13.13. Force Majeure. Neither Party shall be in breach of this Agreement, or otherwise liable to the other, by reason of any delay in performance, or non-performance, of any of its obligations under this Agreement (except payment obligations), arising directly from an act of God, fire, flood, natural disaster, act of terrorism, strike, lock-out, labor dispute, public health emergency, civil commotion, riot, or act of war.
13.14. Children and minors. If you are under 18 years old, then by entering into this Agreement you explicitly stipulate that (i) you have legal capacity to conclude this Agreement or that you have valid consent from a parent or legal guardian to do so and (ii) you understand the JetBrains Privacy Policy available at https://www.jetbrains.com/legal/docs/privacy/privacy.html. You may not enter into this Agreement if you are under 13 years old. IF YOU DO NOT UNDERSTAND THIS SECTION, DO NOT UNDERSTAND THE JETBRAINS PRIVACY POLICY, OR DO NOT KNOW WHETHER YOU HAVE THE LEGAL CAPACITY TO ACCEPT THESE TERMS, PLEASE ASK YOUR PARENT OR LEGAL GUARDIAN FOR HELP.
The previous version of this agreement is available here.
For further information, please contact us at legal@jetbrains.com
Please review and accept this license agreement to proceed with product activation.
I Accept

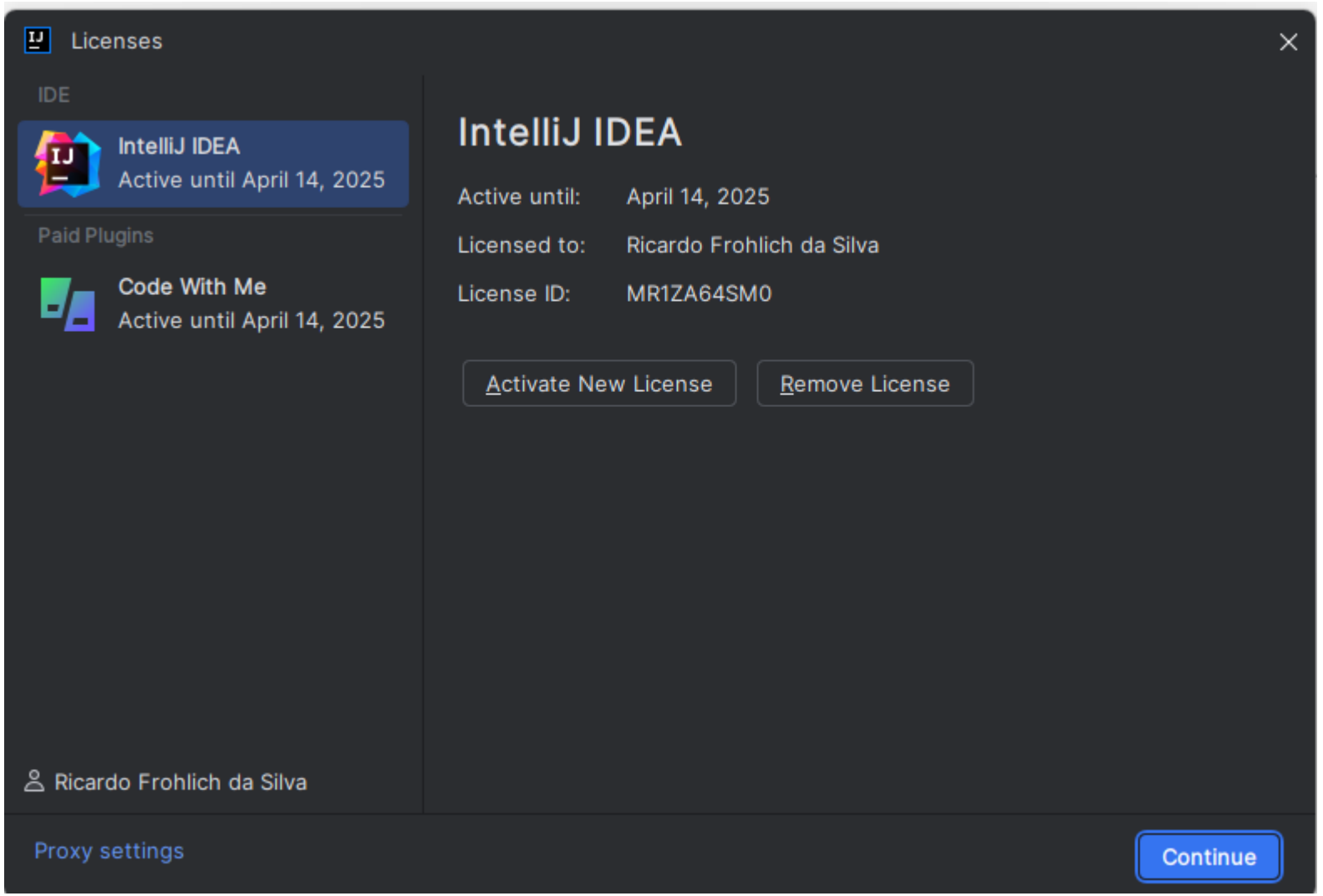
IntelliJ


JET BRAINS
Ricardo Frohlich da Silva
Transactions / Invoices
Notifications
JetBrains Product Pack for Students has successfully been added.
Keep in the loop! Choose which newsletters you'd like to receive, and get all the latest news about your favorite products.
Two-factor authentication (2FA) is available!
To enable an extra layer of security for your JetBrains Account, turn on two-factor auth.
Buy new license
1 Student Pack License
JetBrains Product Pack for Students
Download
Licensed to: Ricardo Frohlich da Silva
License restriction: For educational use only
Valid through: April 14, 2025
Following products included:
CLion
DataGrip
DataSpell
dotCover
dotMemory
dotTrace
GoLand
IntelliJ IDEA Ultimate
PhpStorm
PyCharm
ReSharper
ReSharper C++
Rider
RubyMine
WebStorm
After downloading and installing the software, simply run it and follow the on-screen prompts to sign in with your JetBrains Account.

IntelliJ

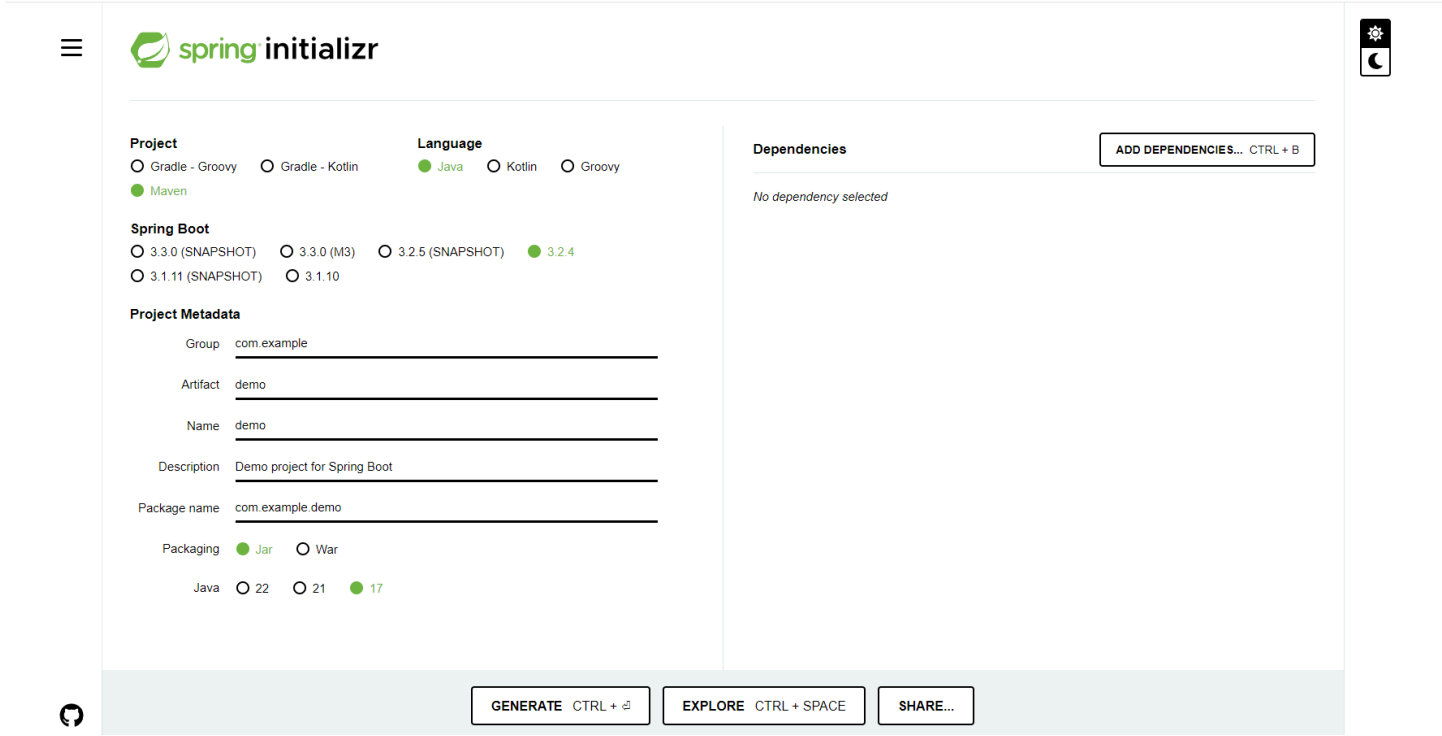

Licenses
IDE
IntelliJ IDEA
Activate
Paid Plugins
Code With Me
Activate to enable
Log in...
IntelliJ IDEA
Plans and pricing ↗
Activate
Start trial
JetBrains Account
Activation code
License server
Paste or drop activation code here
Activate
Proxy settings
Quit IntelliJ IDEA

IntelliJ

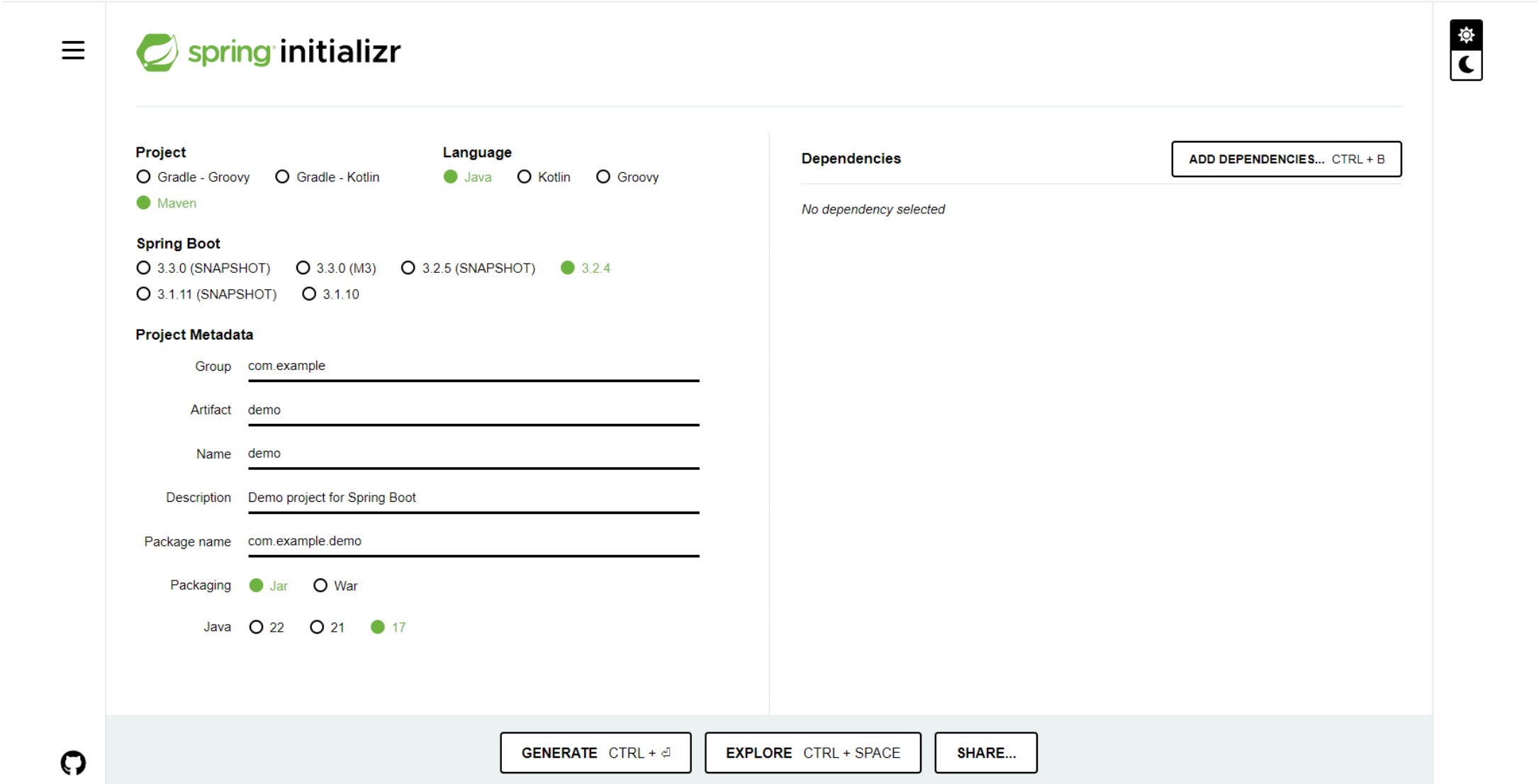

Licenses
IDE
IntelliJ IDEA
Active until April 14, 2025
Paid Plugins
Code With Me
Active until April 14, 2025
IntelliJ IDEA
Active until: April 14, 2025
Licensed to: Ricardo Frohlich da Silva
License ID: MR1ZA64SM0
Activate New License
Remove License
Ricardo Frohlich da Silva
Proxy settings
Continue

Vamos a pratica!

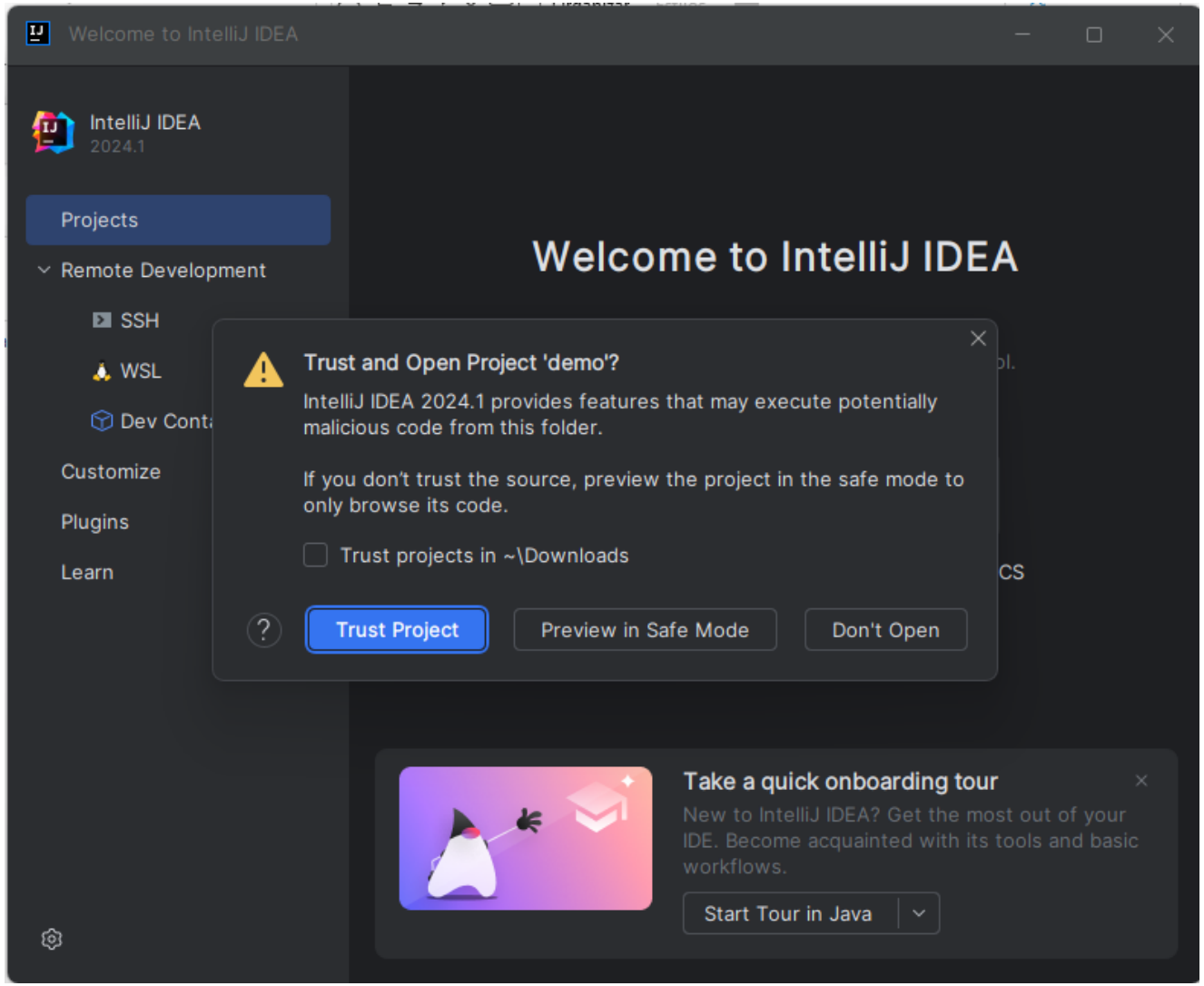
- Agora, que vimos os principais pontos iniciais, vamos a prática!
  - Vá para o site do Spring Initializr https://start.spring.io/



Vamos a pratica!



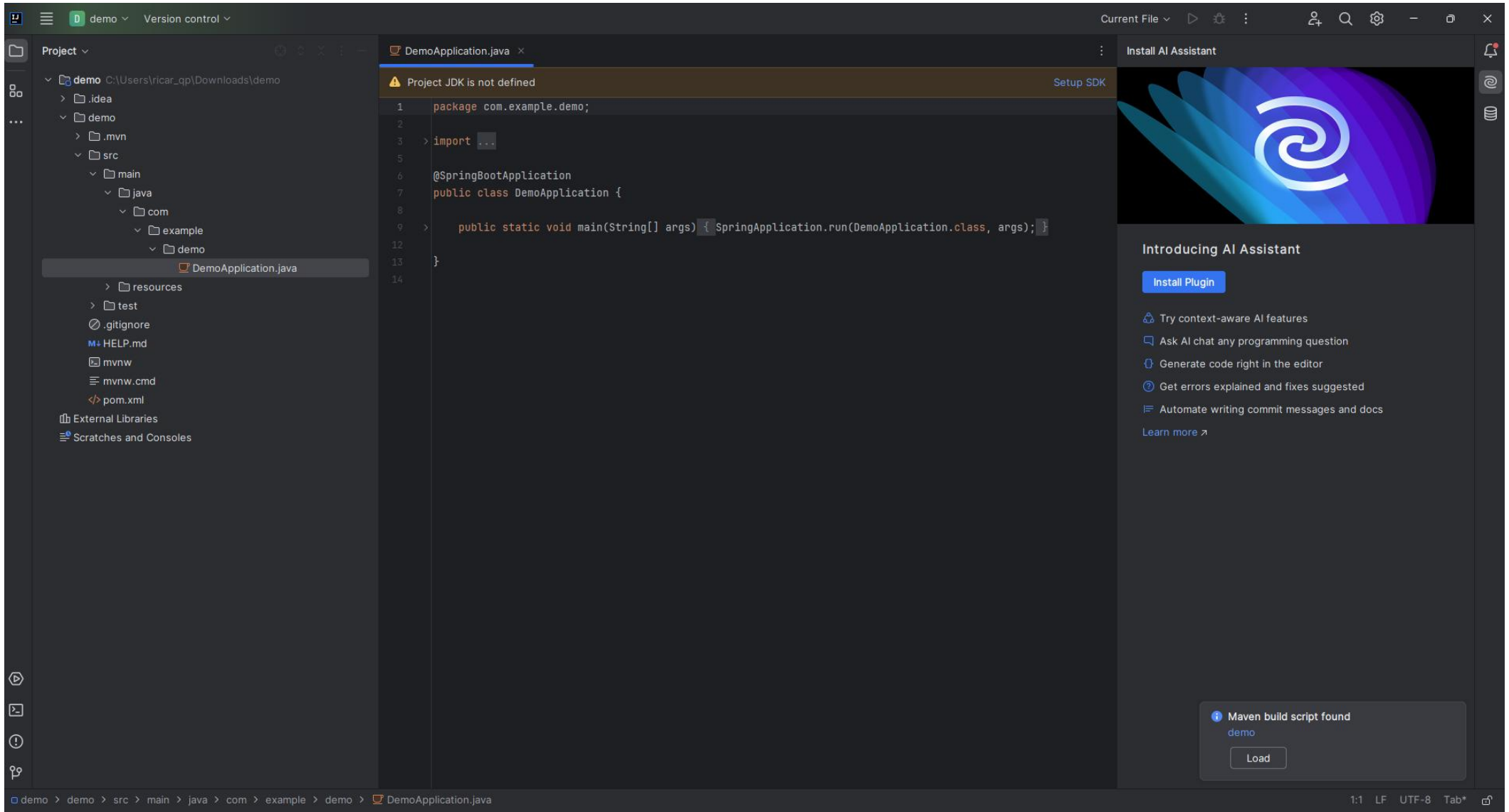
IntelliJ


Welcome to IntelliJ IDEA
IntelliJ IDEA
2024.1
Projects
Remote Development
SSH
WSL
Dev Cont
Customize
Plugins
Learn
Welcome to IntelliJ IDEA
Trust and Open Project 'demo'?
IntelliJ IDEA 2024.1 provides features that may execute potentially malicious code from this folder.
If you don't trust the source, preview the project in the safe mode to only browse its code.
Trust projects in ~\Downloads
Trust Project
Preview in Safe Mode
Don't Open
Take a quick onboarding tour
New to IntelliJ IDEA? Get the most out of your IDE. Become acquainted with its tools and basic workflows.
Start Tour in Java

IntelliJ

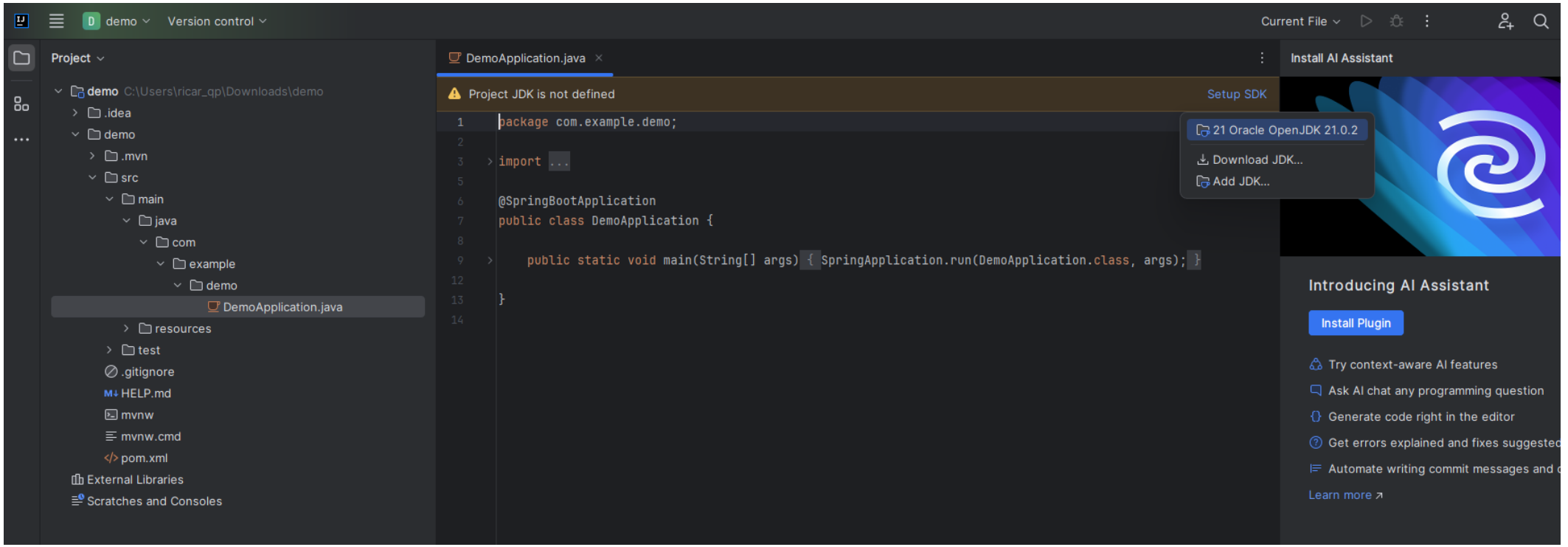
- O IntelliJ IDEA detectará automaticamente que é um projeto Maven e começará a importá-lo.
- Na janela de importação do Maven, certifique-se de que as opções estão configuradas como desejar e clique em "Next".
- Na próxima tela, você pode selecionar quais módulos do projeto deseja importar. Por padrão, todos os módulos devem ser selecionados, então você pode clicar em "Next" novamente.
    - O IntelliJ IDEA começará a importar as dependências do Maven e configurar o projeto.
- Após a importação ser concluída, você verá o projeto aberto no IntelliJ IDEA, pronto para ser desenvolvido.

IntelliJ

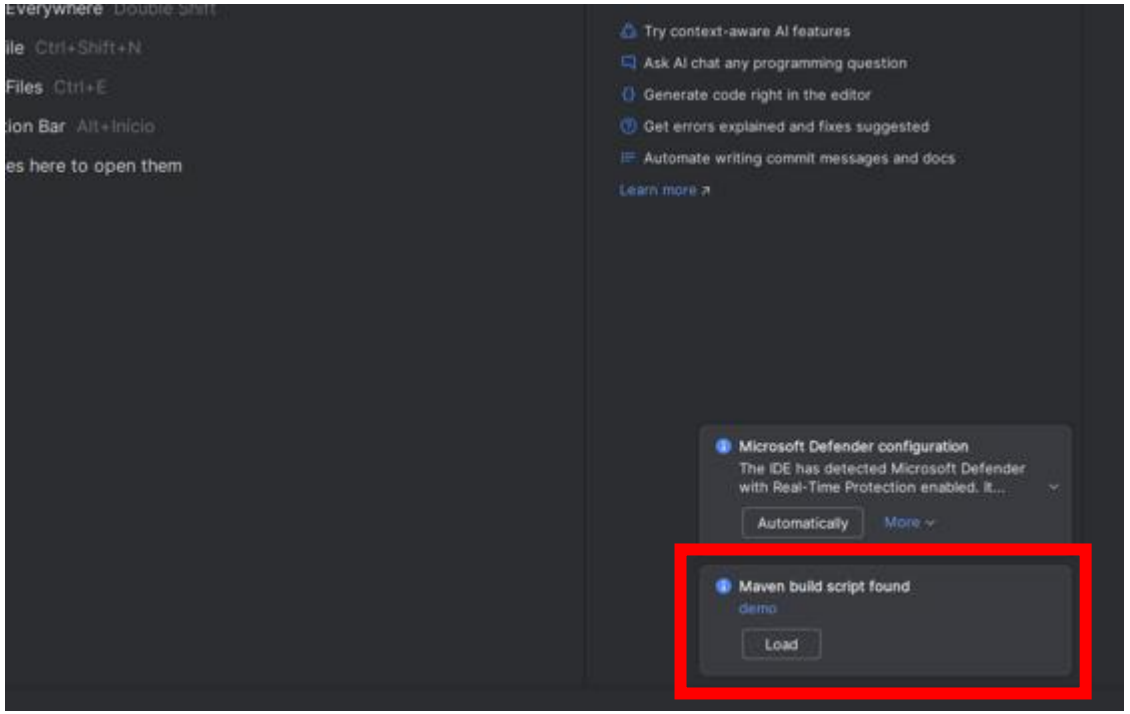

demo
Version control
Current File
Project
demo C:\Users\ricar_qp\Downloads\demo
.idea
demo
.mvn
src
main
java
com
example
demo
DemoApplication.java
resources
test
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
External Libraries
Scratches and Consoles
DemoApplication.java
Project JDK is not defined
Setup SDK
package com.example.demo;
import ...
@SpringBootApplication
public class DemoApplication {
public static void main(String[] args) { SpringApplication.run(DemoApplication.class, args); }
}
Install AI Assistant
Introducing AI Assistant
Install Plugin
Try context-aware AI features
Ask AI chat any programming question
Generate code right in the editor
Get errors explained and fixes suggested
Automate writing commit messages and docs
Learn more
Maven build script found
demo
Load
demo > demo > src > main > java > com > example > demo > DemoApplication.java
1:1 LF UTF-8 Tab*

IntelliJ

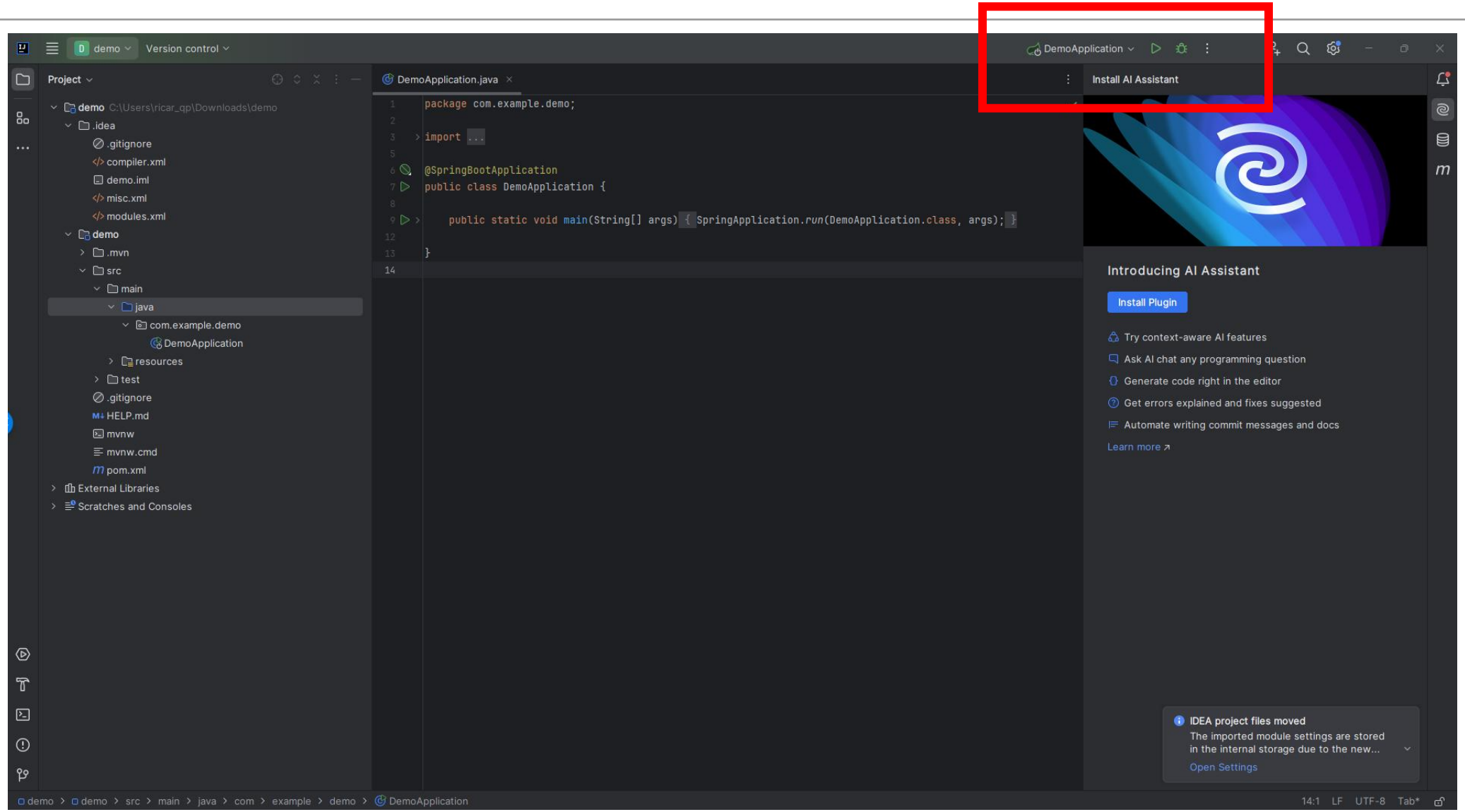
- Na criação do projeto, informamos que o JDK era o 17 e aqui o IntelliJ já pediu para corrigir.



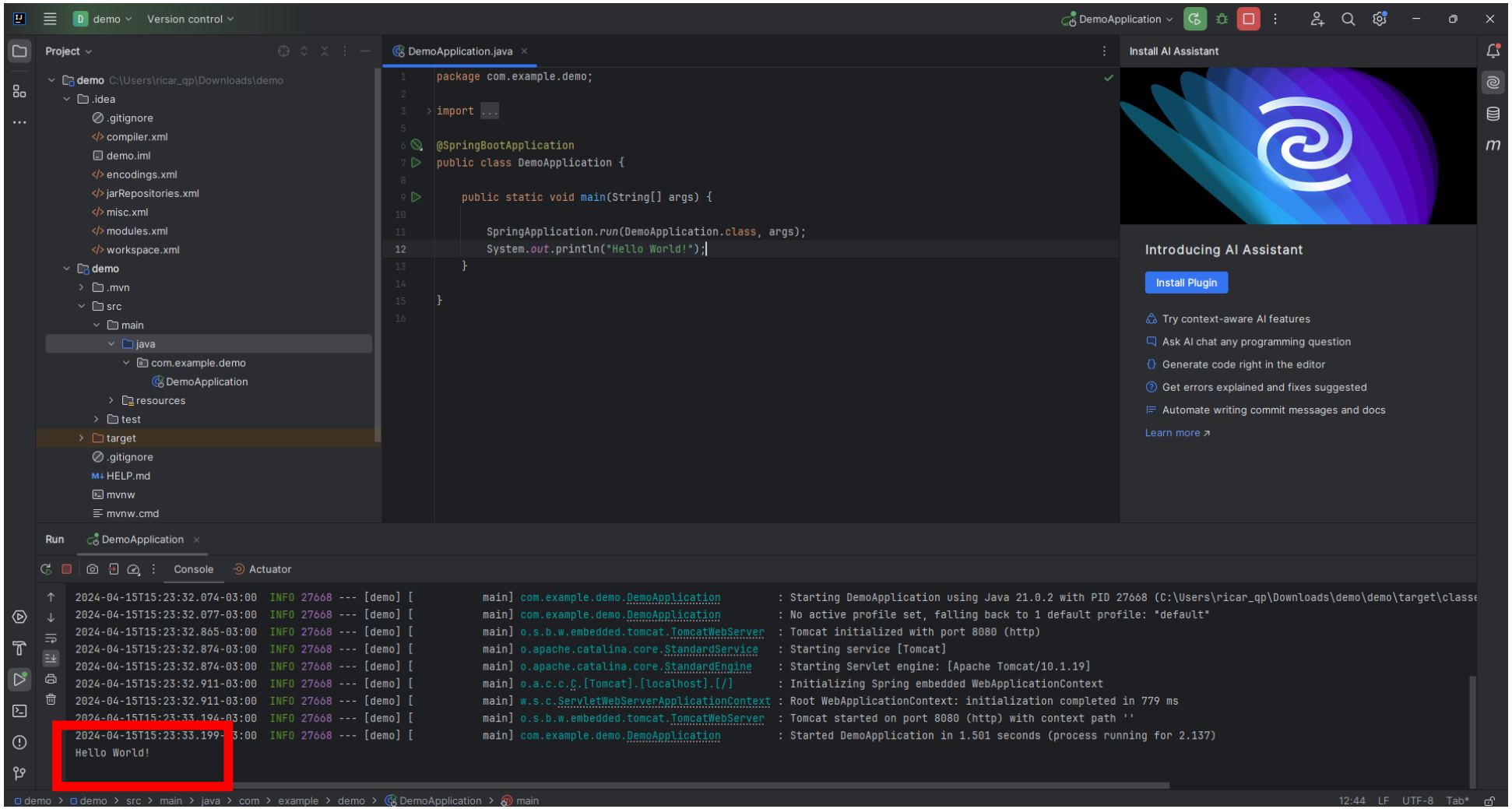
IntelliJ


Everywhere Double Shift
ile Ctrl+Shift+N
Files Ctrl+E
ion Bar Alt+Inicio
es here to open them
Try context-aware AI features
Ask AI chat any programming question
Generate code right in the editor
Get errors explained and fixes suggested
Automate writing commit messages and docs
Learn more
Microsoft Defender configuration
The IDE has detected Microsoft Defender with Real-Time Protection enabled. It...
Automatically
More
Maven build script found
demo
Load

IntelliJ


demo
Version control
DemoApplication
Project
demo C:\Users\ricar_qp\Downloads\demo
.idea
.gitignore
compiler.xml
demo.iml
misc.xml
modules.xml
demo
.mvn
src
main
java
com.example.demo
DemoApplication
resources
test
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
External Libraries
Scratches and Consoles
DemoApplication.java
package com.example.demo;
import ...
@SpringBootApplication
public class DemoApplication {
public static void main(String[] args) { SpringApplication.run(DemoApplication.class, args); }
}
Install AI Assistant
Introducing AI Assistant
Install Plugin
Try context-aware AI features
Ask AI chat any programming question
Generate code right in the editor
Get errors explained and fixes suggested
Automate writing commit messages and docs
Learn more
IDEA project files moved
The imported module settings are stored in the internal storage due to the new...
Open Settings
demo > demo > src > main > java > com > example > demo > DemoApplication
14:1 LF UTF-8 Tab*

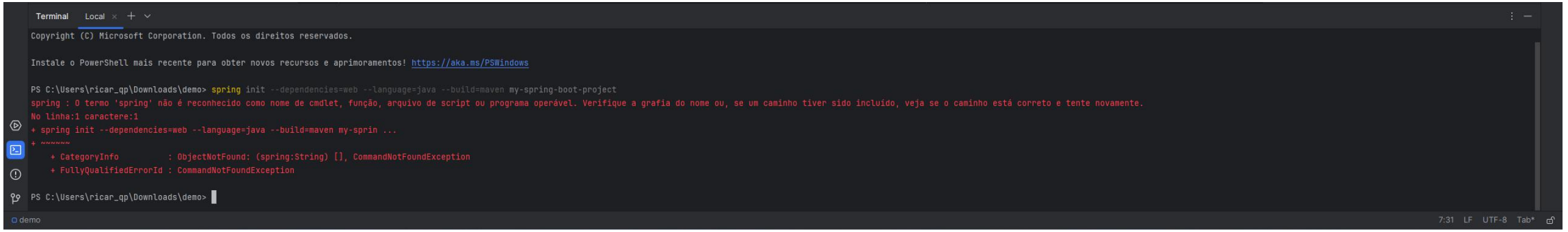
IntelliJ


demo
Version control
DemoApplication
Project
DemoApplication.java
Install AI Assistant
demo C:\Users\ricar_qp\Downloads\demo
.idea
.gitignore
compiler.xml
demo.iml
encodings.xml
jarRepositories.xml
misc.xml
modules.xml
workspace.xml
demo
.mvn
src
main
java
com.example.demo
DemoApplication
resources
test
target
.gitignore
HELP.md
mvnw
mvnw.cmd
package com.example.demo;
import ...
@SpringBootApplication
public class DemoApplication {
public static void main(String[] args) {
SpringApplication.run(DemoApplication.class, args);
System.out.println("Hello World!");
}
}
Introducing AI Assistant
Install Plugin
Try context-aware AI features
Ask AI chat any programming question
Generate code right in the editor
Get errors explained and fixes suggested
Automate writing commit messages and docs
Learn more
Run
DemoApplication
Console
Actuator
2024-04-15T15:23:32.074-03:00 INFO 27668 --- [demo] [main] com.example.demo.DemoApplication : Starting DemoApplication using Java 21.0.2 with PID 27668 (C:\Users\ricar_qp\Downloads\demo\demo\target\classe
2024-04-15T15:23:32.077-03:00 INFO 27668 --- [demo] [main] com.example.demo.DemoApplication : No active profile set, falling back to 1 default profile: "default"
2024-04-15T15:23:32.865-03:00 INFO 27668 --- [demo] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2024-04-15T15:23:32.874-03:00 INFO 27668 --- [demo] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2024-04-15T15:23:32.874-03:00 INFO 27668 --- [demo] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.19]
2024-04-15T15:23:32.911-03:00 INFO 27668 --- [demo] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2024-04-15T15:23:32.911-03:00 INFO 27668 --- [demo] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 779 ms
2024-04-15T15:23:33.194-03:00 INFO 27668 --- [demo] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2024-04-15T15:23:33.199-03:00 INFO 27668 --- [demo] [main] com.example.demo.DemoApplication : Started DemoApplication in 1.501 seconds (process running for 2.137)
Hello World!
demo > demo > src > main > java > com > example > demo > DemoApplication > main
12:44 LF UTF-8 Tab*

Iniciando um projeto por linha de comando utilizando Spring Boot CLI

- spring init --dependencies=<dependencies> --language=<language> --build=<build_tool> <project_name>
- Substitua:
  - <dependencies> pelas dependências desejadas para o seu projeto (por exemplo, web para uma aplicação web Spring Boot)
  - <language> pelo idioma desejado (por exemplo, java)
  - <build_tool> pela ferramenta de construção desejada (por exemplo, maven ou gradle)
  - e <project_name> pelo nome do seu projeto.
- spring init --dependencies=web --language=java --build=maven my-spring-boot-project

Iniciando um projeto por linha de comando utilizando Spring Boot CLI



- O erro ocorreu pois não foi encontrado o CLI do Spring.
- Para fazer o download, acesse:
  - https://docs.spring.io/spring-boot/docs/current/reference/html/getting-started.html#getting-started.installing.cli

Iniciando um projeto por linha de comando utilizando Spring Boot CLI

‹ Back to index

1. Introducing Spring Boot
2. System Requirements
3. Installing Spring Boot
   3.1. Installation Instructions for the Java Developer
   3.2. Installing the Spring Boot CLI
      3.2.1. Manual Installation
      3.2.2. Installation with SDKMAN!
      3.2.3. OSX Homebrew Installation
      3.2.4. MacPorts Installation
      3.2.5. Command-line Completion
      3.2.6. Windows Scoop Installation
4. Developing Your First Spring Boot Application
5. What to Read Next

3.2. Installing the Spring Boot CLI

The Spring Boot CLI (Command Line Interface) is a command line tool that you can use to quickly prototype with Spring.

You do not need to use the CLI to work with Spring Boot, but it is a quick way to get a Spring application off the ground without an IDE.

3.2.1. Manual Installation

You can download the Spring CLI distribution from one of the following locations:

- spring-boot-cli-3.2.4-bin.zip
- spring-boot-cli-3.2.4-bin.tar.gz

Once downloaded, follow the INSTALL.txt instructions from the unpacked archive. In summary, there is a `spring` script (`spring.bat` for Windows) in a `bin/` directory in the `.zip` file. Alternatively, you can use `java -jar` with the `.jar` file (the script helps you to be sure that the classpath is set correctly).

3.2.2. Installation with SDKMAN!

SDKMAN! (The Software Development Kit Manager) can be used for managing multiple versions of various binary SDKs, including Groovy and the Spring Boot CLI. Get SDKMAN! from sdkman.io and install Spring Boot by using the following commands:

```shell
$ sdk install springboot
$ spring --version
Spring CLI v3.2.4
```

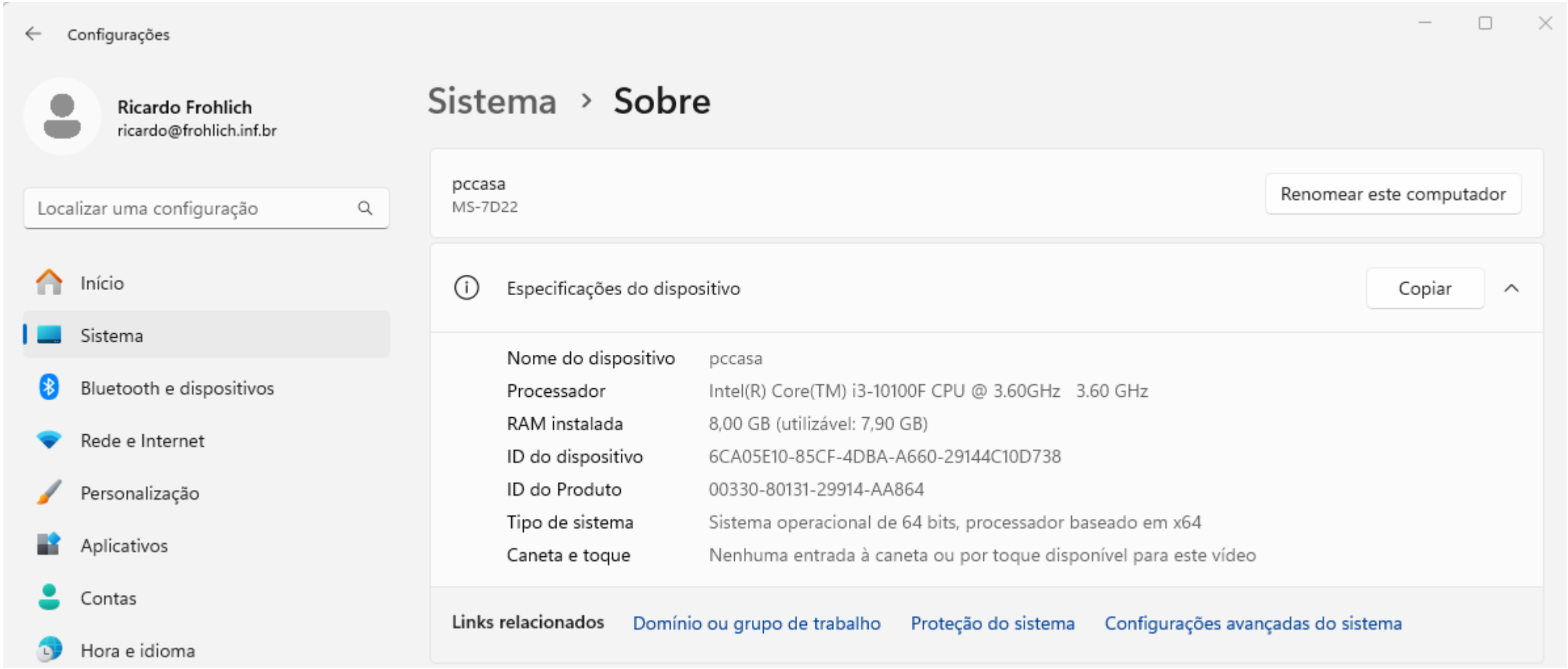
If you develop features for the CLI and want access to the version you built, use the following commands:

```shell
$ sdk install springboot dev /path/to/spring-boot/spring-boot-cli/target/spring-boot-cli-3.2.4-bin/spring-3.2.4/
$ sdk default springboot dev
$ spring --version
Spring CLI v3.2.4
```

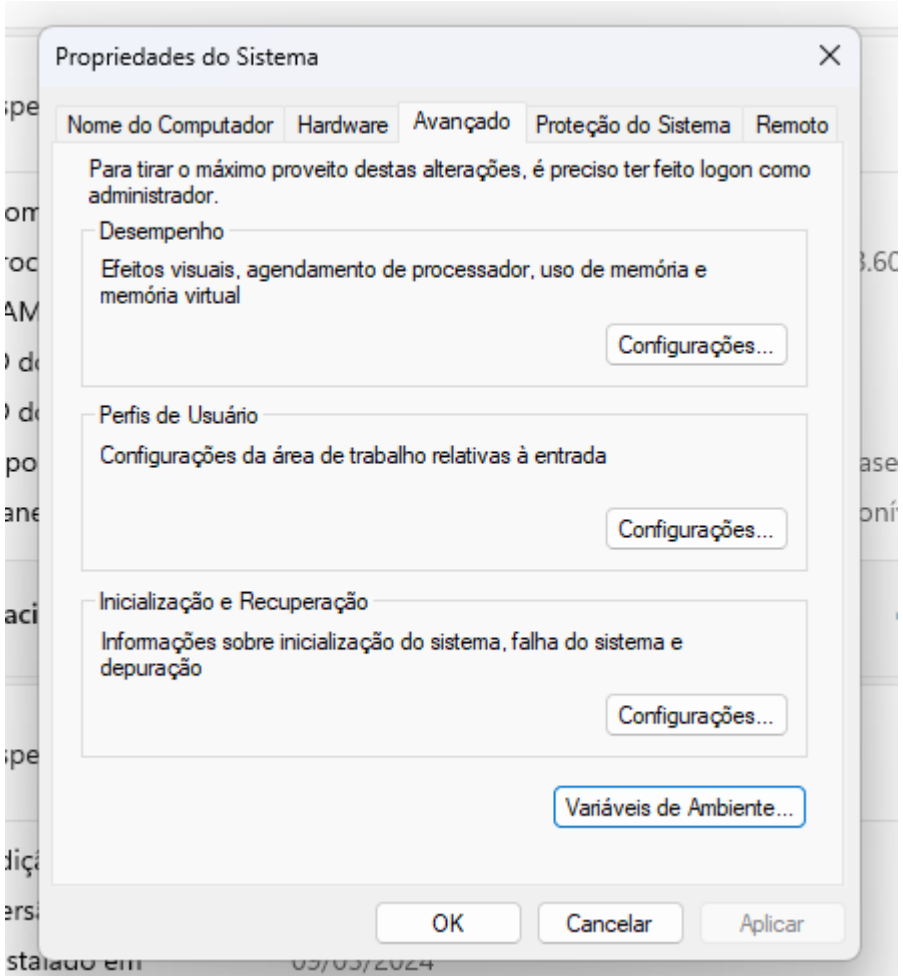
Iniciando um projeto por linha de comando utilizando Spring Boot CLI

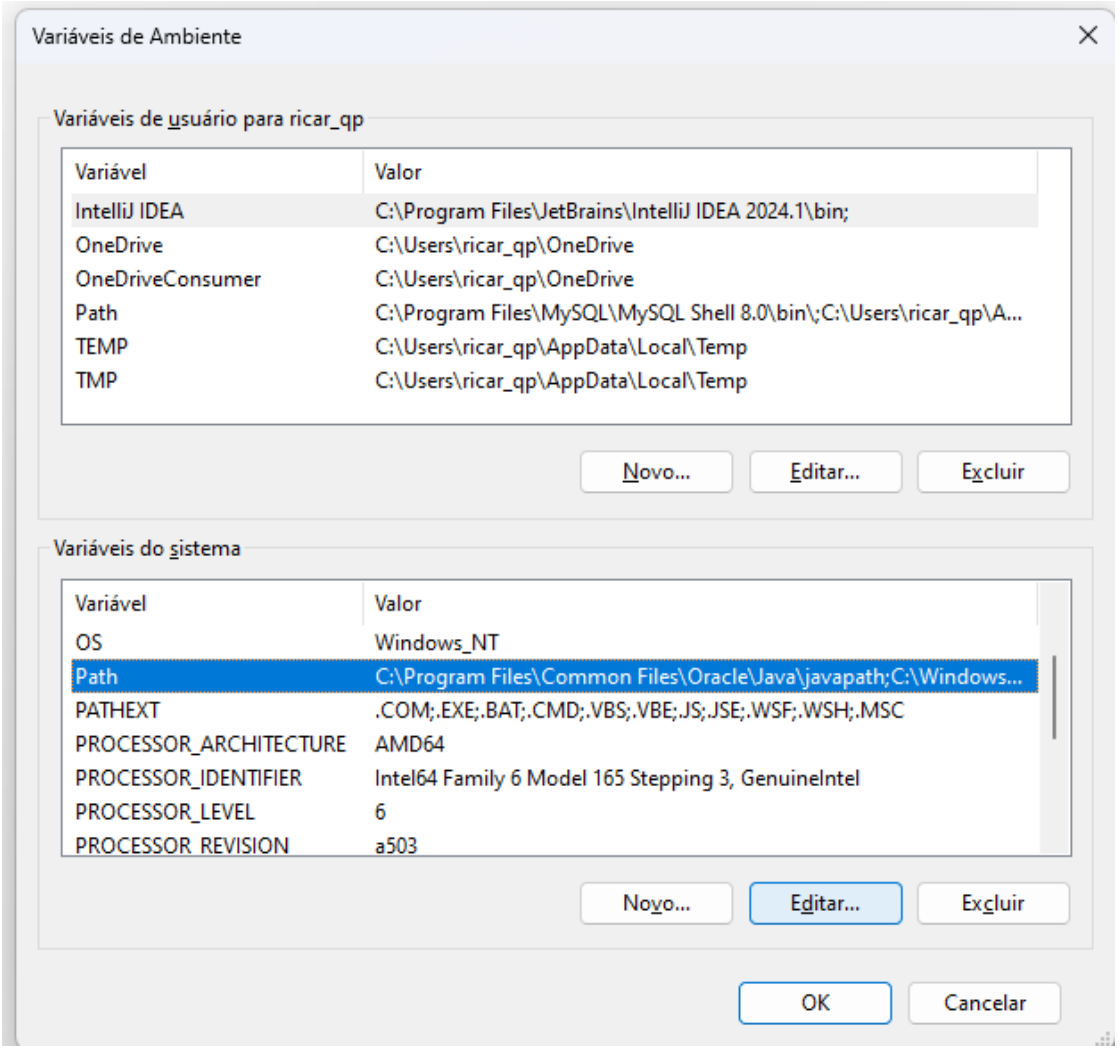
- Faça o download do zip, descompacte e coloque em algum lugar a pasta.
  - Clique com o botão direito do mouse no ícone "Meu Computador" (ou "Este PC") e selecione "Propriedades".
  - No painel esquerdo, clique em "Configurações avançadas do sistema".
  - Na janela Propriedades do Sistema, clique no botão "Variáveis de Ambiente".
  - Na seção "Variáveis do Sistema", encontre a variável chamada "Path" e clique em "Editar".
  - Na janela Editar Variável de Sistema, clique em "Novo" e adicione o caminho para o diretório onde o Spring Boot CLI foi extraído.
  - Clique em "OK" para fechar todas as janelas.

Iniciando um projeto por linha de comando utilizando Spring Boot CLI

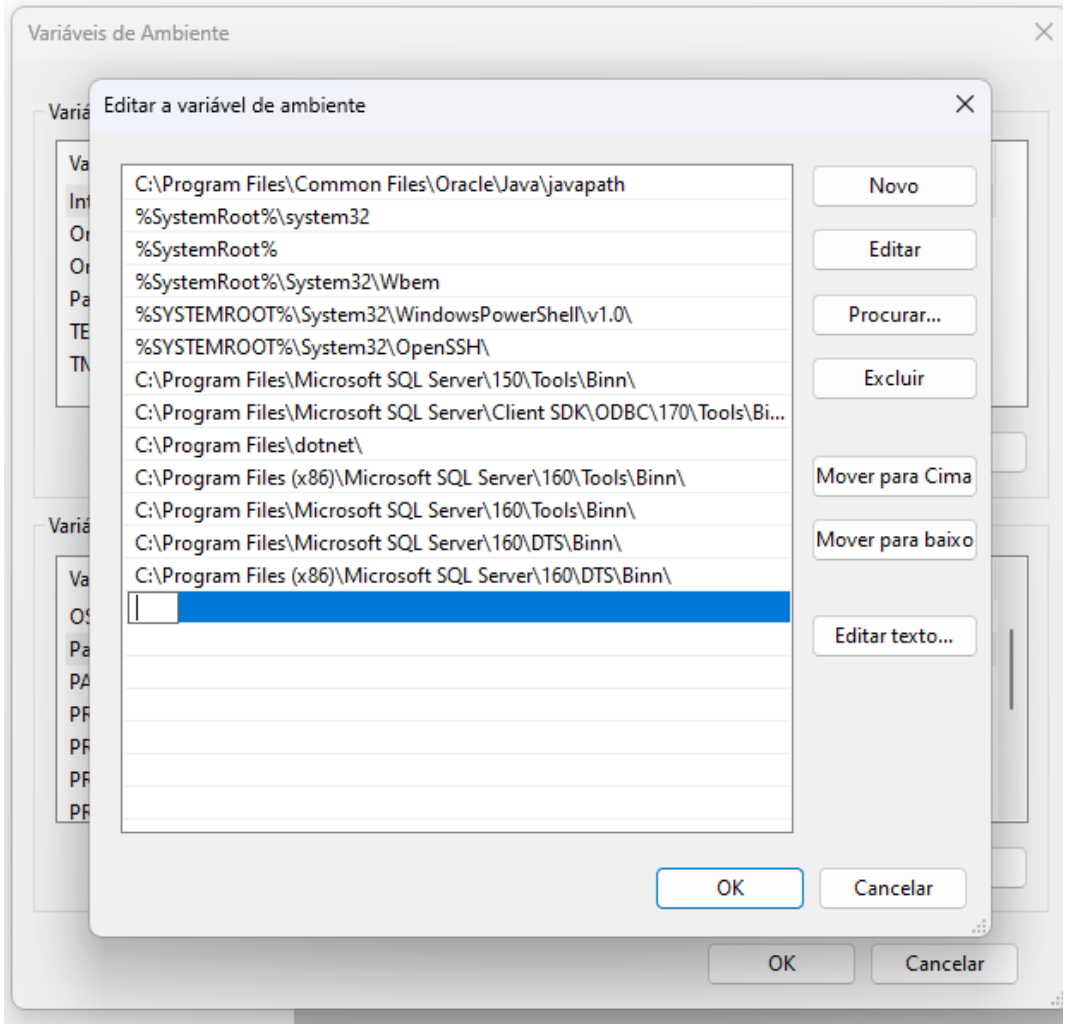


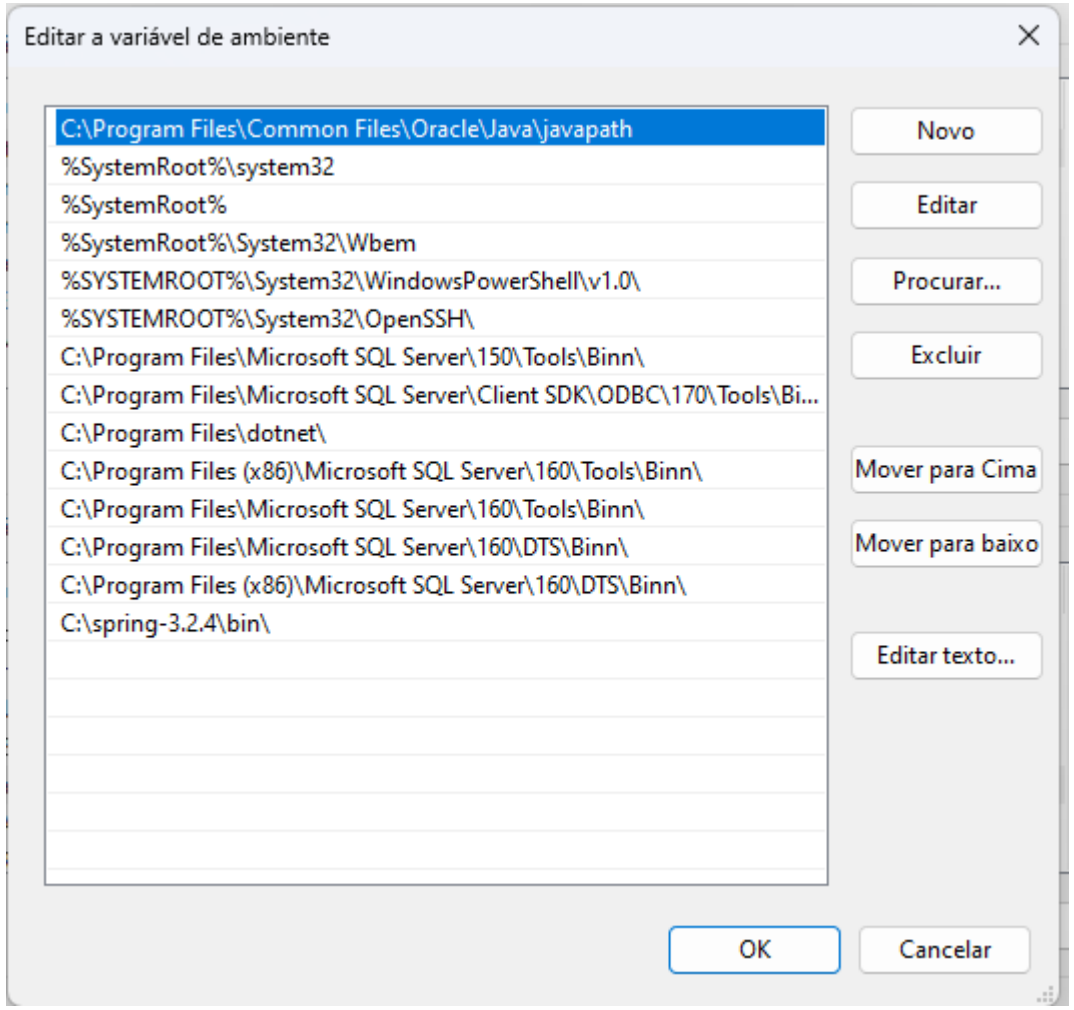
Iniciando um projeto por linha de comando utilizando Spring Boot CLI



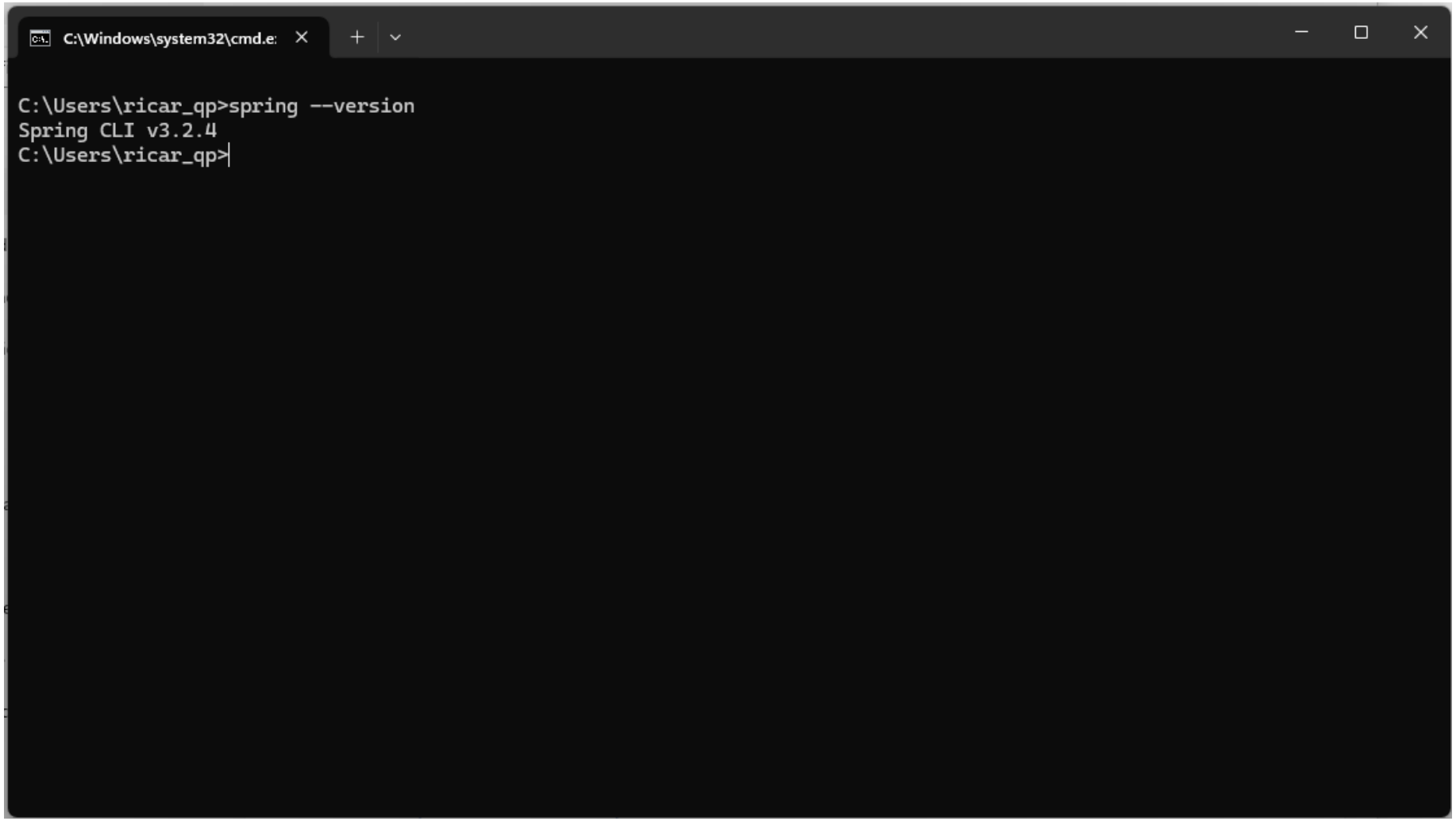


Iniciando um projeto por linha de comando utilizando Spring Boot CLI

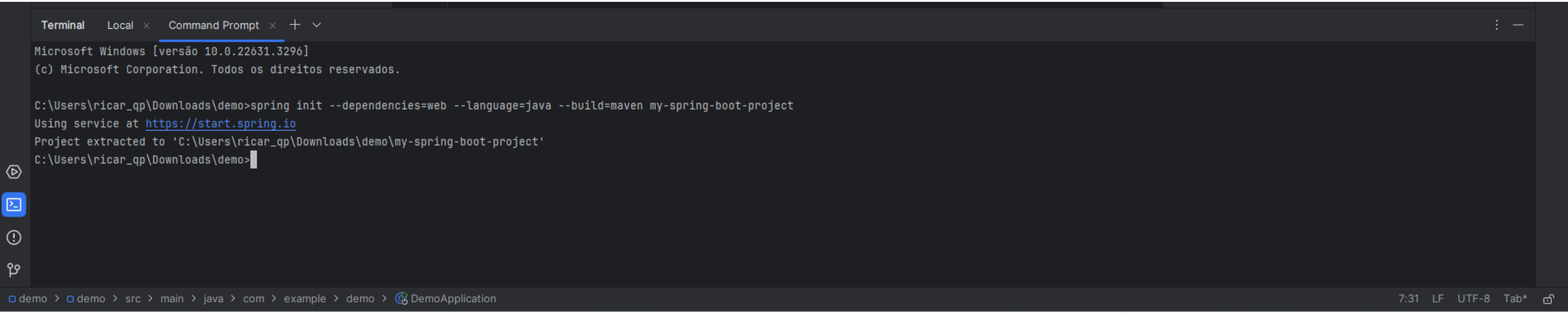




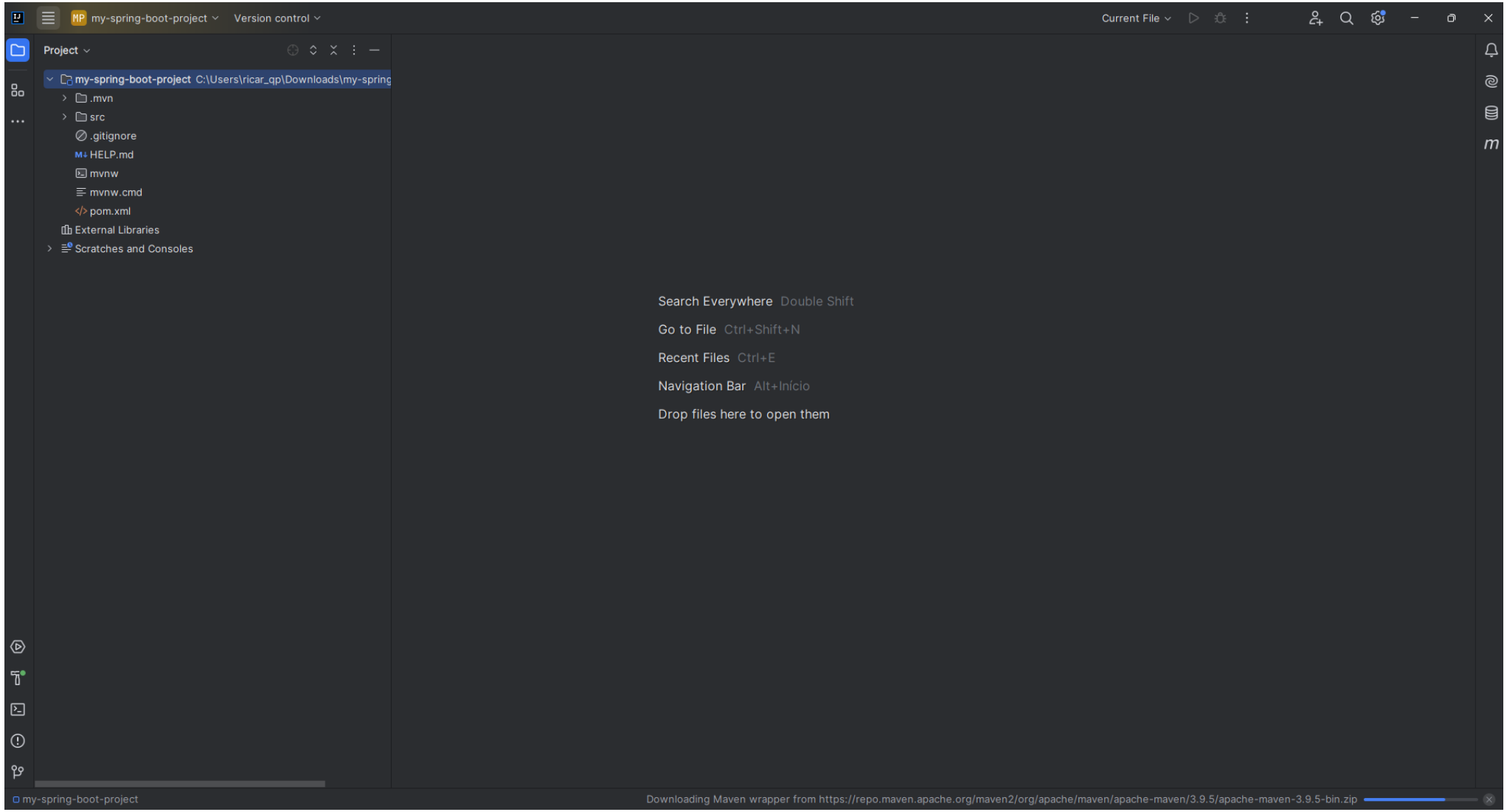
Iniciando um projeto por linha de comando utilizando Spring Boot CLI



Iniciando um projeto por linha de comando utilizando Spring Boot CLI



Iniciando um projeto por linha de comando utilizando Spring Boot CLI



Gerenciando dependências

- O pom.xml descreve o projeto como um modelo de objetos.
- Ele contém metadados sobre o projeto e suas configurações, permitindo que o Maven saiba como compilar, testar, empacotar e distribuir o projeto.
- No pom.xml, são especificadas as coordenadas do projeto, incluindo o groupId, artifactId e version. O groupId identifica exclusivamente o projeto entre todos os projetos disponíveis. O artifactId é o identificador único do artefato (geralmente o nome do projeto). E a version indica a versão atual do projeto.

Gerenciando dependências

- Uma das **partes mais importantes** do pom.xml são as dependências do projeto.
  - As dependências são especificadas no pom.xml para que o Maven possa baixar automaticamente as bibliotecas necessárias para o projeto.
  - As dependências podem incluir bibliotecas de terceiros, frameworks e outras dependências de projeto interno.

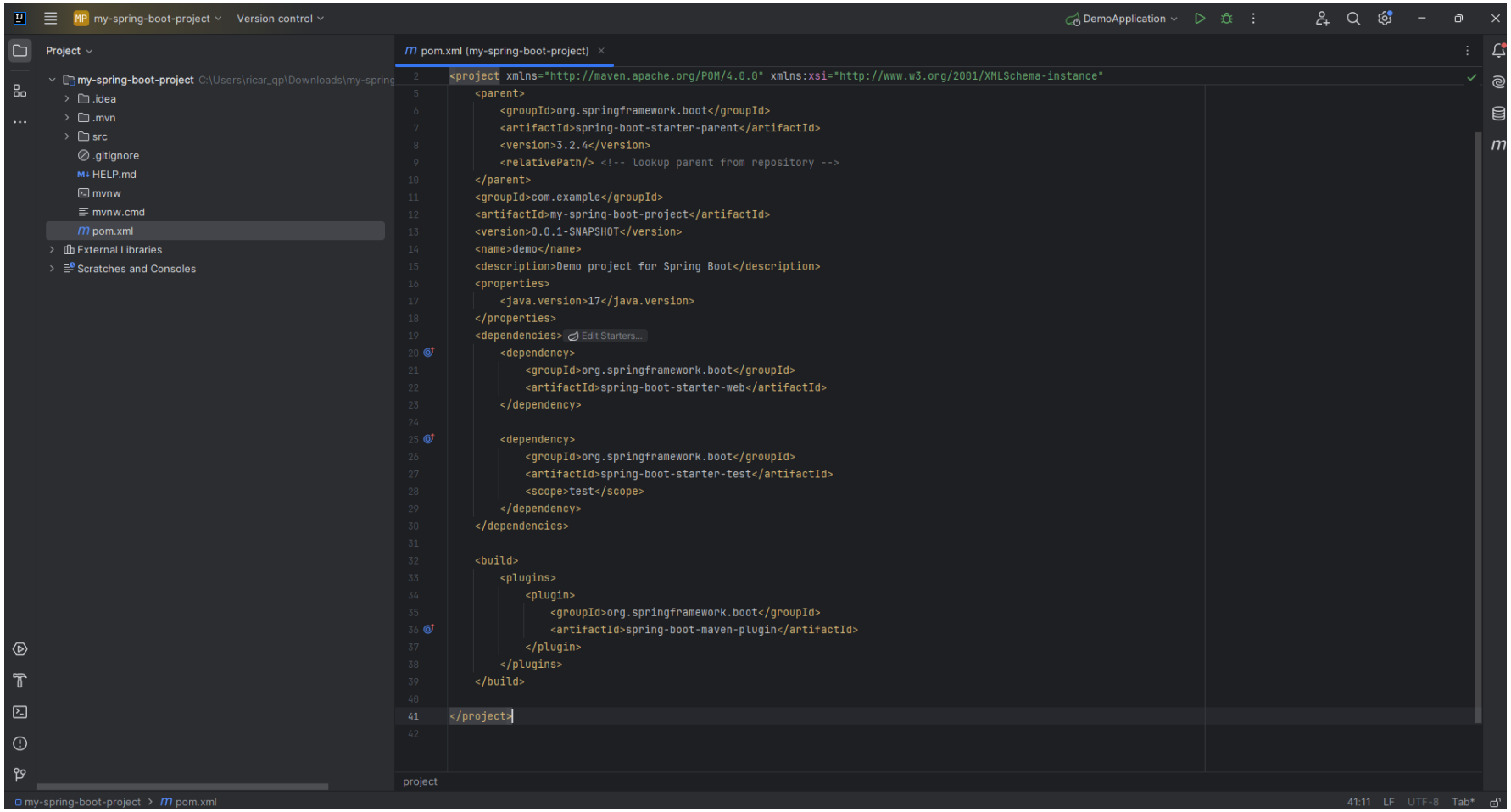
Gerenciando dependências

- Os plugins Maven são ferramentas que executam tarefas específicas durante o ciclo de vida de construção do projeto.
    - No pom.xml, você pode configurar plugins para compilar código, executar testes, gerar documentação, empacotar o projeto, entre outras tarefas.
- O pom.xml também contém configurações relacionadas ao processo de construção do projeto, como configurações de compilação, recursos, testes, empacotamento e distribuição

Gerenciando dependências

- O Maven suporta a herança de pom.xml, o que significa que você pode ter um pom.xml principal que define configurações comuns para vários subprojetos.
    - Isso facilita a manutenção e a consistência entre projetos relacionados.
- No pom.xml, você também pode especificar repositórios Maven onde o Maven pode procurar e baixar dependências que não estão disponíveis localmente.
    - O repositório Maven Central é o repositório padrão, mas você pode adicionar repositórios adicionais, se necessário.

Gerenciando dependências manipulando o pow.xml



Gerenciando dependências manipulando o pow.xml

pom.xml (my-spring-boot-project)

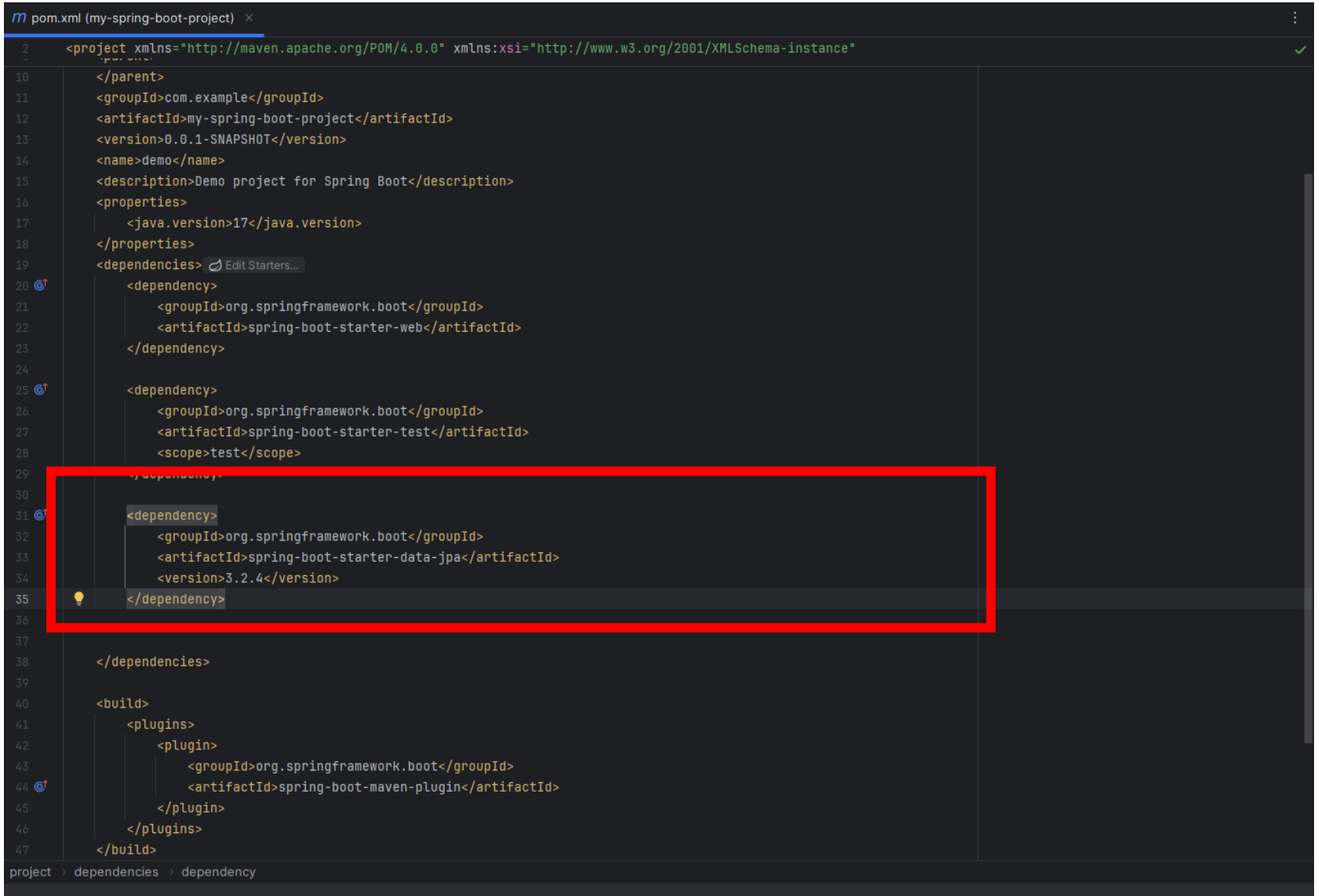
```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.4</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>my-spring-boot-project</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>demo</name>
    <description>Demo project for Spring Boot</description>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies> Edit Starters...
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

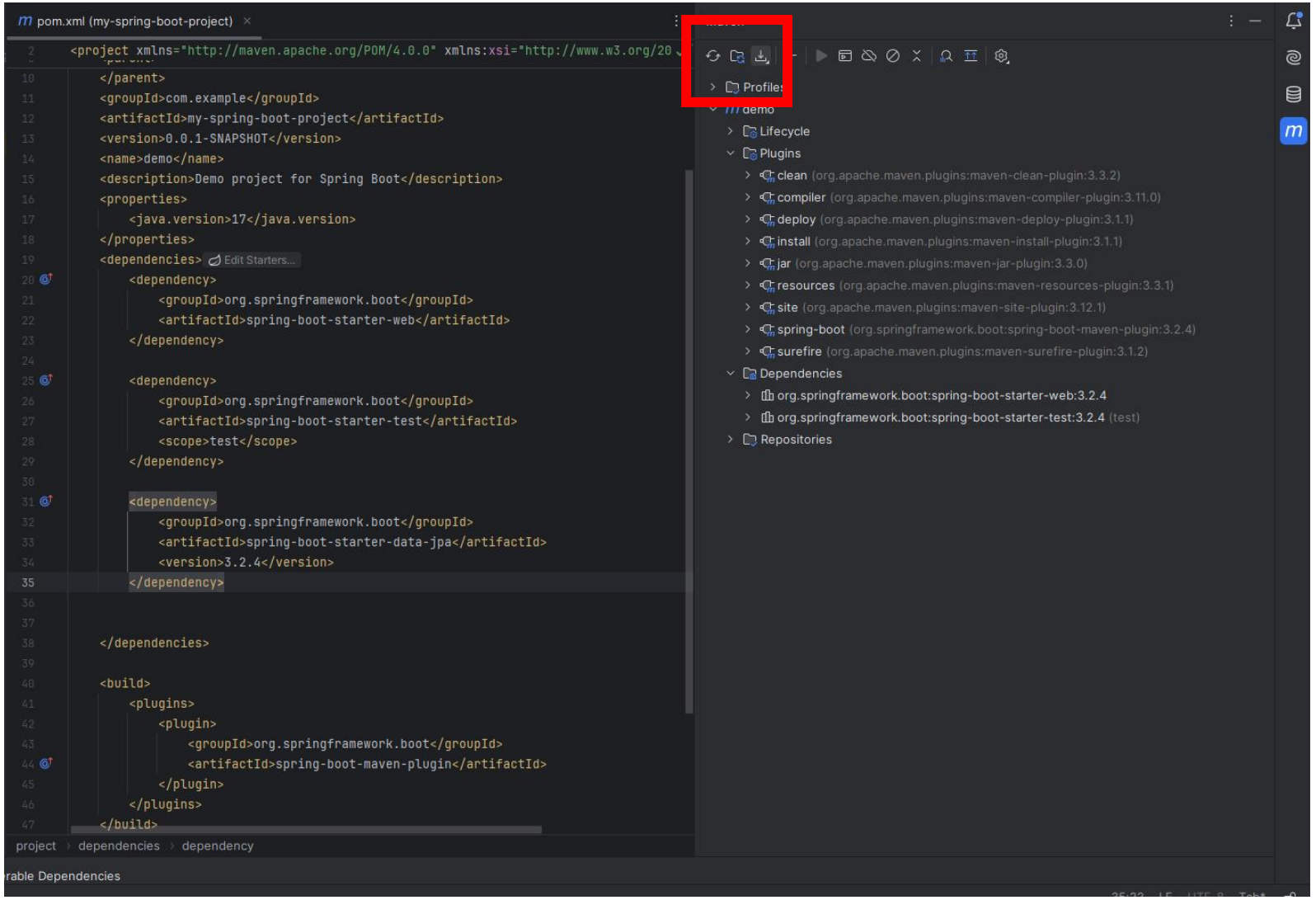
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

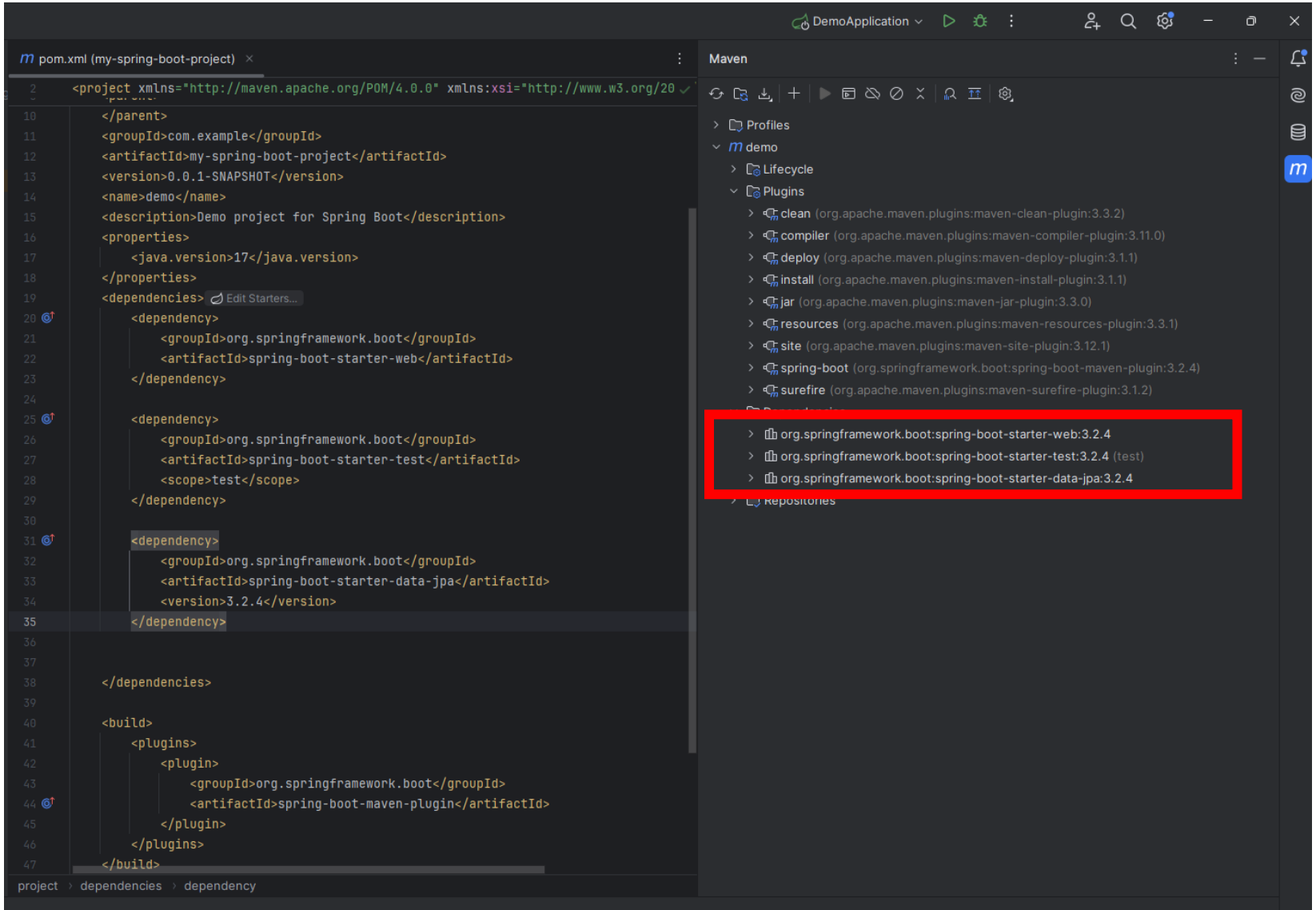
Gerenciando dependências manipulando o pow.xml



Gerenciando dependências manipulando o pow.xml



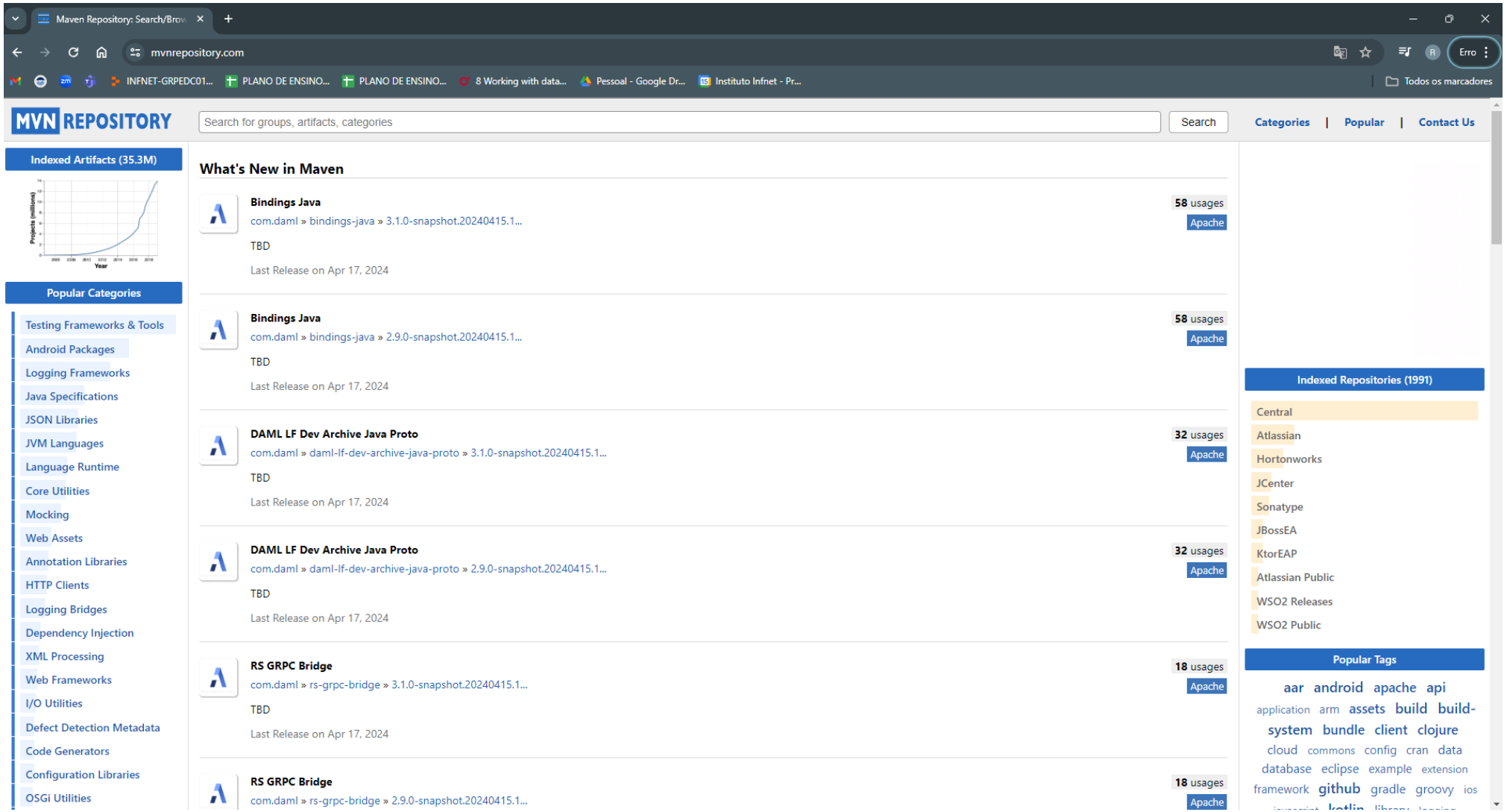
Gerenciando dependências manipulando o pow.xml



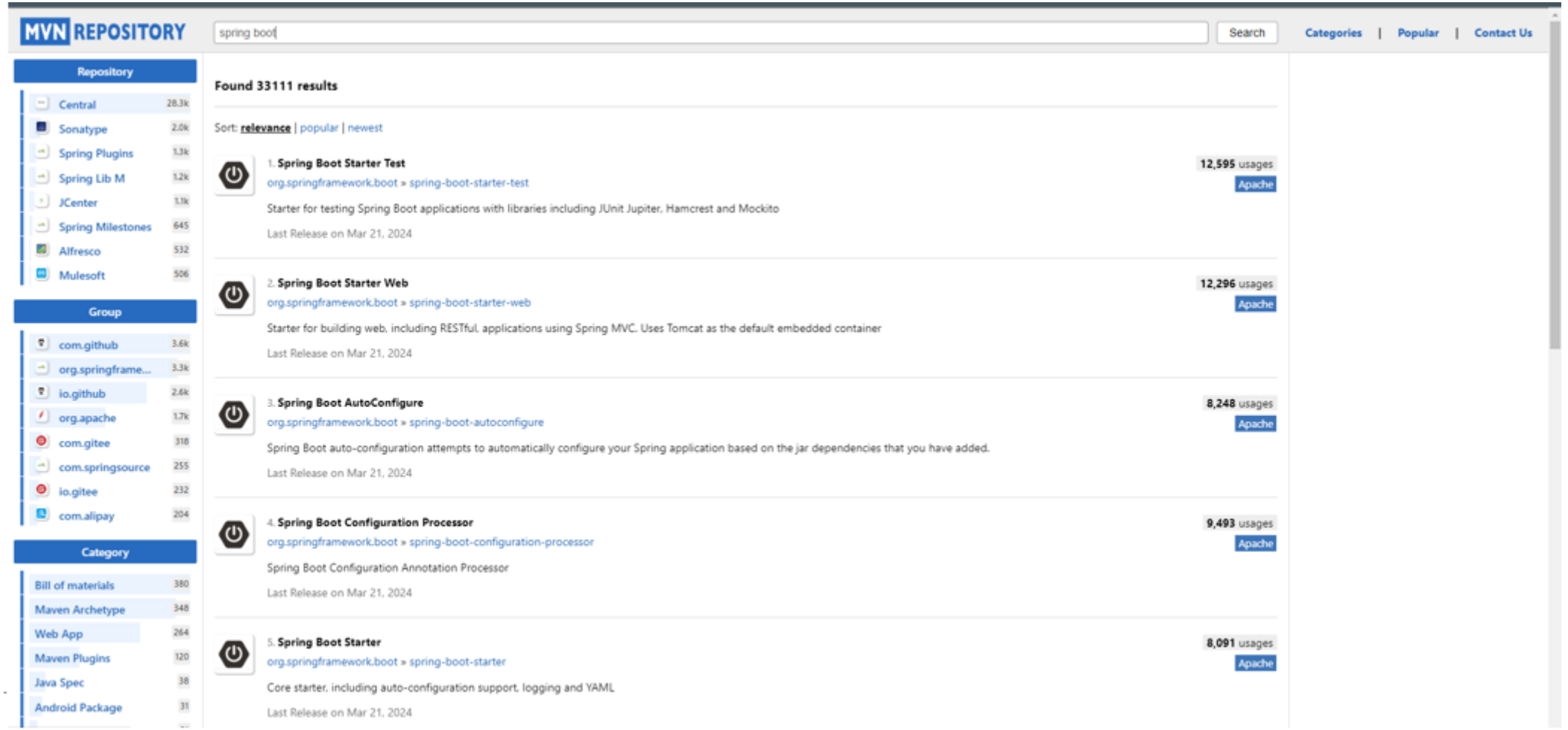
Gerenciando dependências manipulando o pow.xml

- E onde encontrar as dependências?
  - https://mvnrepository.com/

Gerenciando dependências manipulando o pow.xml

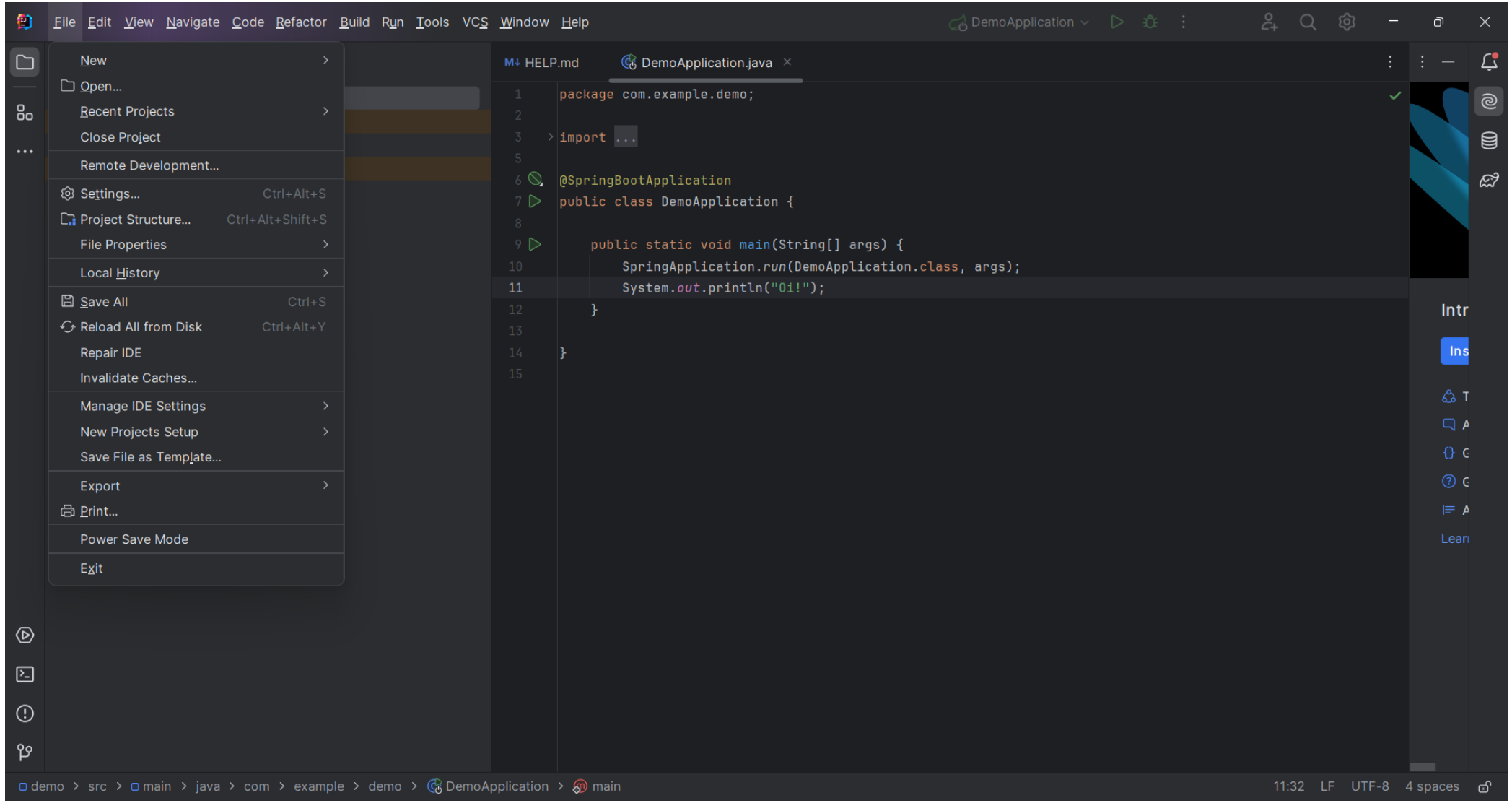


Gerenciando dependências manipulando o pow.xml

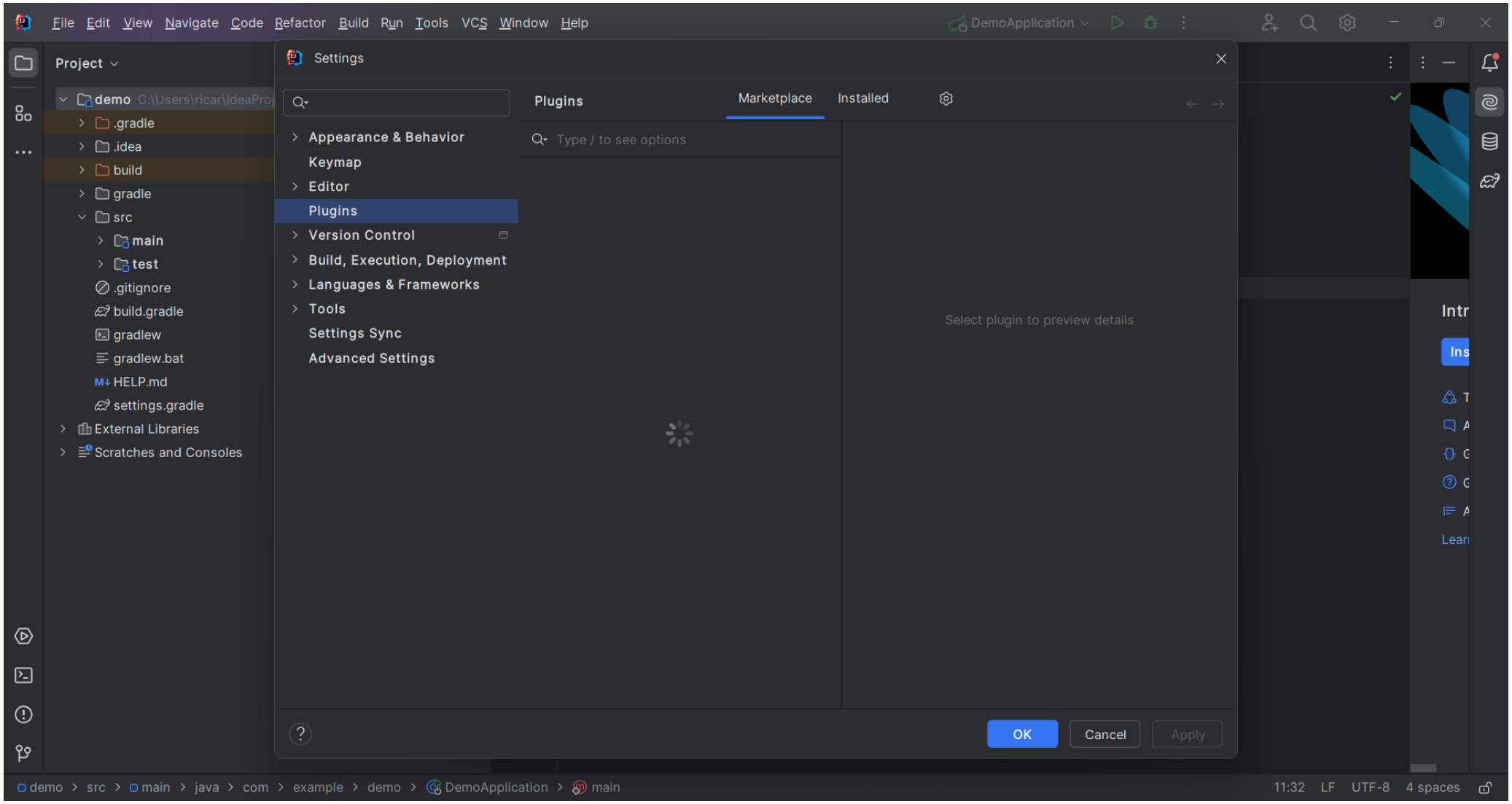


Criando projeto Spring boot diretamente no IntelliJ

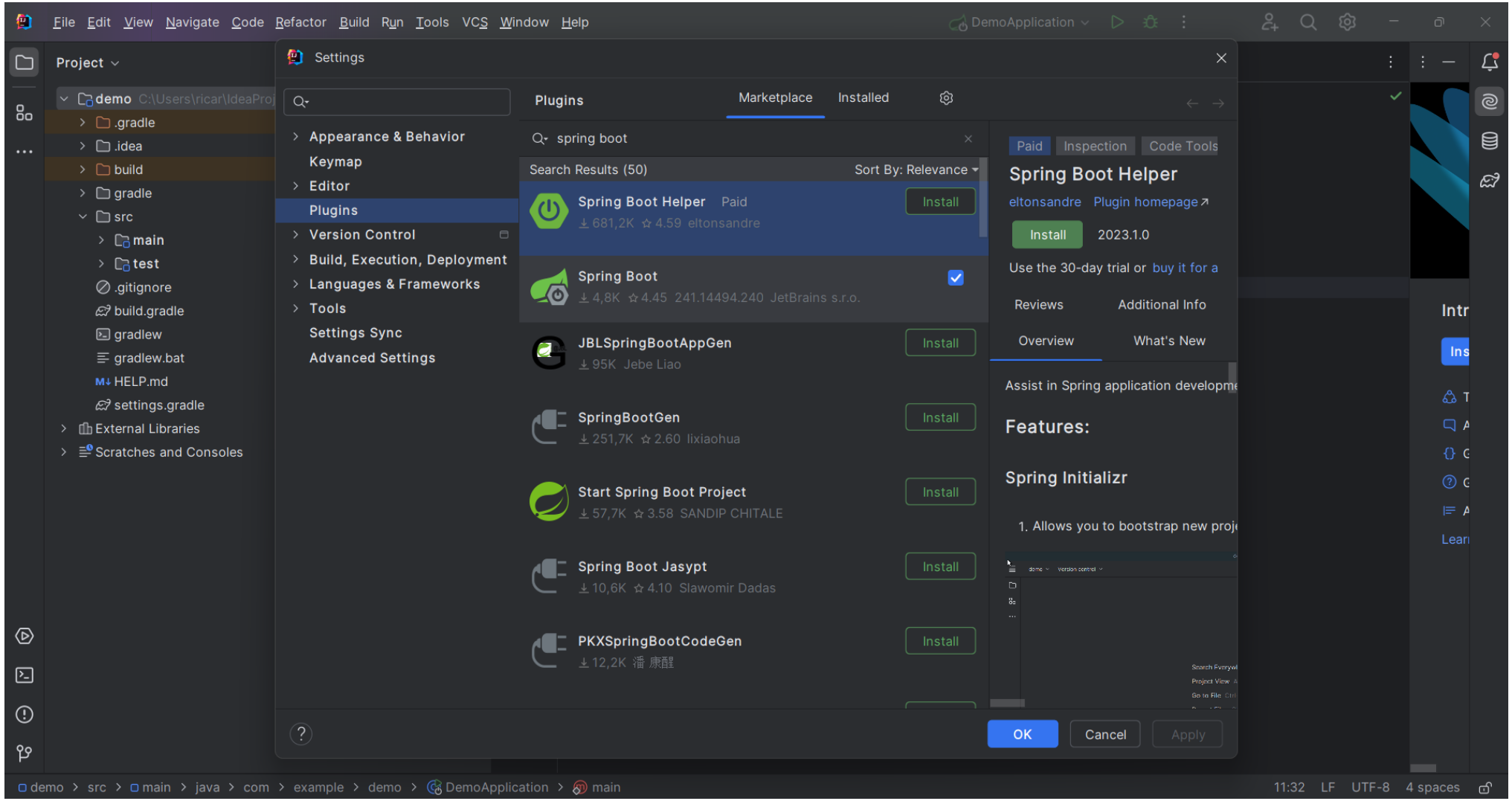
Criando projeto Spring boot diretamente no IntelliJ



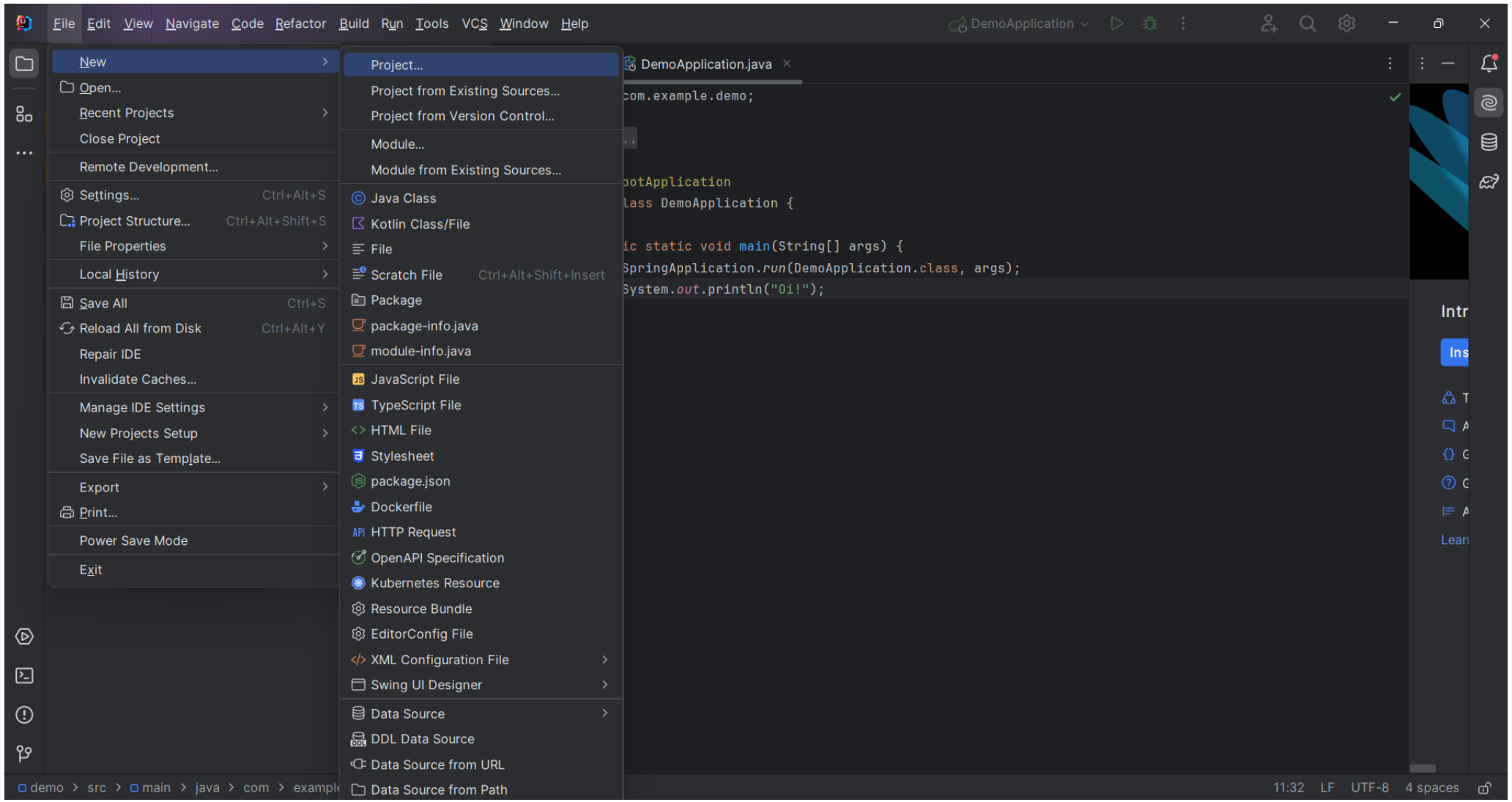
Criando projeto Spring boot diretamente no IntelliJ



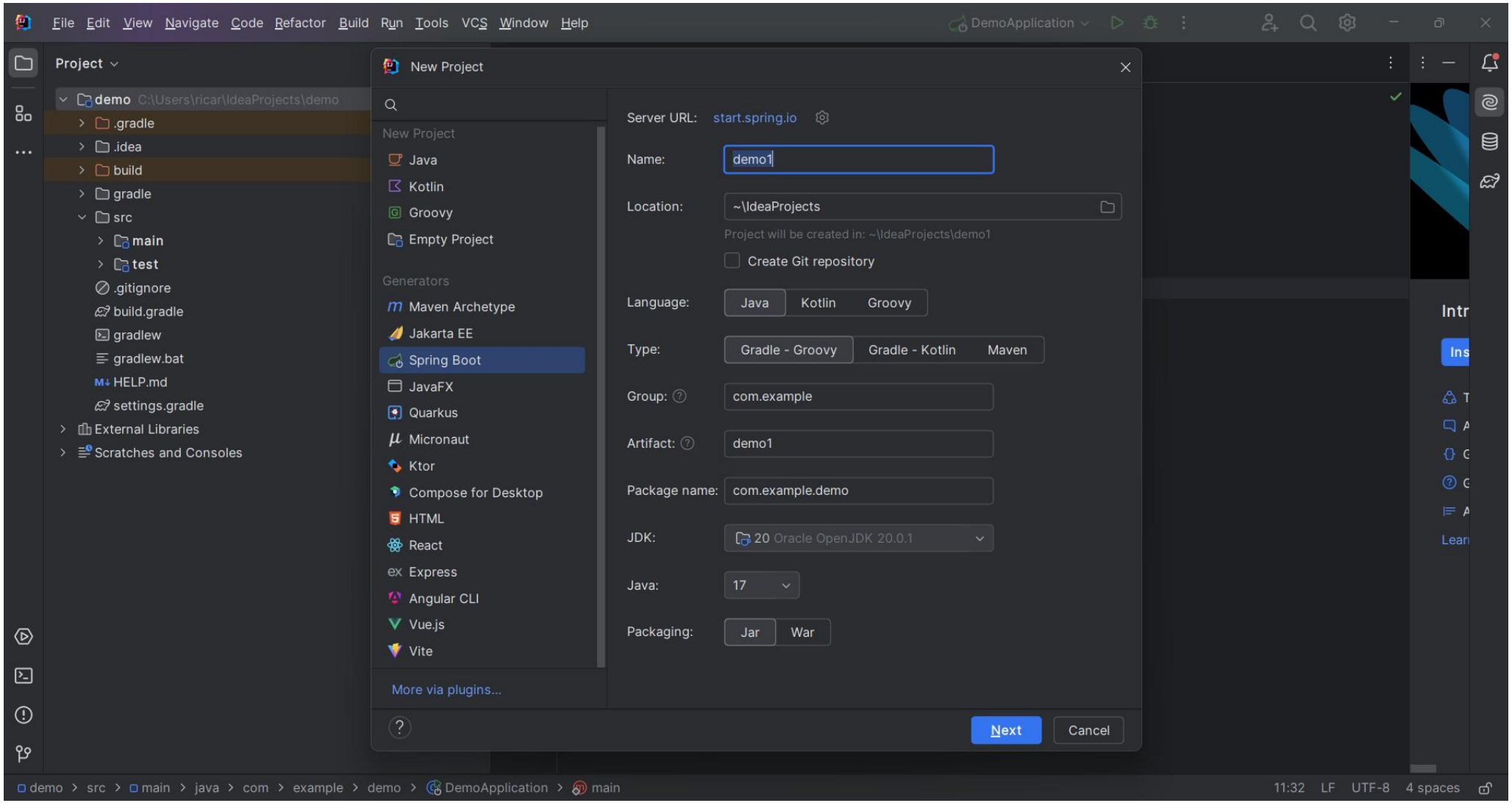
Criando projeto Spring boot diretamente no IntelliJ



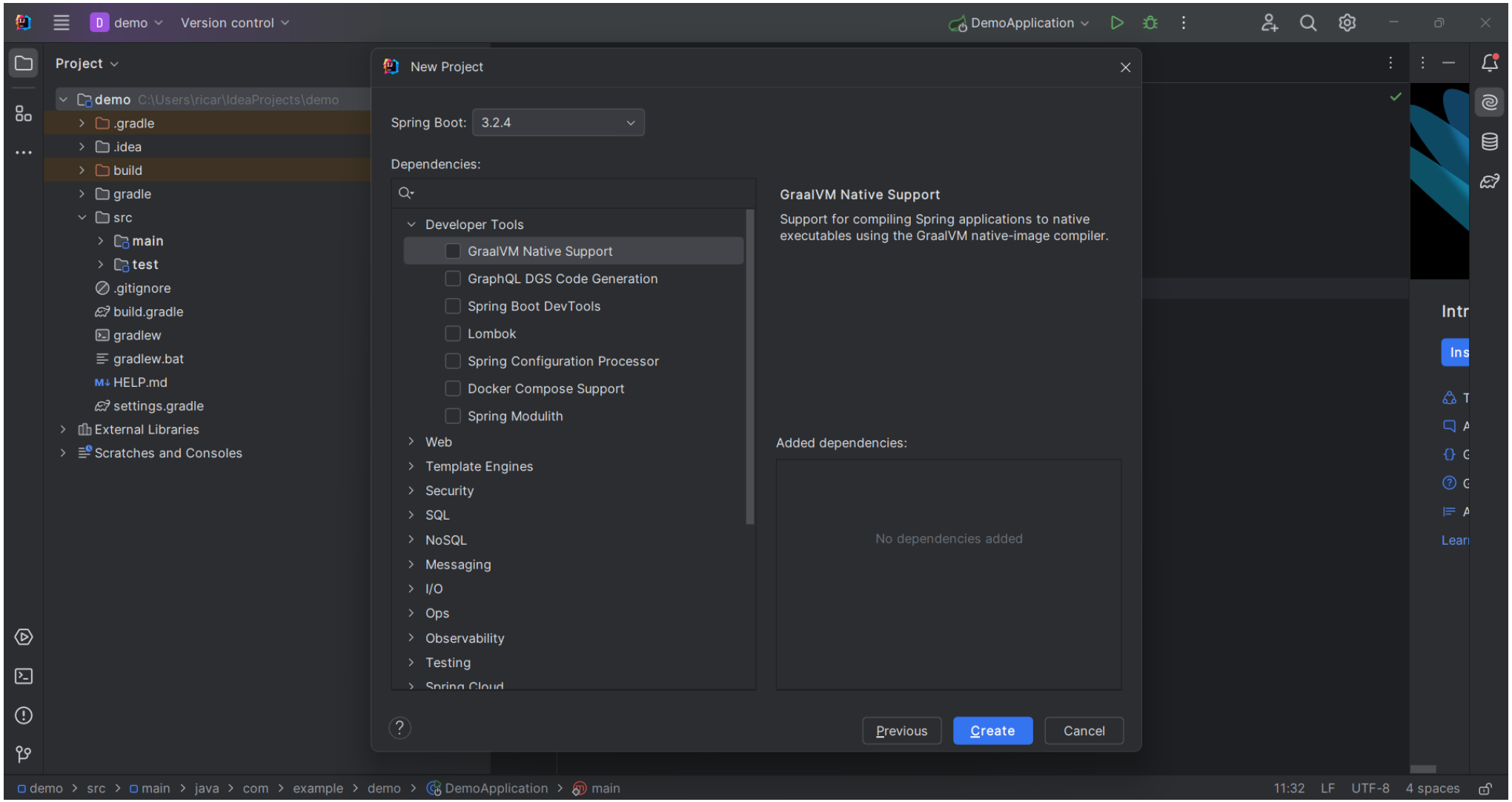
Criando projeto Spring boot diretamente no IntelliJ



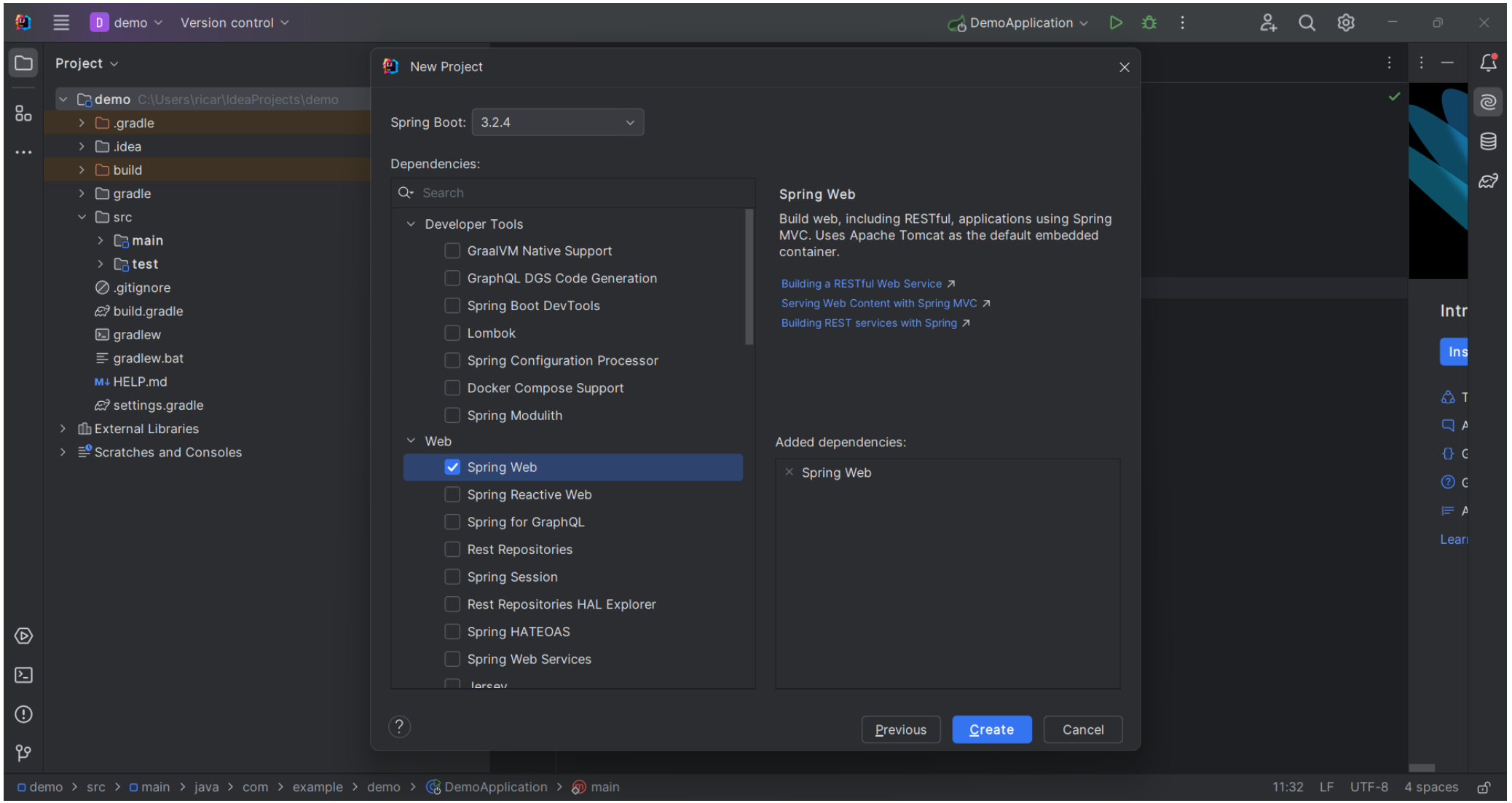
Criando projeto Spring boot diretamente no IntelliJ



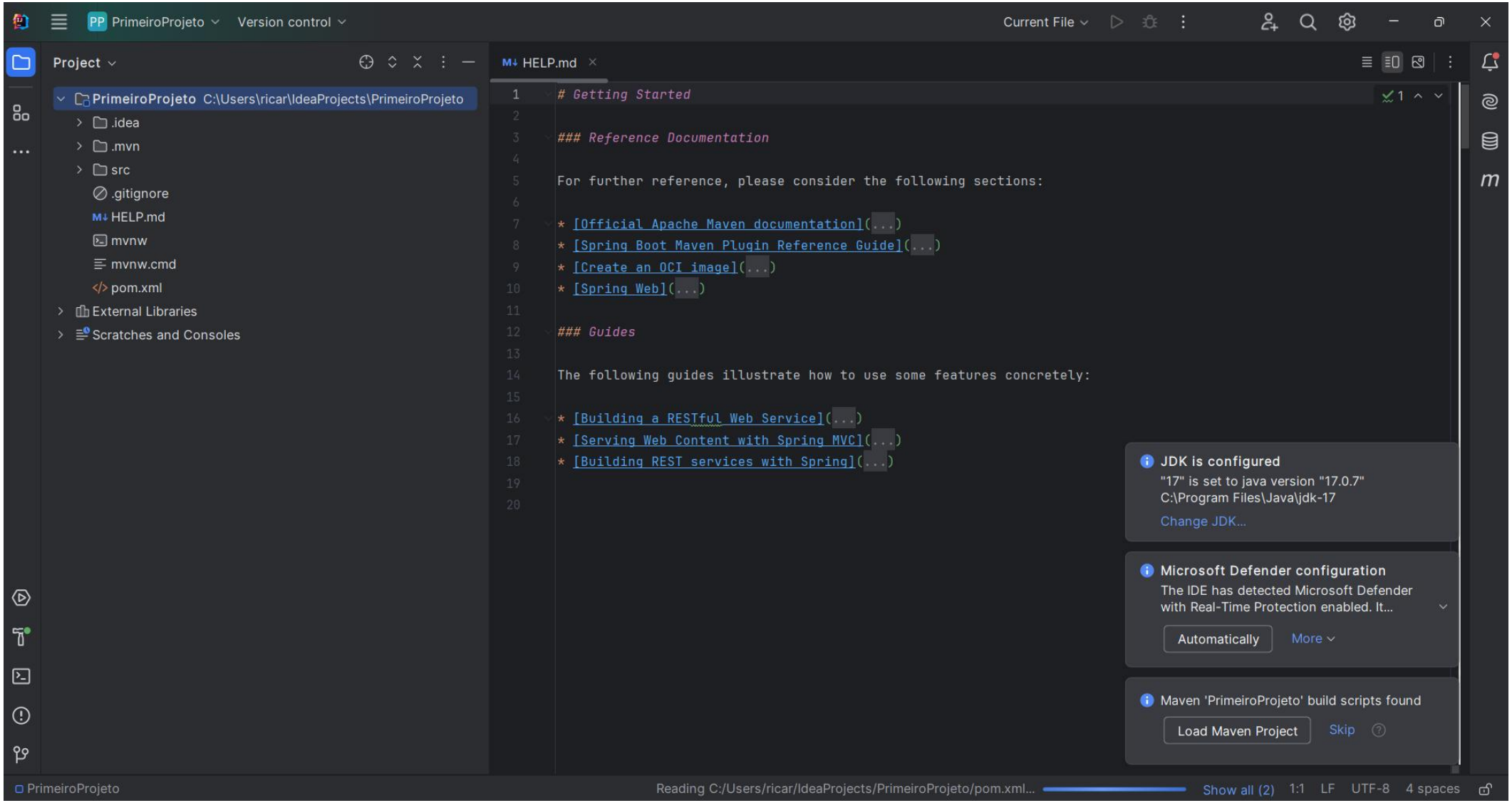
Criando projeto Spring boot diretamente no IntelliJ



Criando projeto Spring boot diretamente no IntelliJ



Criando projeto Spring boot diretamente no IntelliJ



Mostrando mensagem na inicialização da aplicação

DemoApplication.java

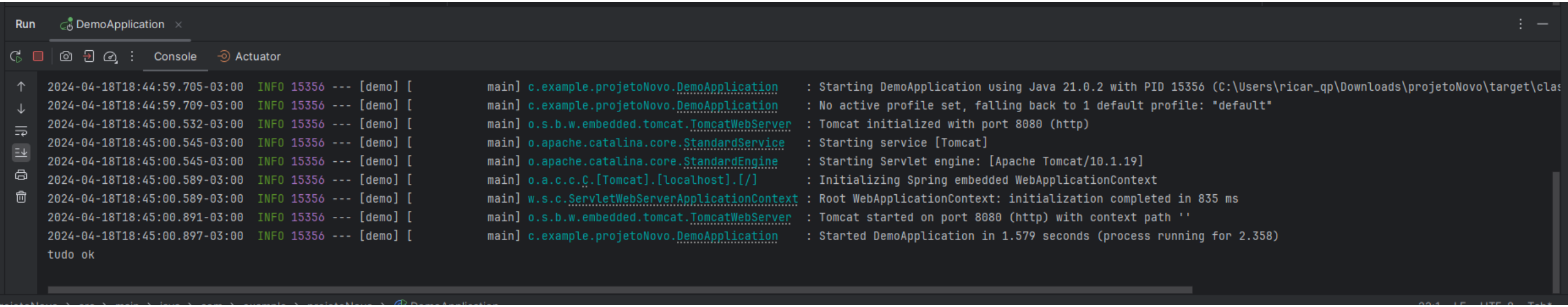
```java
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

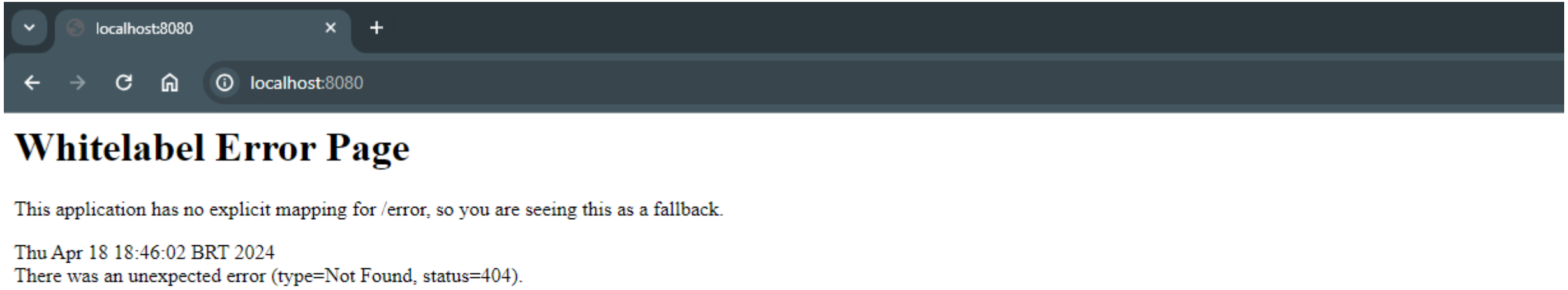
    @Bean
    CommandLineRunner init(){
        return args -> {
            System.out.println("tudo ok");
        };
    }
}
```

Mostrando mensagem na inicialização da aplicação



Mostrando mensagem na inicialização da aplicação

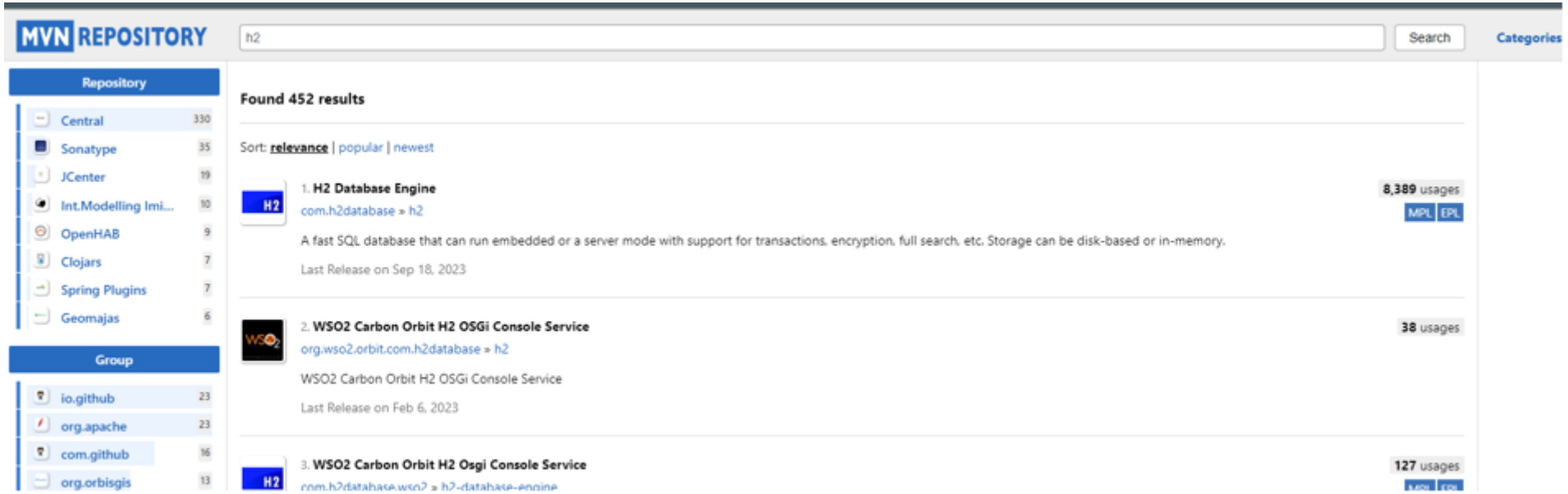
- Mas aqui, ainda não tem nada..



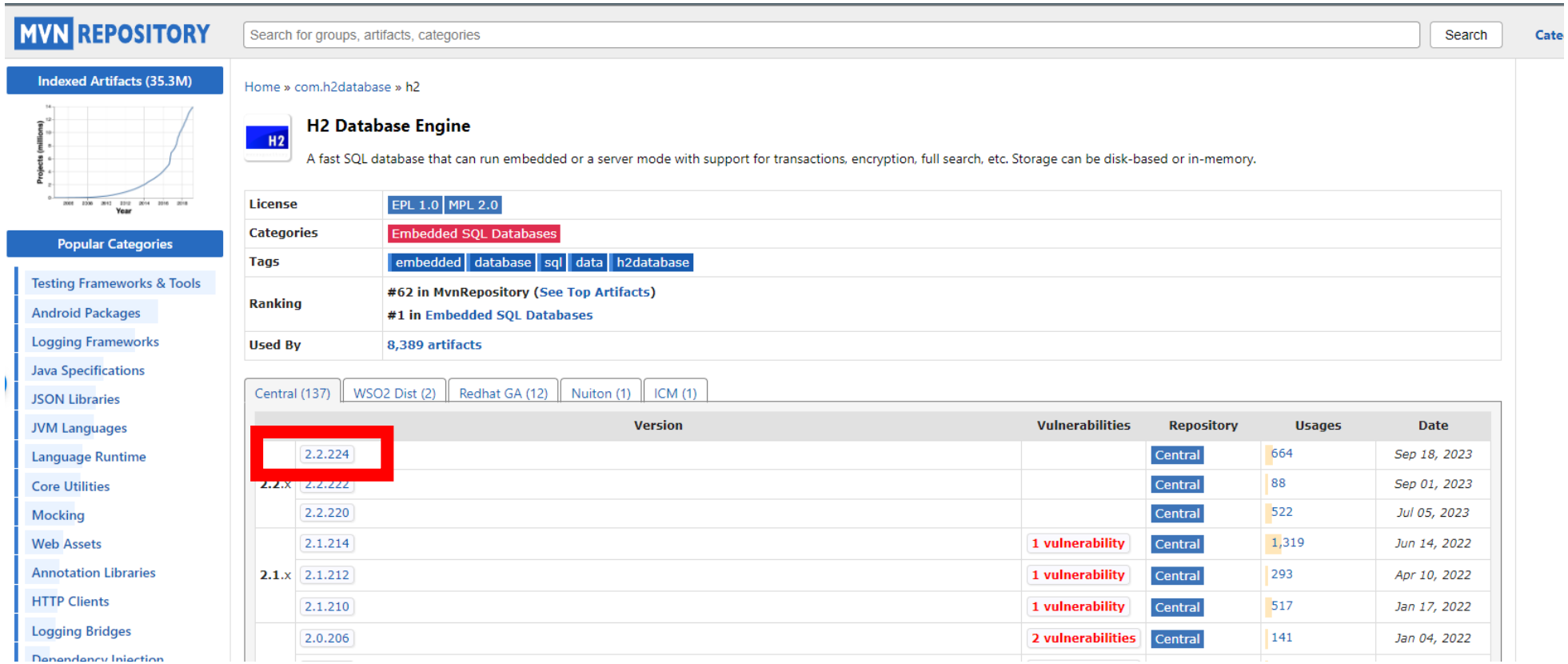
Gerenciando dependências manipulando o pow.xml

- Vamos adicionar o H2 Database Engine
  - É um sistema de gerenciamento de banco de dados relacional (SGBDR) escrito em Java.
    - Ele é conhecido por ser leve, rápido e fácil de usar.
  - O H2 pode ser executado em vários modos, incluindo em memória, o que o **torna ideal para testes e desenvolvimento**, bem como para aplicações que exigem um banco de dados embutido.

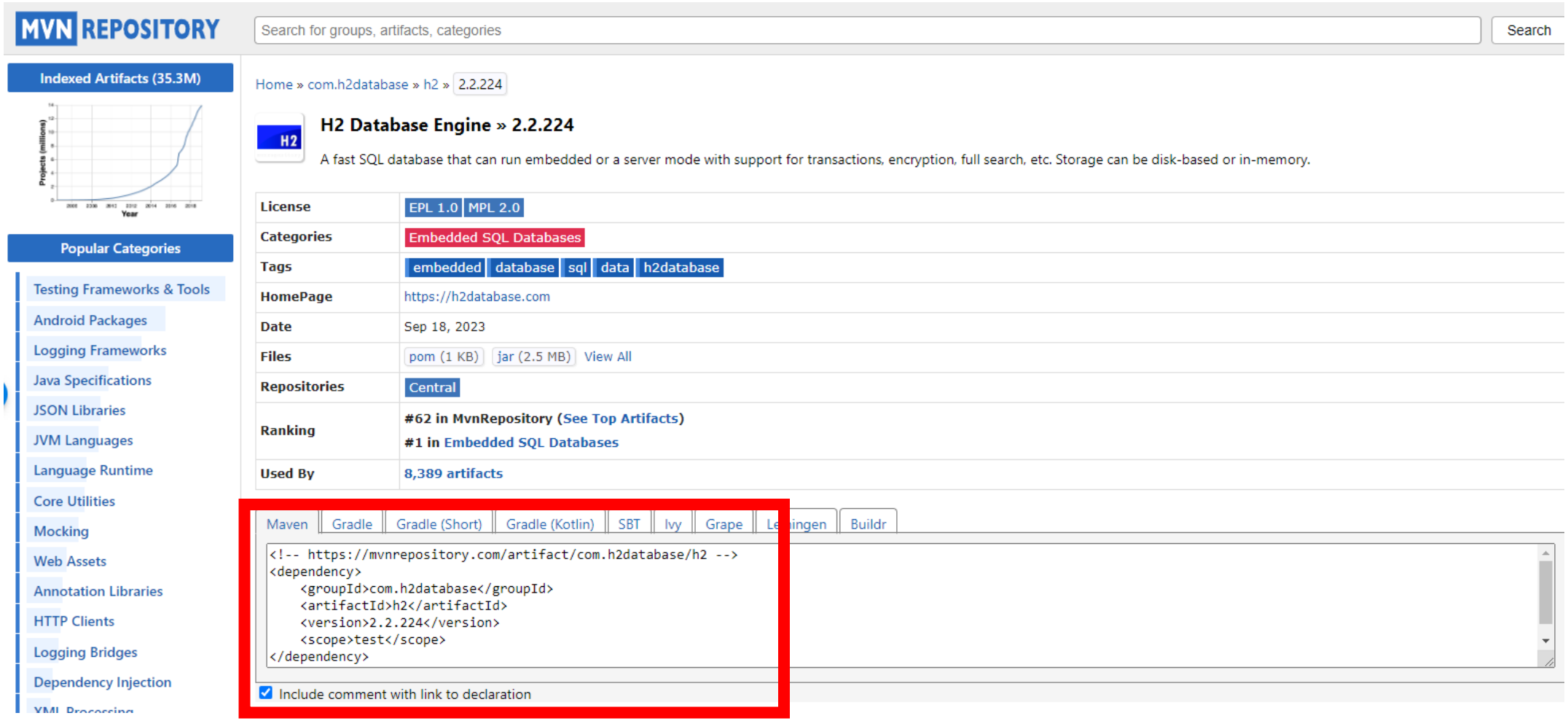
Gerenciando dependências manipulando o pow.xml



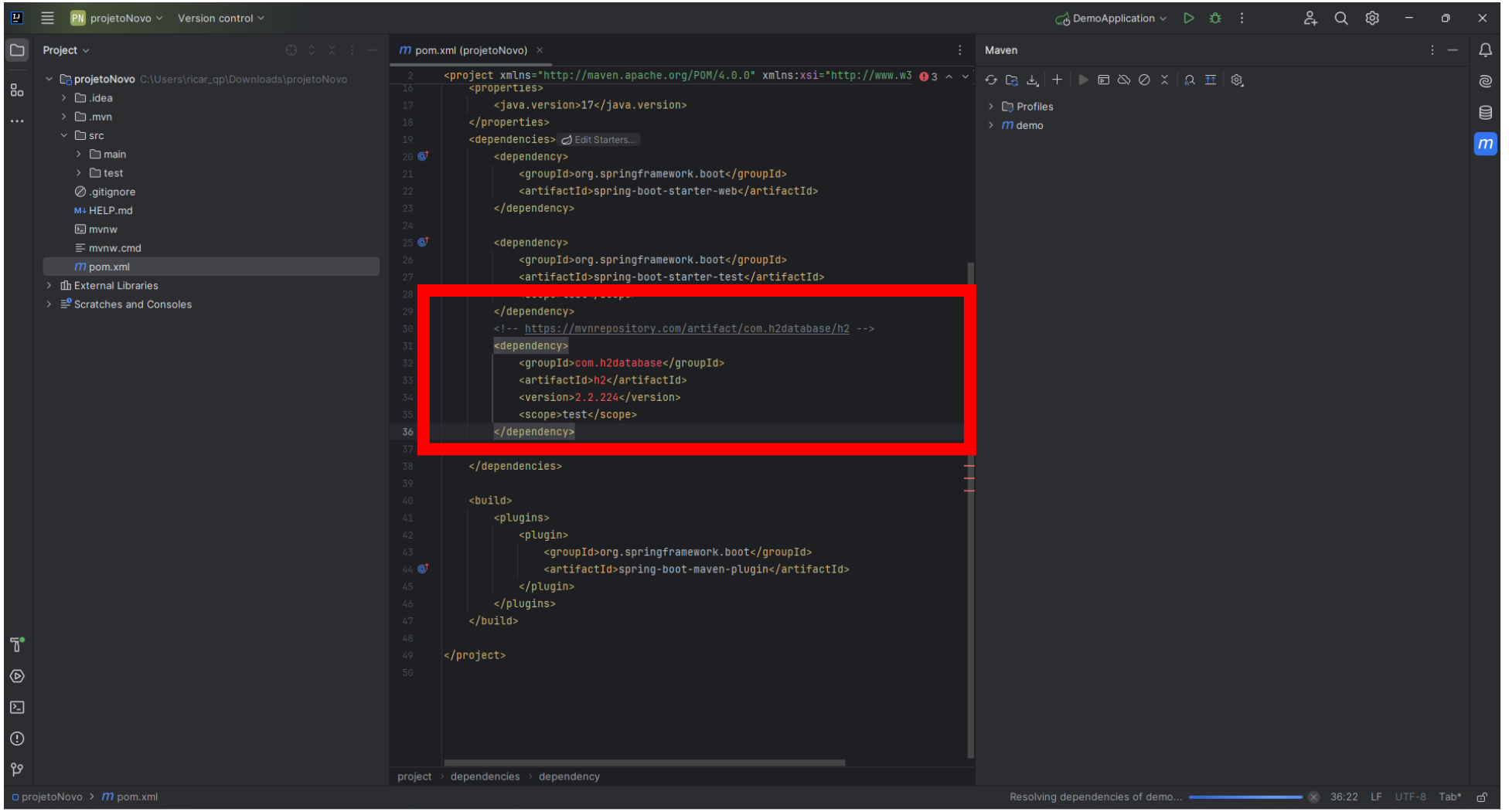
Gerenciando dependências manipulando o pow.xml



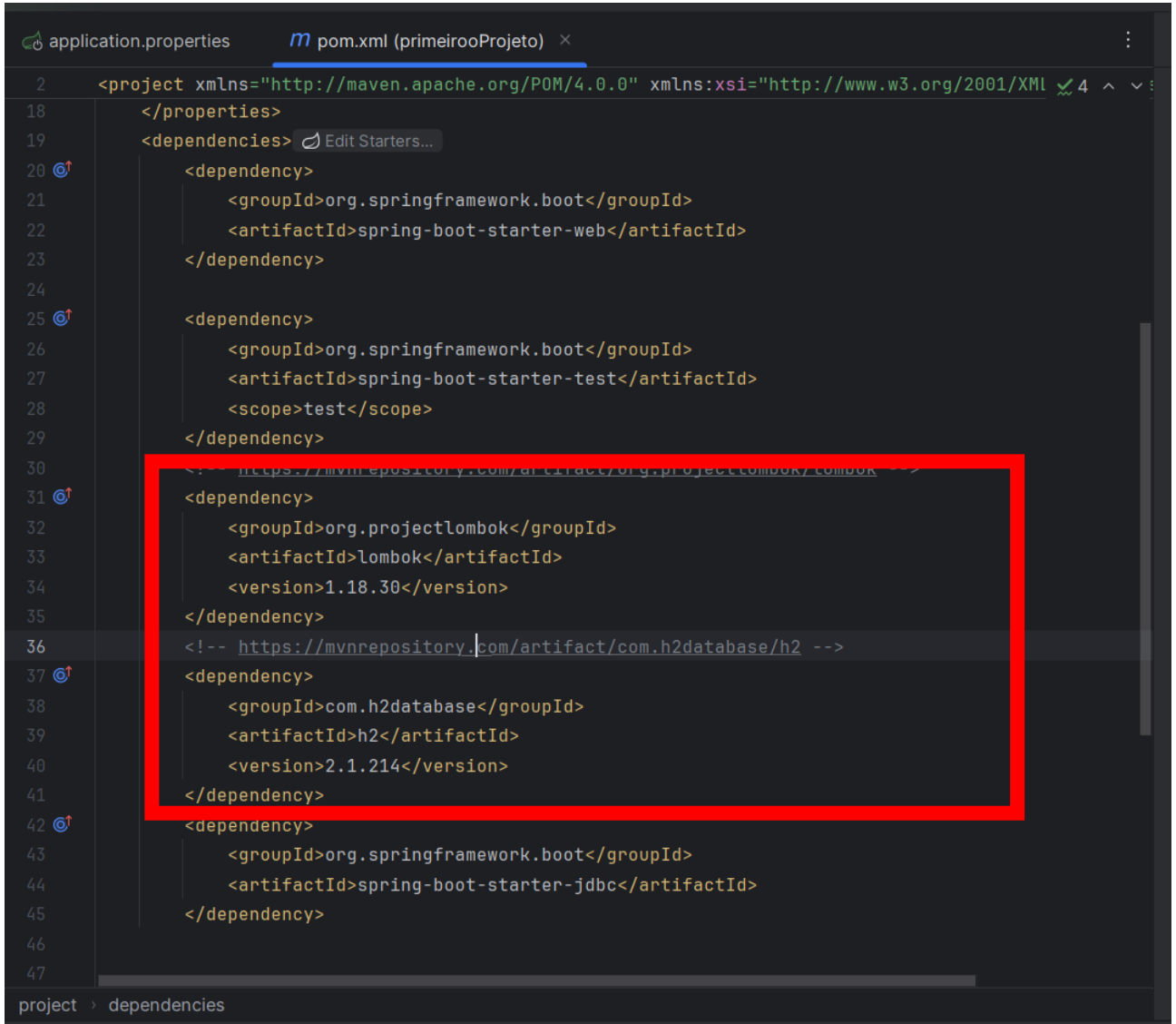
Gerenciando dependências manipulando o pow.xml



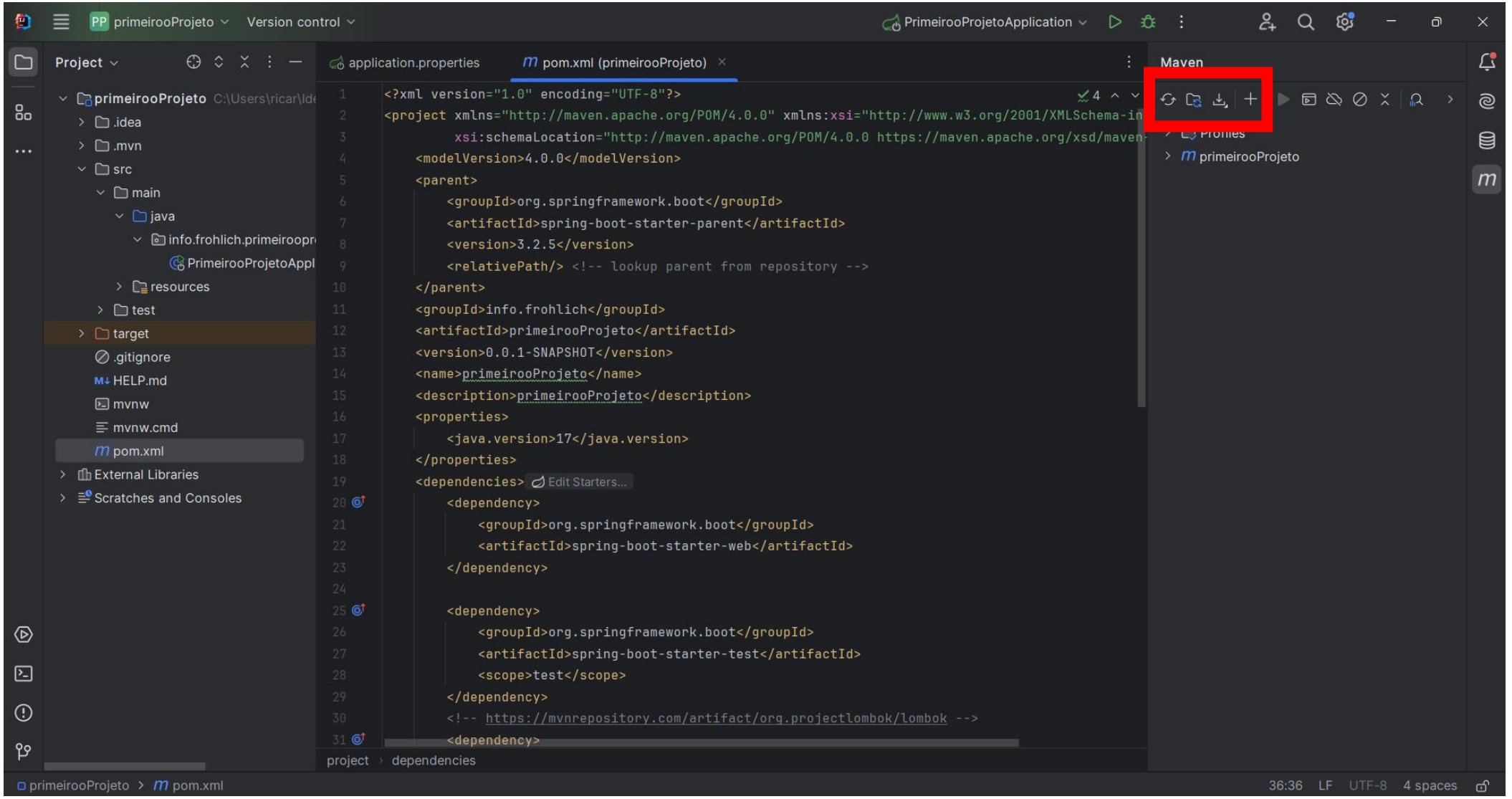
Gerenciando dependências manipulando o pow.xml



Gerenciando dependências manipulando o pow.xml



Gerenciando dependências manipulando o pow.xml



Recurso de autoconfiguração do Spring Boot

- Spring Boot oferece um recurso chamado autoconfiguração, que reduz ou elimina a necessidade de escrever código repetitivo ao configurar sua aplicação.

- O Spring Boot utiliza a autoconfiguração para configurar automaticamente diversos aspectos da sua aplicação com base nas dependências presentes no classpath.

- Autoconfiguração do Spring Boot possibilita que você utilize:

  - Dependências do Starter: O Spring Boot fornece "starters" para várias tecnologias comuns, como Spring MVC, Spring Data JPA, Spring Security, entre outros. Esses starters incluem dependências pré-configuradas para a tecnologia escolhida, o que simplifica muito a configuração da sua aplicação.

  - Autoconfiguração de Bancos de Dados: Se você estiver usando um banco de dados, o Spring Boot pode configurar automaticamente o DataSource e o EntityManagerFactory com base nas propriedades definidas no arquivo application.properties ou application.yml. Isso elimina a necessidade de configurar manualmente esses componentes.

Recurso de autoconfiguração do Spring Boot

- Autoconfiguração do Spring Boot possibilita que você utilize:
  - Autoconfiguração de Web MVC: Se você estiver construindo uma aplicação web, o Spring Boot pode configurar automaticamente o DispatcherServlet, mapeamentos de URL, conversores de mensagem, etc., com base nas dependências e configurações encontradas.
  - Autoconfiguração de Segurança: O Spring Boot pode configurar automaticamente a segurança da sua aplicação com base nas dependências e configurações presentes. Você pode personalizar facilmente a segurança adicionando suas próprias regras e configurações.
  - Autoconfiguração de Logging: O Spring Boot pode configurar automaticamente o sistema de logging da sua aplicação com base nas dependências presentes. Por exemplo, se o Logback estiver no classpath, o Spring Boot configurará automaticamente o Logback para você.

Configurando o H2

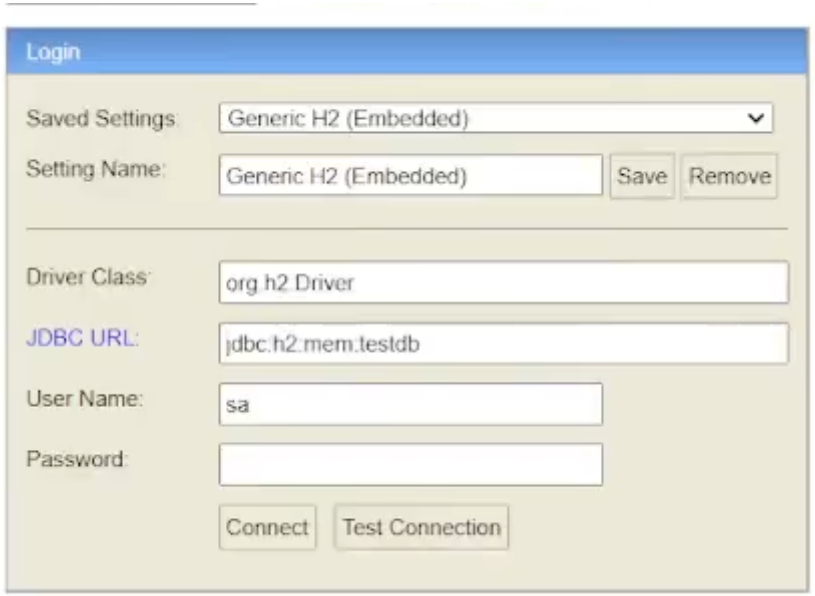
- Aqui é um exemplo de como o Spring Boot pode auxiliar com o recurso de autoconfiguração

```
spring.application.name=projetoNovo

spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Configurando o H2



Recurso de autoconfiguração do Spring Boot

- Podemos configurar o nível de log sem precisar configurar manualmente o Logback ou Log4j.

```
13    # Configuração do nível de log para o pacote com.example
14    logging.level.com.example=DEBUG
```

APIs RESTful

- As APIs RESTful, seguindo o estilo arquitetural REST (*Representational State Transfer*), são projetadas para serem eficientes, confiáveis e fáceis de escalar, utilizando a web existente de forma mais eficaz.

- Elas têm várias características distintas que as tornam uma escolha popular para desenvolvimento web moderno:

  - Uso de Métodos HTTP Padrão: APIs RESTful usam métodos padrão fornecidos pelo protocolo HTTP, como GET, POST, PUT, DELETE e PATCH, de maneira apropriada para realizar operações CRUD (Create, Read, Update, Delete). Cada um desses métodos tem um propósito específico:

    - GET é usado para recuperar dados.

    - POST é usado para criar novos recursos.

    - PUT é usado para atualizar recursos existentes.

    - DELETE é usado para excluir recursos.

    - PATCH é usado para fazer atualizações parciais em um recurso.

Spring Boot

- Spring Boot é uma ferramenta poderosa que simplifica o processo de configuração e desenvolvimento de novas aplicações Spring, oferecendo várias vantagens significativas para o desenvolvimento rápido:

  - Configuração Automática: Spring Boot tenta automaticamente configurar seu projeto com base nas dependências que você adicionou ao projeto. Por exemplo, se o Spring Data JPA está no classpath e você não configurou manualmente um DataSource, Spring Boot configura uma para você.

  - Starters e Dependências: Os starters do Spring Boot são templates que contêm um conjunto de dependências pré-configuradas que você pode incluir no seu projeto para que este funcione imediatamente. Isso elimina a necessidade de definir e gerenciar versões compatíveis para cada biblioteca.

  - Servidor Embarcado: Spring Boot vem com servidores de aplicação embutidos como Tomcat, Jetty ou Undertow, para que você não precise configurar um servidor web separadamente. Isso facilita o desenvolvimento e o teste de aplicações web rapidamente.

  - Prontidão para Produção: Spring Boot oferece recursos como métricas, verificações de saúde e configuração externa que são essenciais para aplicações prontas para a produção. Isso torna mais fácil ir do desenvolvimento para a produção sem problemas adicionais de configuração.

  - Sem Código XML: Não é necessário escrever longos arquivos de configuração XML que eram comuns em versões anteriores do Spring. Configurações podem ser feitas usando anotações e arquivos de propriedades.

Arquitetura do Spring e Seu Ecossistema

- A arquitetura do Spring Framework é modular, consistindo em vários projetos que fornecem uma ampla gama de funcionalidades:
  - **Spring Core:** No coração do Spring, oferecendo recursos fundamentais como injeção de dependência e inversão de controle.
  - **Spring AOP:** Suporta a programação orientada a aspectos, incluindo a separação de preocupações operacionais e técnicas, que facilita a manutenção do código.
  - **Spring Data:** Simplifica o trabalho de persistência de dados, trabalhando com bancos de dados e operações de dados sem a necessidade de escrever muito código.
  - **Spring MVC:** Um framework web que usa o modelo MVC (Model-View-Controller) para projetar e desenvolver aplicações web.
  - **Spring Security:** Oferece segurança robusta para aplicações Spring, tanto em nível de autenticação quanto de autorização.
  - **Spring Cloud:** Facilita o desenvolvimento de alguns dos padrões comuns em sistemas distribuídos (por exemplo, configuração centralizada, registro de serviço, roteamento, etc.).

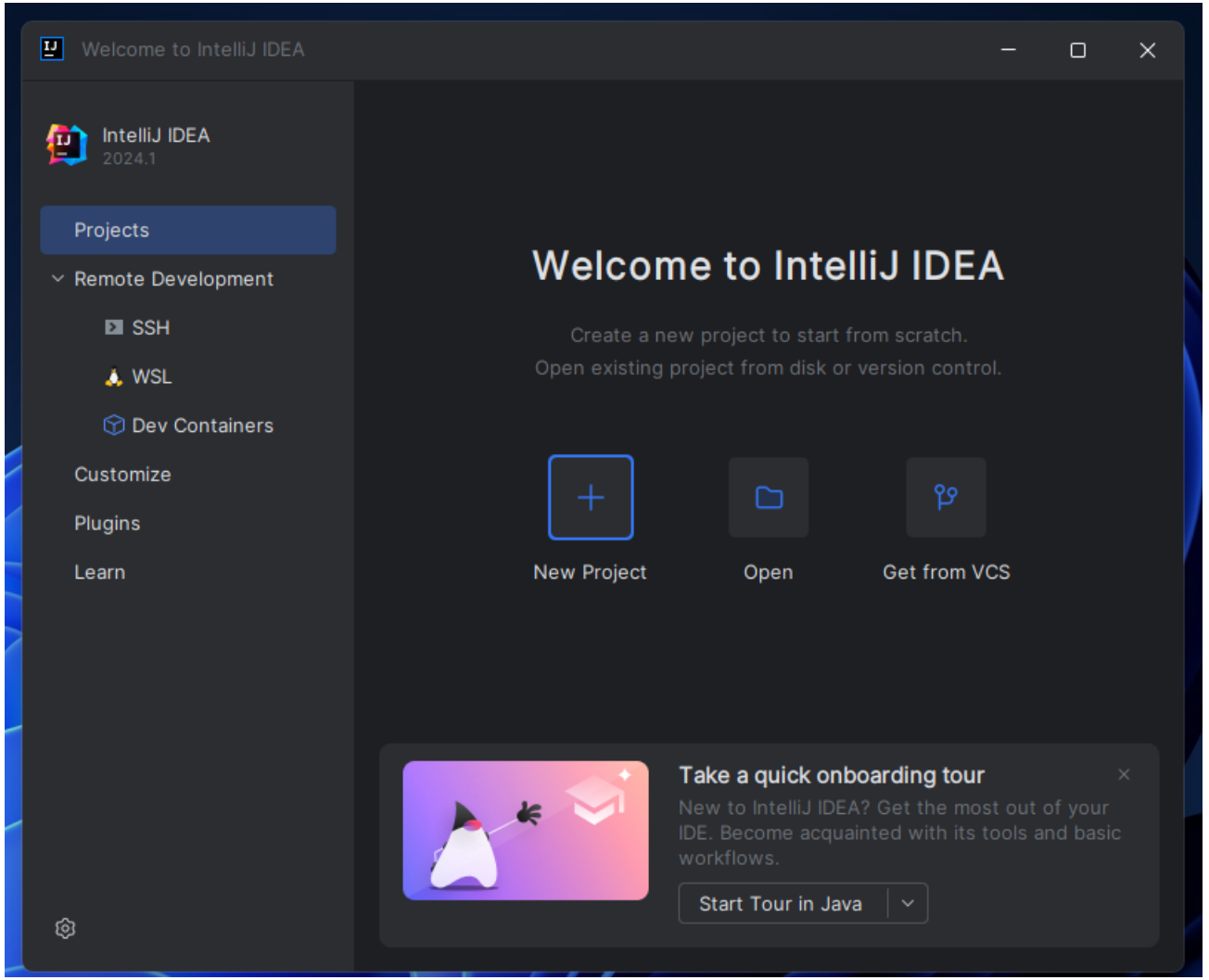
Visão Geral sobre Spring Data, Spring MVC e Spring Security

- Spring Data
  - Simplifica a persistência de dados, oferecendo uma maneira mais simples e consistente de acessar bancos de dados usando padrões de acesso a dados, incluindo suporte para JPA, MongoDB, Cassandra, Redis, e muitos outros.
  - Fornece uma abstração para reduzir a quantidade de código repetitivo necessário para operações de banco de dados.
- Spring MVC
  - Framework de aplicação web que implementa o modelo MVC. Permite construir aplicações web de maneira flexível com o suporte para integração com outras tecnologias de visualização de dados, como JSP e Thymeleaf.
  - Gerencia solicitações HTTP, a interação com modelos de dados e a renderização de vistas.

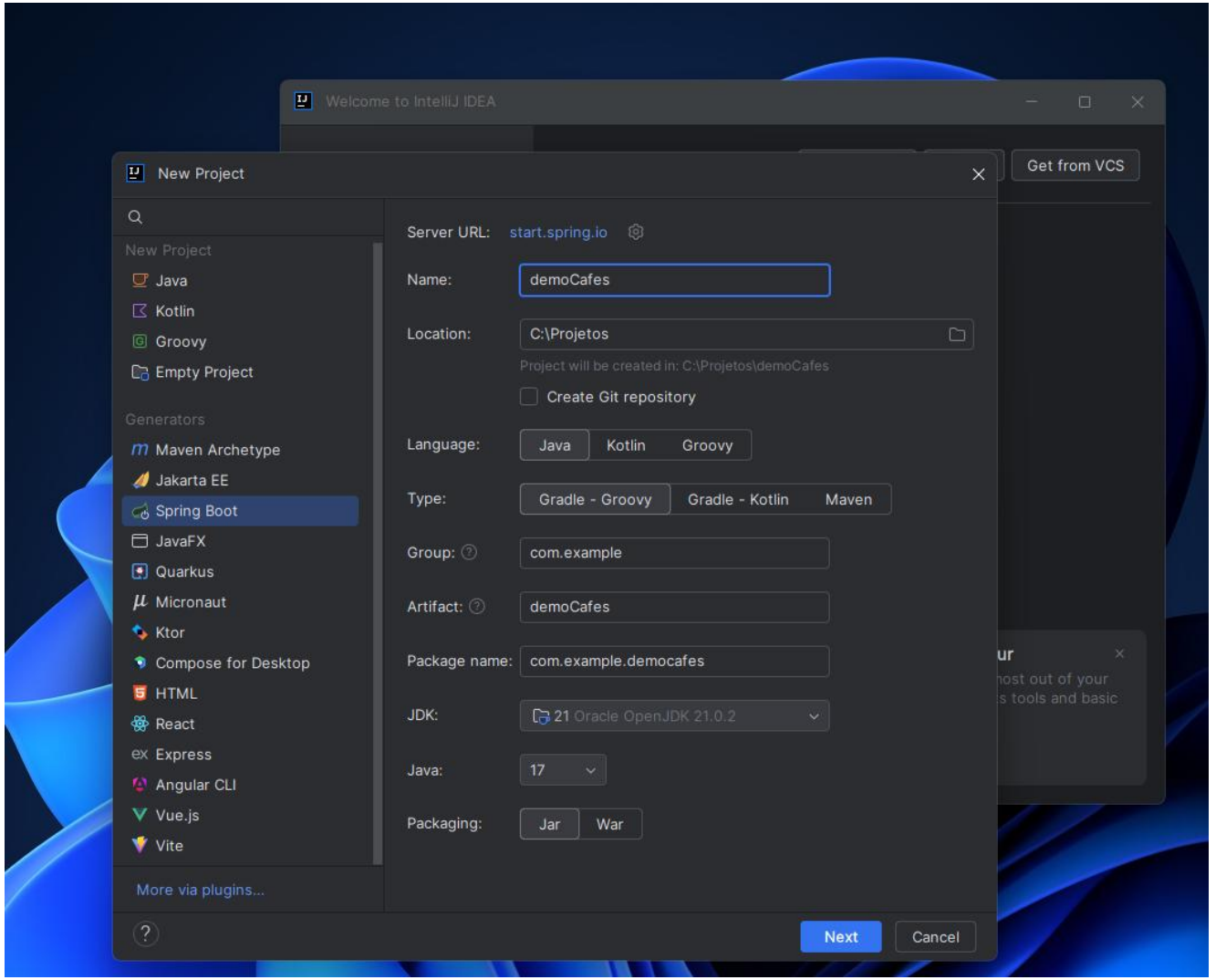
Visão Geral sobre Spring Data, Spring MVC e Spring Security

- Spring Security
  - Fornece autenticação e autorização, protegendo aplicações contra ameaças comuns como sessões fixas, clickjacking, e cross site request forgery (CSRF).
  - É altamente personalizável para atender a requisitos de segurança complexos e pode ser facilmente estendido para atender normas específicas de segurança.
  - Esses componentes do Spring fazem dele uma escolha poderosa para desenvolvedores que buscam uma plataforma robusta, segura e de fácil manutenção para aplicações empresariais.

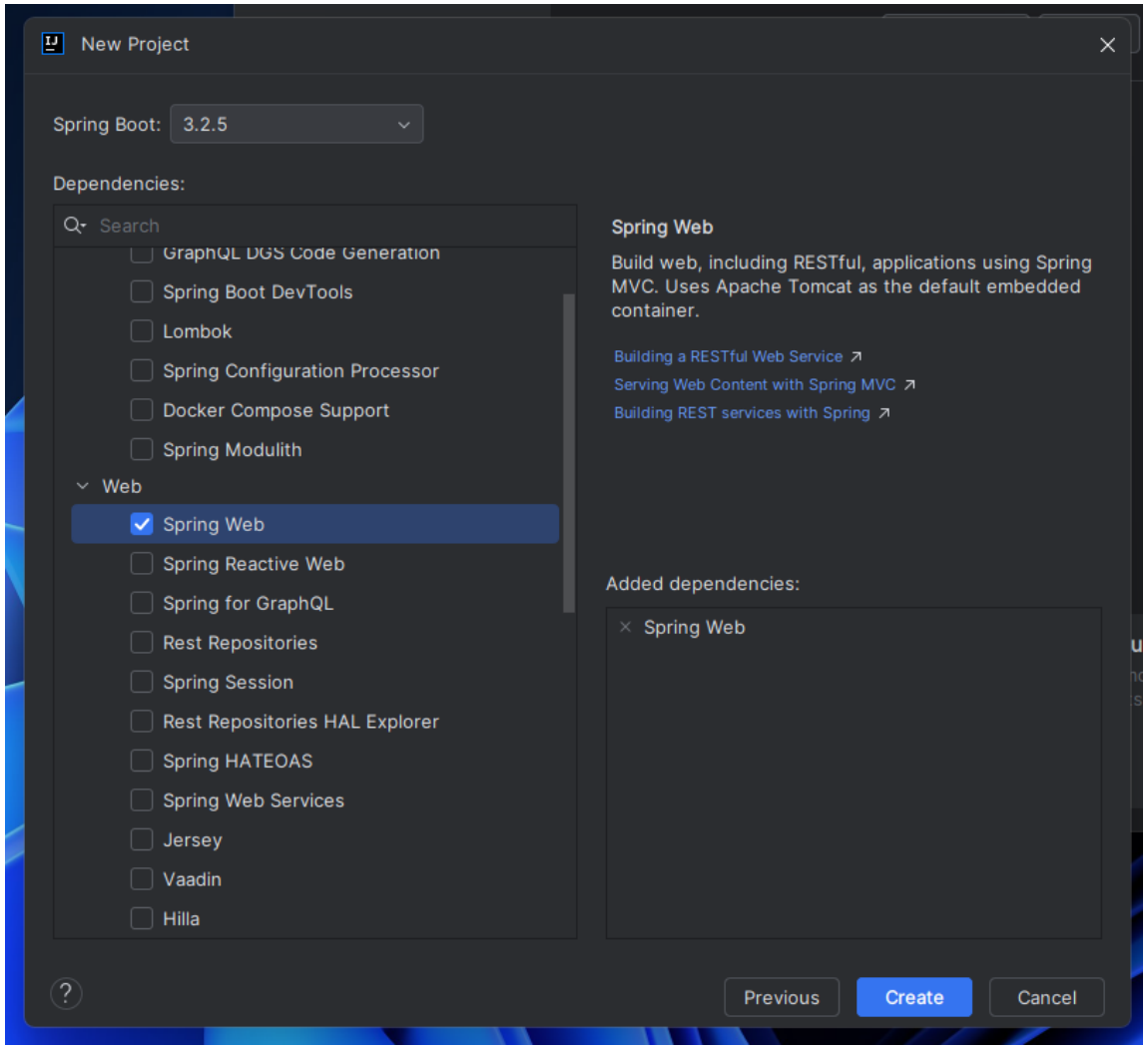
Criando uma API simples



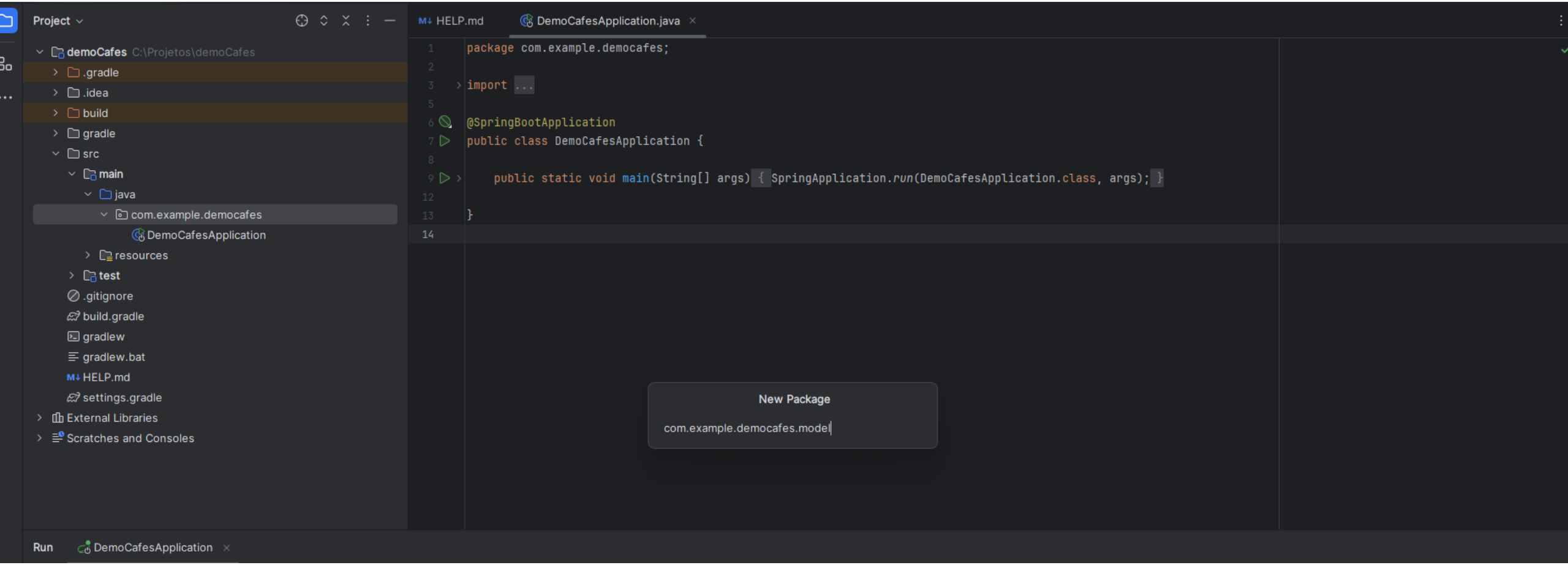
Criando uma API simples



Criando uma API simples



Criando uma API simples



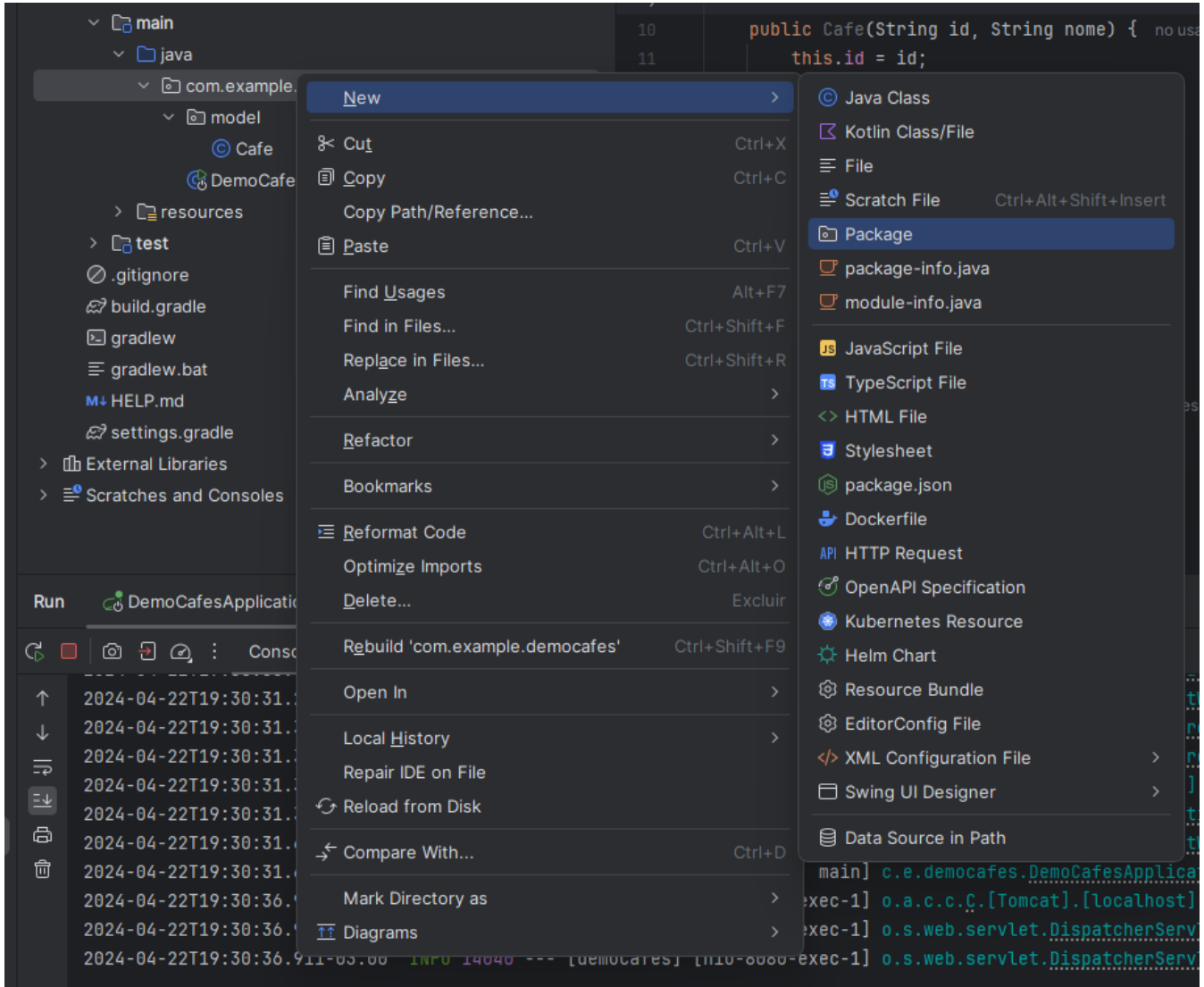
Criando uma API simples

```java
public class Cafe {  7 usages
    private int id;  2 usages
    private String nome;  3 usages

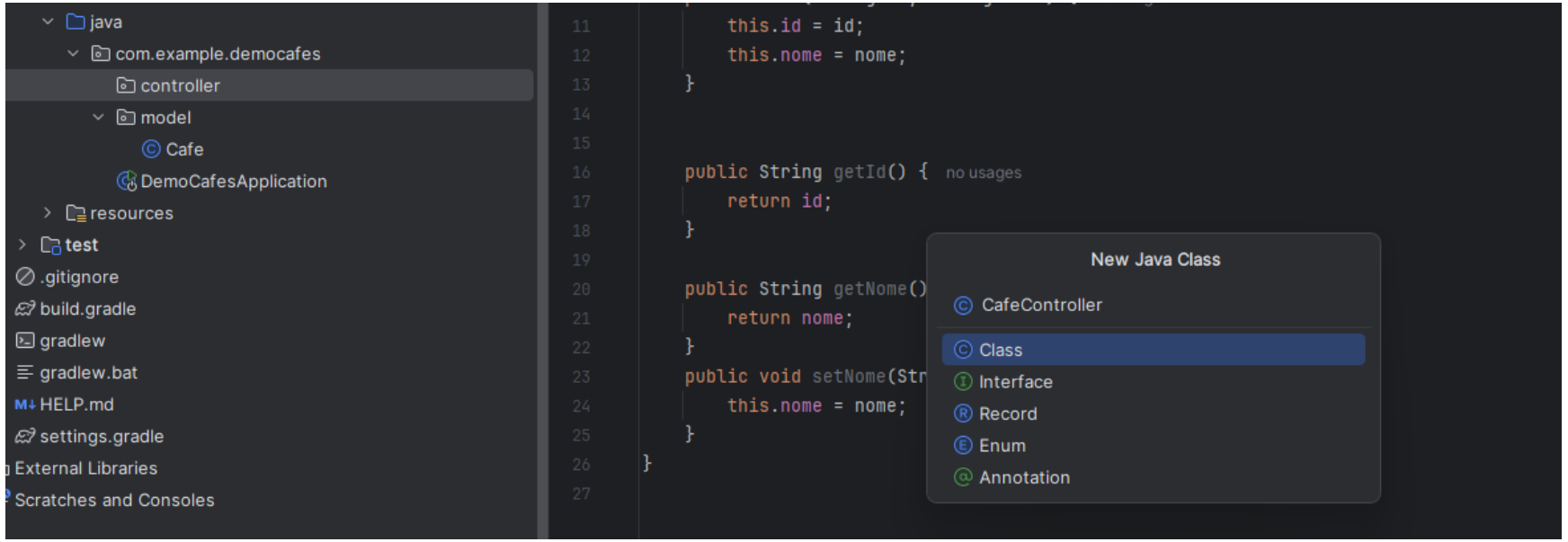

    public Cafe(int id, String nome) {  4 usages
        this.id = id;
        this.nome = nome;
    }
    public int getId() {  no usages
        return id;
    }

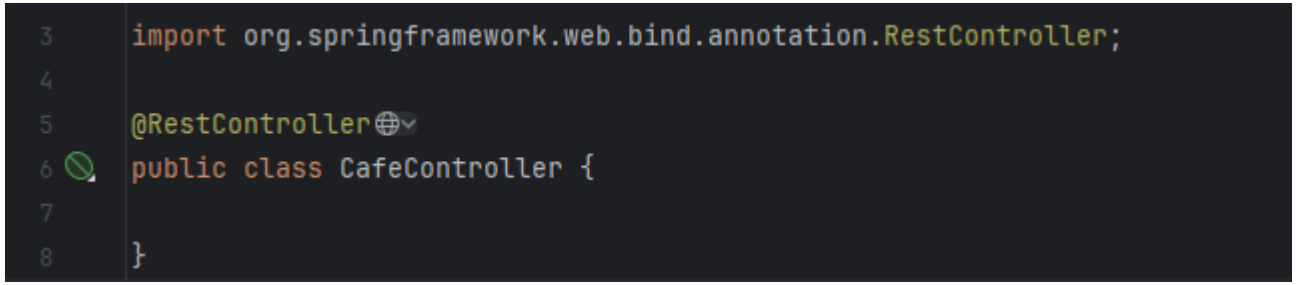
    public String getNome() {  no usages
        return nome;
    }
    public void setNome(String nome) {  no usages
        this.nome = nome;
    }
}
```

Criando uma API simples



Criando uma API simples





@RequestMapping

- @RequestMapping é uma anotação do Spring Framework em Java usada para mapear solicitações HTTP a métodos de manipulação dentro de um controlador (geralmente um controlador Spring MVC).

- Ele define a URL para acessar os métodos de um controlador e também especifica o tipo de solicitação HTTP que o método manipula.

Criando uma API simples

```java
@RestController
public class CafeController {
    private List<Cafe> cafes = new ArrayList<>();  4 usages


    public CafeController() {
        cafes.addAll(List.of(
                new Cafe( id: 1,  nome: "Café Cereza"),
                new Cafe( id: 2,  nome: "Café Ganador"),
                new Cafe( id: 3,  nome: "Café Lareño"),
                new Cafe( id: 4,  nome: "Café Três Pontas")
        ));
    }


    @GetMapping("/cafes")
    Iterable<Cafe> getCafes() {
        return cafes;
    }
```

Criando uma API simples

```java
@RestController
public class CafeController {
    private List<Cafe> cafes = new ArrayList<>();  4 usages


    public CafeController() {
        cafes.addAll(List.of(
                new Cafe( id: 1,  nome: "Café Cereza"),
                new Cafe( id: 2,  nome: "Café Ganador"),
                new Cafe( id: 3,  nome: "Café Lareño"),
                new Cafe( id: 4,  nome: "Café Três Pontas")
        ));
    }


    @GetMapping("/cafes")
    Iterable<Cafe> getCafes() {
        return cafes;
    }


    @GetMapping("/cafes/{id}")
    Optional<Cafe> getCafePorId(@PathVariable int id) {
        for (Cafe c: cafes) {
            if (c.getId() == id) {
                return Optional.of(c);
            }
        }
        return Optional.empty();
    }
```

Classe Optional

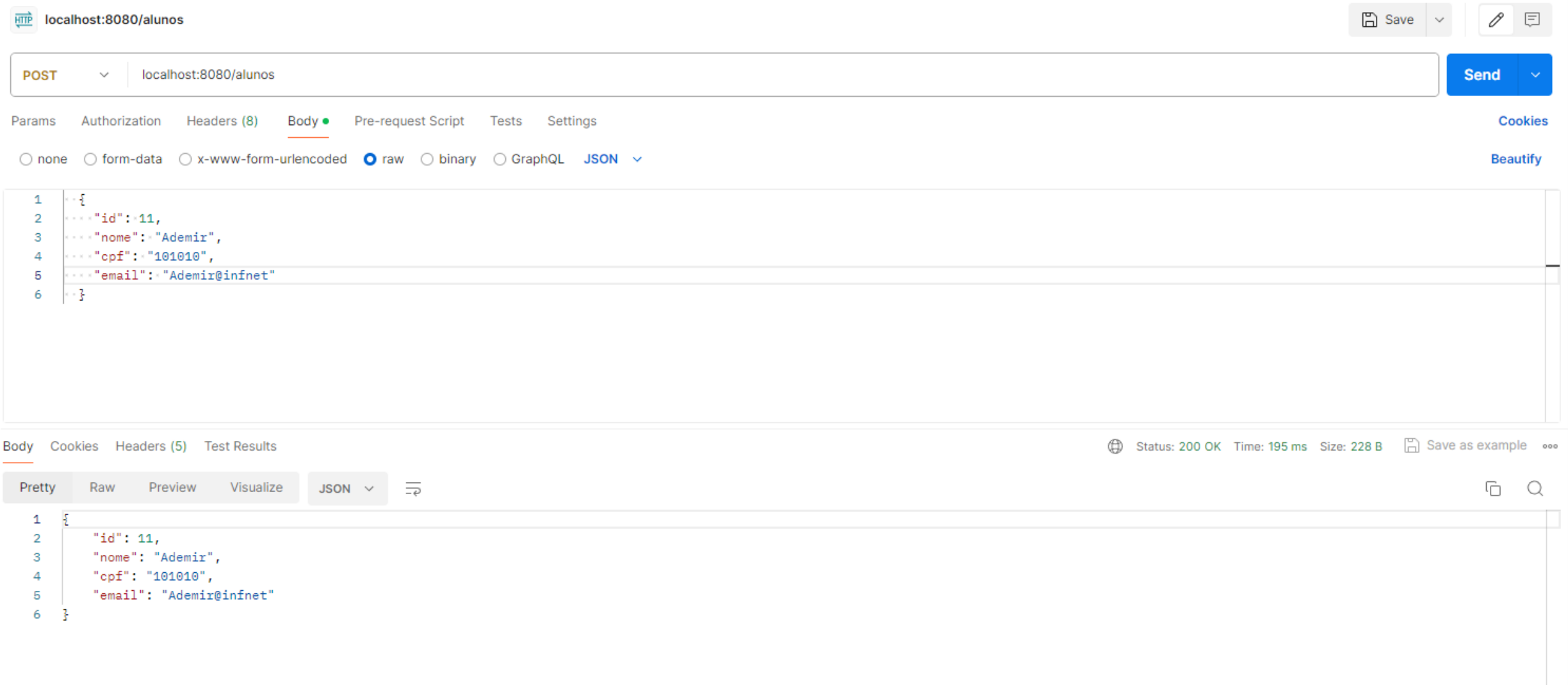
- Classe Optional é um contêiner que pode ou não conter um valor não nulo. Introduzida no Java 8, ela oferece uma abordagem melhor para lidar com null, evitando NullPointerExceptions e escrevendo códigos mais claros e limpos.

- O principal propósito do Optional é fornecer um tipo que encapsula um valor opcional, ou seja, um valor que pode estar presente ou ausente.

- Isso é particularmente útil em situações onde retornos de métodos podem não ter valor para retornar e normalmente retornariam null.

- Usando Optional, o método pode retornar um Optional vazio (sem valor) ao invés de null, tornando explícito para o consumidor do método que a ausência de valor é uma possibilidade.

Criando uma API simples



Adicionando dependências

```
25      <dependency>
26          <groupId>org.springframework.boot</groupId>
27          <artifactId>spring-boot-starter-test</artifactId>
28          <scope>test</scope>
29      </dependency>
30      <dependency>
31          <groupId>org.springframework.boot</groupId>
32          <artifactId>spring-boot-starter-data-jpa</artifactId>
33      </dependency>
34
35      <dependency>
36          <groupId>com.h2database</groupId>
37          <artifactId>h2</artifactId>
38          <scope>runtime</scope>
39      </dependency>
```

Classe Aluno

```java
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import org.springframework.boot.autoconfigure.domain.EntityScan;

@Entity  19 usages
public class Aluno {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String nome;  3 usages
    private String cpf;  3 usages
    private String email;  3 usages

    public Aluno(int id, String nome, String cpf, String email) {  9 usages
        this.id = id;
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
    }
```

Classe Aluno

```java
25      public Aluno() { //isso aqui é mandatório do JPA
26
27      }
28
29      public int getId() {  3 usages
30          return id;
31      }
32
33      public void setId(int id) {  no usages
34          this.id = id;
35      }
36
37      public String getNome() {  no usages
38          return nome;
39      }
40
41      public void setNome(String nome) {  no usages
42          this.nome = nome;
43      }
44
45      public String getCpf() {  no usages
46          return cpf;
47      }
```

Interface

```
4
5   interface AlunoRepository extends CrudRepository<Aluno, Integer>{  no usages
6
7   }
```

Construtor da Controller

```java
@RestController
public class AlunoController {
    private final AlunoRepository alunoRepository;  2 usages

    public AlunoController(AlunoRepository alunoRepository) {
        this.alunoRepository = alunoRepository;
        alunoRepository.saveAll(List.of(
                new Aluno( nome: "Antonio",  cpf: "123456789",  email: "antonio@infnet"),
                new Aluno( nome: "Charles",  cpf: "123456789",  email: "Charles@infnet"),
                new Aluno( nome: "Daniel",  cpf: "123456789",  email: "Daniel@infnet"),
                new Aluno( nome: "Fabio",  cpf: "123456789",  email: "Fabio@infnet"),
                new Aluno( nome: "Gabriel",  cpf: "123456789",  email: "Gabriel@infnet"),
                new Aluno( nome: "Jessica",  cpf: "123456789",  email: "Jessica@infnet"),
                new Aluno( nome: "Jose Augusto",  cpf: "123456789",  email: "Jose Augusto@infnet"),
                new Aluno( nome: "Pedro",  cpf: "123456789",  email: "Pedro@infnet"),
                new Aluno( nome: "Ricardo",  cpf: "123456789",  email: "Ricardo@infnet")
        ));
    }
```

Get

```
30      @GetMapping("/alunos")
31      Iterable<Aluno> getAlunos() {
32          return alunoRepository.findAll();
33      }
```

Get por id

```
36      @GetMapping("/alunos/{id}")
37      Optional<Aluno> getAlunosPorId(@PathVariable int id) {
38          return alunoRepository.findById(id);
39      }
```

Delete

```java
@DeleteMapping("/alunos/{id}")
ResponseEntity<Aluno> deleteAluno(@PathVariable int id) {
    if(alunoRepository.existsById(id)) {
        alunoRepository.deleteById(id);
        return ResponseEntity.ok().build();
    }
    return ResponseEntity.notFound().build();
}
```

Delete

```java
@DeleteMapping("/alunos/{id}")
ResponseEntity<Aluno> deleteAluno(@PathVariable int id) {
    if(alunoRepository.existsById(id)) {
        alunoRepository.deleteById(id);
        return ResponseEntity.ok().build();
    }
    return ResponseEntity.notFound().build();
}
```

Post

```
49      @PostMapping("/alunos")
50      ResponseEntity<Aluno> addAluno(@RequestBody Aluno aluno) {
51          //if(!alunoRepository.existsById(aluno.getId()))  aqui nao adianta este teste pois o meu ID é autoincrement
52          if(!aluno.getNome().isEmpty()&&!aluno.getCpf().isEmpty()){ //aqui eu verifico se o nome não é vazio
53              alunoRepository.save(aluno);
54              return ResponseEntity.ok().build();
55          }
56          return ResponseEntity.badRequest().build();
57          //ResponseEntity.status() -aqui retorna o código da chamada no serviço
58      }
```

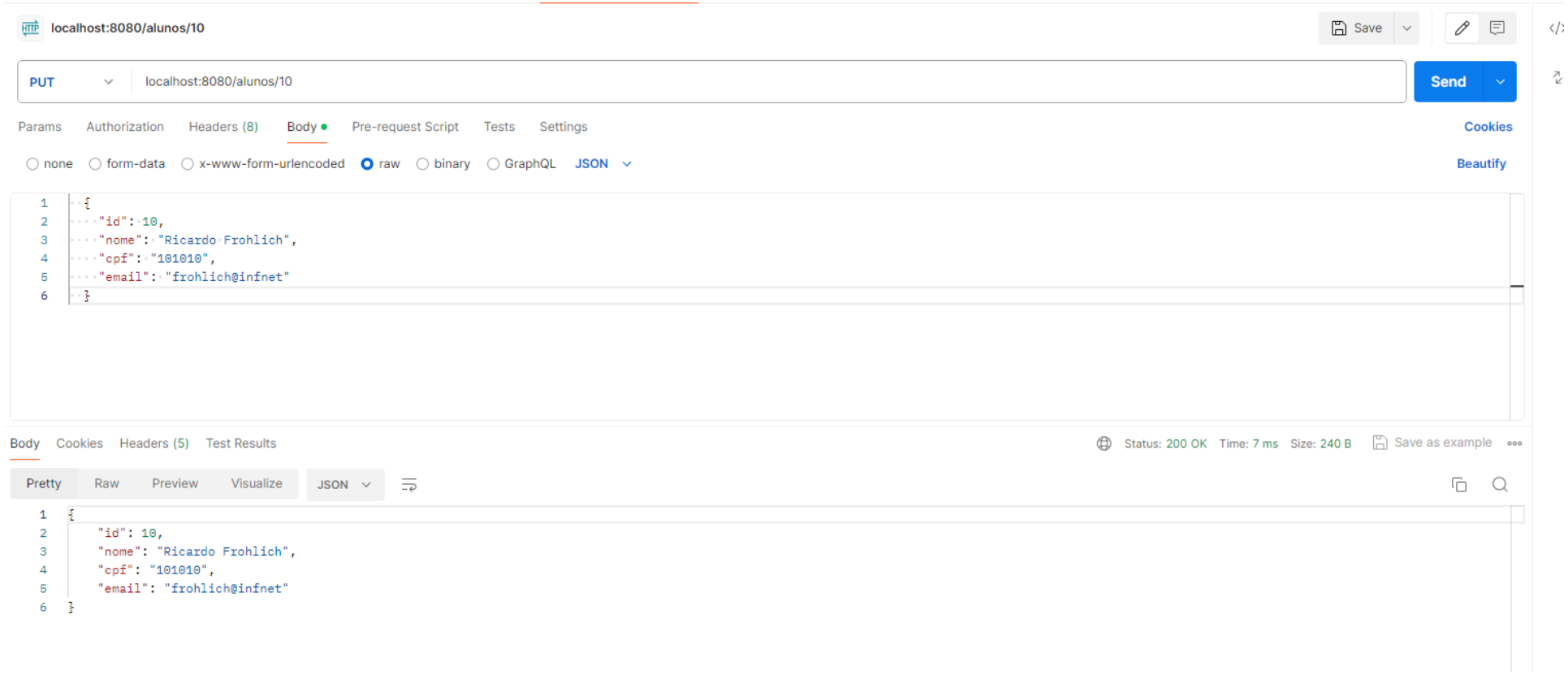
Put

```
61      @PutMapping("/alunos/{id}")
62      ResponseEntity<Aluno> updateAluno(@PathVariable int id, @RequestBody Aluno aluno) {
63        // se o ,id nao existir eu quero inserir
64          //e se o id existi eu vou editar o registro
65          return alunoRepository.findById(id) Optional<Aluno>
66                  .map(existingAluno -> {
67                      aluno.setId(existingAluno.getId());
68                      return ResponseEntity.ok(alunoRepository.save(aluno));
69                  }) Optional<ResponseEntity<...>>
70                  .orElseGet(() -> ResponseEntity.status(HttpStatus.CREATED).body(alunoRepository.save(aluno))); //aqui que como nao encontrou o id no bd, eu insiro por post
71                          //ResponseEntity.notFound().build()); //aqui no momento eu so to criando um erro
72      }
```

Criando uma API simples

```
61      @PutMapping("/alunos/{id}")
62      Aluno putAluno(@PathVariable int id, @RequestBody Aluno aluno){
63          int alunoIndex = -1;
64          for(Aluno a: alunos) {
65              if(a.getId() == id){
66                  alunoIndex = alunos.indexOf(a);
67                  alunos.set(alunoIndex, aluno);
68              }
69          }
70          //return aluno;
71          //melhorando o codigo para caso o aluno nao seja encontrado na lista, eu quero inserir ele
72          return (alunoIndex == -1) ? postAluno(aluno) : aluno;
73      }
```

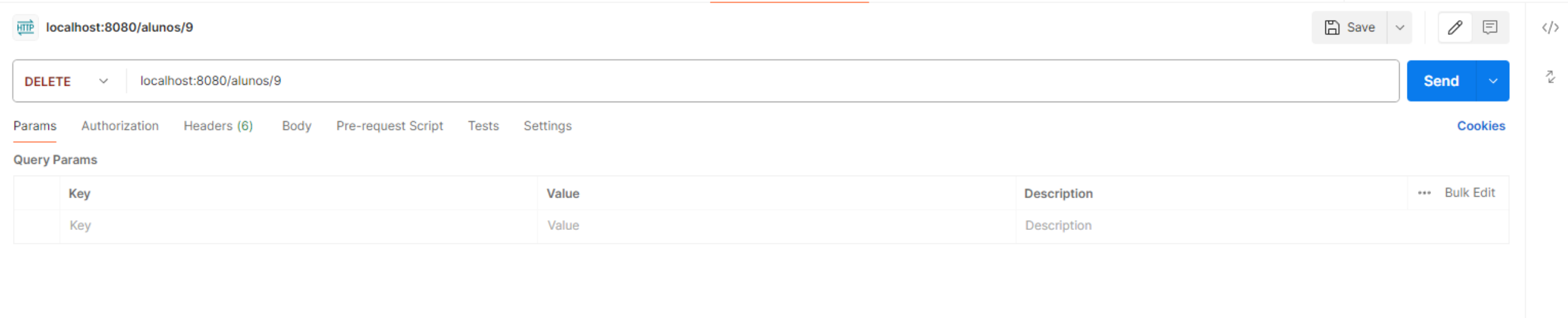
Criando uma API simples



Criando uma API simples

```
75      @DeleteMapping("/alunos/{id}")
76      void deleteAluno(@PathVariable int id) {
77          alunos.removeIf(aluno -> aluno.getId() == id);
78      }
79
```

Criando uma API simples



Atividade

- Desenvolver uma API RESTful usando o Spring Boot para permitir a adição, consulta, edição e exclusão de usuários em uma lista.
  - A classe Usuario deverá ser criada com os seguintes campos (mínimo de 5 campos):
    - Nome
    - Sobrenome
    - Email
    - Idade
    - Gênero
    - (Outros campos opcionais, conforme necessário)